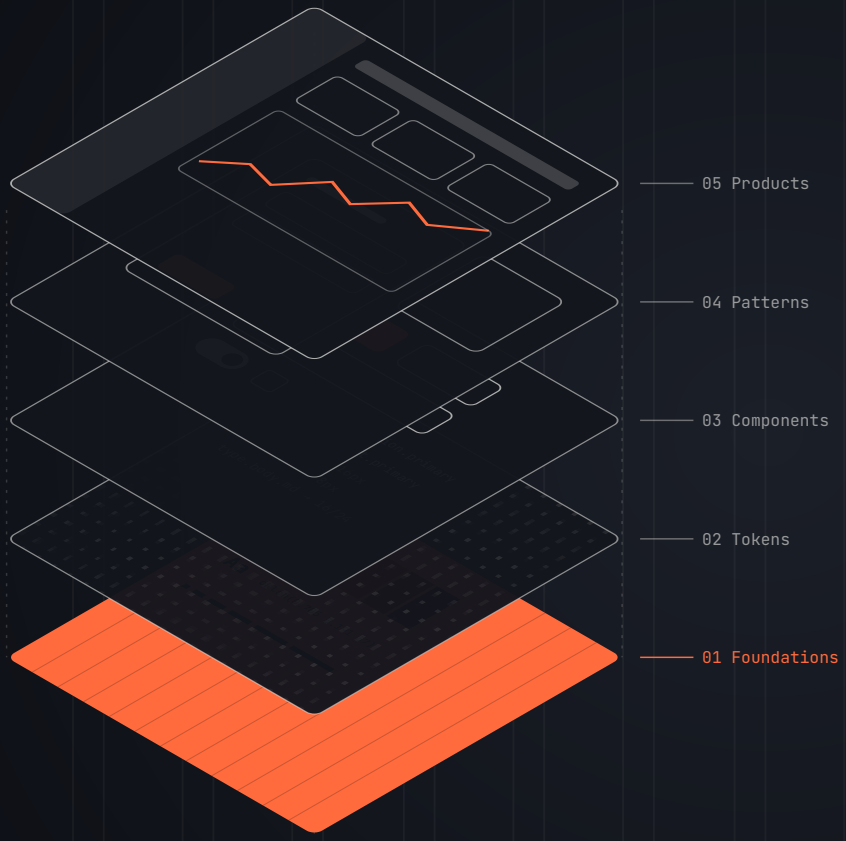


The first layer is the one everything else stands on.

The Design Systems Playbook

A complete, plain-language guide to design systems: why they matter, how to build their foundations, components and patterns, and how to run them for years. For anyone starting from zero.

AUTHOR
Aniket Pandey

PUBLISHED BY
Layer One, Bangalore

WEB
layerone.co.in

YEAR
2026

BEFORE YOU START

A guide for anyone who needs to understand design systems, starting from zero.

Most writing about design systems is either a sales pitch or a technical tutorial. Neither helps a founder decide whether to invest, a product manager understand what their design team is asking for, or a new designer see how all the pieces fit together. This book tries to do exactly that.

We've written it the way we'd explain things across a table: in plain language, with real situations, and with the reasoning behind every recommendation. If you've never worked with a design system, you should be able to follow every page. If you've worked with them for years, we hope you'll still find useful frameworks, honest numbers and a few new angles, especially on the day-to-day work of running a system.

It's written by **Aniket Pandey**, who leads **Layer One**, a design systems studio in Bangalore. Aniket has built design systems at IBM (for clients including Cardinal Health and Riyadh Air), led the design system at CARS24 across 10+ products in three countries, and works on design system tooling and governance at JPMorgan Chase.[1] The book combines that experience with the published work of practitioners such as Nathan Curtis, Brad Frost, Ben Callahan, Dan Mall and Alla Kholmatova, and with the public documentation of mature systems from IBM, Google, Atlassian, Adobe, Salesforce and the UK government.

How sure are we? Four labels

Not everything in this book is equally certain, and we think you deserve to know which is which. Claims that matter carry one of four labels:

STANDARD

A formal standard, like the accessibility guidelines (WCAG 2.2). If you need to comply, it's not optional.

DOCUMENTED

Published research, a case study or official documentation. Always cited, with a warning when the study is small or run by a vendor.

PRACTICE

Our recommendation from practice. Reasoned and tested, but not universal. Disagree if your situation is different.

ILLUSTRATIVE

A worked example built on stated assumptions. It shows a method, not a measured result.

We don't invent statistics, case studies or results. Worked examples use **Ledgerly**, a fictional business finance product, so you never confuse an illustration with a real client. Every colour value, contrast ratio and type size in the figures is calculated by the book's build script, not typed by hand.

EDITION

1.0 — November 2026

The complete edition: five parts, twenty-seven chapters, from strategy to scale.

LICENCE

Free to read and share

Please share it with attribution to Layer One. Please don't resell it or republish it as your own.

GET IN TOUCH

layerone@proton.me

Corrections and disagreements are welcome. We'd rather be right than consistent.

CONTENTS

What's inside

Part 01

Understanding design systems: does your company actually need one?

01 What a design system actually is 9

The five-button problem · A definition you can actually use · Style guide, UI kit, component library, design system · What's inside a complete design system · How it connects design, engineering, product and business · Why it matters more as you grow

02 Who needs a design system, and when 19

The seven questions everyone asks · Where are you today? A five-stage map · A decision tree for your next planning meeting · Signs you've outgrown what you have · Four companies, four different answers · Are you ready? A checklist

03 What a design system is worth 29

Three ways a design system saves you money · What's actually been measured (and what hasn't) · Nine outcomes, and why each one happens · Numbers you can start tracking this quarter · When the value shows up, and when it doesn't

04 The economics of design systems 35

A quick word on the numbers · What makes a design system cheap or expensive · Who you need · How long it takes · Three realistic budgets · A dedicated team, or your existing team? · The cost nobody budgets for: keeping it running · Does it pay off? A simple way to work it out · Start small, and earn the next step

05 Running a design system day to day 47

It's a product, not a project · Who owns it? Four ways to organise · Who decides what goes in? · From "we need this" to "it's in the system" · Contributions: letting product teams help · Shipping changes without breaking anything · Getting people to actually use it · Keeping it healthy for years

Part 02

Building the foundations of a design system

06 Understanding design foundations 60

What foundations are · Why foundations matter: blast radius · From a designer's decision to the product's code

07 Colour systems 66

Why colour goes wrong · How colour is described · Building a palette, step by step · Making sure people can read it · Naming colours by their job · Colours that respond: hover, press and focus · Dark mode, brands and white-labelling · Questions teams always ask about colour

08 Typography systems 83

Choosing a font · Text sizes that work together · Naming text by its job · Text on different screen sizes · Readable for everyone · Different products need different type · Turning text styles into tokens

09 Spacing systems 94

Why spacing matters so much · Picking a base unit · The spacing scale · Space inside things, space between things · Compact and comfortable · Spacing on different screens, and common mistakes

10 Grid and layout systems 102

What a grid is · Where the layout changes: breakpoints · Stretchy, fixed, or both · Laying out a dashboard · Phones, desktops and what lines up with what · Layouts for complex business software

11 How foundations work together 110

One button, five layers · Into the code · What happens when something changes · Are your foundations ready?

Part 03

Components: designing, building and looking after them

12 Components, explained 117

What a component is · Anatomy: the parts of a component · Options: types, sizes and settings · States: one component, many situations · Small parts and big parts · How many components do you need?

13 Deciding what to build first 124

Start with what you already have · Count what matters · Rank it: what to build first · Build, borrow or buy? · Saying "no" and "not yet" · A realistic first year

14 Designing a component that lasts 131

Start from real use, not a blank canvas · Design every state and every edge · Accessible from the start · The words are part of the component · "Ready to build": what a finished design includes · Test it in the real product

15 Building components in code 138

What engineers mean by a component · Keeping design and code in step · Settings or building blocks? · No hard-coded values, ever · How you know a component works · Packaging and shipping components

16 A library people can actually use 146

If people can't find it, they'll rebuild it · Organising the design library · Documentation that actually gets read · A component's life, from idea to retirement · Knowing what's actually being used · When a team needs something different

Part 04

Component architecture in depth

17 Component complexity 155

Not all components are equal · Four levels of complexity · Measuring complexity: a simple scoring method · How complexity changes the way you work · Choosing the simplest component that works · Keeping complexity under control

18 Component architecture 162

What "architecture" means for components · Layers, and the one rule that keeps them healthy · Separating how it works from how it looks · Four ways to put components together · Organising the files and packages · Dependencies, ownership and change

19 Props, in depth 169

Props are a promise · The kinds of props · Designing good props · How props actually work · Props in Figma, and documenting them · Changing props without breaking anyone · Prop patterns to avoid

20 Components and tokens 177

Where components meet tokens · Which parts of a component become tokens · Naming component tokens · When does a component deserve its own tokens? · Tokens in Figma and in code · Keeping tokens and components in sync

21 Components as code 184

A well-built component, line by line · Ways to style components · Accessibility, in code · Choosing the technology · Keeping design and engineering aligned · Quality beneath the surface

22 A casebook of real situations 191

When teams need different things · Migrations: retiring the old and bringing in the new · Changes that could break things · When something goes wrong · Pushing the limits · Saying no, and listening

Part 05

Patterns, content and scale

23 Patterns: from components to solutions 200

Words and sentences · Forms · Tables, lists and finding things · Navigation · Feedback: loading, empty, error and success · Documenting and looking after patterns

24 Content and language in the system 207

Why words belong in the system · Voice and tone · A shared vocabulary · Formulas for common messages · Writing for everyone · Getting ready for other languages

25 Scaling across products, brands and platforms 214

What changes when you scale · Several brands from one system · Several platforms from one source · Families of systems · Governance and funding across business units · Legacy products and long migrations

26 Tools, automation and AI 221

Tools serve the process, not the other way round · The toolchain, mapped · The right tools for your stage · Automation that pays for itself · Where AI helps today · Using AI responsibly

27 Putting it all together: your first twelve months 228

The whole book on one page · A twelve-month plan · The rhythms of a healthy system · The master checklist · Ten ways design systems fail, and the antidote · A closing note

Back

Reference

– References 235

– About Layer One & the author 241

HOW TO READ THIS BOOK

You don't have to read it in order.

Part 01 is about decisions: do you need a design system, what's it worth, what does it cost, and how do you run it? Part 02 covers the foundations everything is built on. Part 03 introduces components, and Part 04 goes deep into how they're structured, configured and coded. Part 05 covers patterns, content, scaling, tools and a twelve-month plan. Pick the route that fits your job.

Founder or CEO

Do we need this, what will it cost, and when?

02 Who needs one
04 Economics
03 What it's worth
27 First twelve months

Product manager

Why does my team want this, and what changes?

01 What it is
05 Running it
13 What to build first
23 Patterns · 27 Plan

Design leader

How do I plan, staff and run it?

02 → 05
13, 16, 17
25 → 27

Designer

How do I build foundations and components well?

06 → 11
12, 14, 16
19, 20, 23, 24

Engineer

What do we build, ship and maintain?

05, 06, 11
15
17 → 22
26 Tools

Things you'll see along the way

SCENARIOS

Grey panels tell a real-world situation in three beats: what was happening, what went wrong, and what a design system changes. They're composites of what we see in practice, not specific clients.

NEW WORDS

When we introduce a term for the first time, you'll find a short plain-English definition in a "New word" band right beside it. You never need to know the jargon in advance.

THE SHORT VERSION

Every chapter ends with a dark summary box of three or four things to remember, so you can skim a chapter's conclusions in thirty seconds.



01

Understanding design systems: does your company actually need one?

Before anyone opens Figma, five questions decide whether a design system will succeed: what it is, whether you need one now, what it's worth, what it costs, and how it will be run. This part answers all five, in plain language.

01 What a design system actually is

02 Who needs one, and when

03 What it's worth

04 The economics

05 Running a design system day to day

CHAPTER 01

01

What a design system actually is

Most people picture a design system as a Figma file full of buttons. That's part of it, but only a small part. Let's start from the beginning, with a situation you've probably already lived through.

READ THIS IF

You keep hearing "design system" and you're not sure everyone means the same thing, including you.

WHAT YOU'LL LEAVE WITH

01 A plain-English definition you can repeat to anyone.

02 The difference between a style guide, a UI kit, a component library and a design system.

03 Why a design system matters more and more as a company grows.

IN THIS CHAPTER

01.1 The five-button problem

01.2 A definition you can actually use

01.3 Style guide, UI kit, component library, design system

01.4 What's inside a complete design system

01.5 How it connects design, engineering, product and business

01.6 Why it matters more as you grow

01.1

The five-button problem

Picture a product team with five designers. Each of them is working on a different part of the app: one on onboarding, one on billing, one on settings, and so on. None of them is careless. Each one designs a good-looking button for their screens.

A year later, someone counts. The app has five slightly different primary buttons. They're all roughly blue. One has rounded corners, one is a bit taller, one uses a bolder font, one turns a darker blue on hover and another turns lighter. Engineering has built each of them separately, because each designer handed over a different spec.

Nobody decided to have five buttons. It just happened, one reasonable decision at a time.

Now the company changes its brand colour. Somebody has to find all five buttons, in every screen, in design files and in code, and update each one. They'll miss a few. The product will look a little off for months. And the same thing has quietly happened to text styles, form fields, spacing, error messages and icons.

SCENARIO **"Why does our app look like three different companies built it?"**

THE SITUATION

A growing SaaS company has three product squads. Each squad has its own designer and its own engineers, and each one moves fast.

WHAT GOES WRONG

Every squad solves the same problems slightly differently: tables, filters, empty screens, error messages. Customers notice. So does the sales team during demos.

WHAT A DESIGN SYSTEM CHANGES

The squads agree on one way to do the common things, build those once, and share them. Each squad spends its time on what makes its part of the product unique.

That, in one story, is why design systems exist. They're not about making things pretty. They're about making sure that many people, working in parallel, end up building *one* product instead of several that happen to share a logo.

01.2

A definition you can actually use

Here's the definition we use with clients. It's deliberately simple:

IN ONE SENTENCE

A design system is a shared set of decisions, reusable parts and working agreements that helps many people build one consistent product, plus the people and process that keep it up to date.

Let's unpack the three ingredients, because most design systems that fail are missing one of them.

01 · DECISIONS

"What we've agreed"

Which colours we use and for what. How big our text is. How much space goes between things. How we write error messages. The reasons behind the look.

02 · REUSABLE PARTS

"What we've built"

Ready-made buttons, inputs, menus and tables, available to designers in their design tool *and* to engineers in code, plus guidance on how to use them.

03 · AGREEMENTS

"How we work together"

Who looks after the system, how people ask for new things, how changes are announced, and what happens when two teams want different things.

Think of it like a restaurant kitchen. The **decisions** are the recipes. The **reusable parts** are the prepared ingredients and the tools on the line. The **agreements** are how the kitchen runs: who orders stock, who changes the menu, how a new dish gets approved. A kitchen with great recipes but no process still serves inconsistent food on a busy night.

When a system is missing one of the three, you can usually spot it:

IF YOU ONLY HAVE…	WHAT HAPPENS
Decisions	You have a beautiful PDF of brand rules that nobody opens while they're working.
Reusable parts	You have a library, but it copies whatever inconsistency already existed, because nobody agreed on the rules first.
Decisions and parts, but no agreements	It works brilliantly for a year. Then it drifts out of date, teams stop trusting it, and they start building their own versions again.

That last row is the most common failure we see. Which is why Layer One's first working rule is:

> ## A design system is an agreement, not a library.
>
> Rule #1 — Layer One

01.3

Style guide, UI kit, component library, design system

These four terms get used as if they mean the same thing. They don't, and mixing them up leads to real budget mistakes. Let's take them one at a time.

NEW WORD

Component A reusable piece of interface, like a button, a text field or a dropdown, that's built once and used everywhere.

A style guide is a set of written rules about how the brand looks: the logo, colours, fonts, photography style and tone of voice. Think of the brand book your marketing agency handed over. It describes what things should look like, but it doesn't build anything.

A UI kit is a file in a design tool like Figma that contains ready-made pieces of interface (buttons, inputs, icons) that designers can drag into their screens. It makes designers faster and more consistent with each other. But engineers can't use it. They still have to rebuild everything in code by looking at the designs.

A component library is the same idea on the engineering side: buttons, inputs and menus written once in code, which engineers import into the product instead of writing their own. It makes engineers faster. But it often doesn't say *when* to use which piece, and it's frequently built separately from the designers' UI kit, so the two slowly drift apart.

A design system connects all of this. The designers' parts and the engineers' parts are the same parts, with the same names, built on the same underlying decisions, with guidance on how to use them and a team that keeps everything in sync.

WHAT IT GIVES YOU	STYLE GUIDE	UI KIT	COMPONENT LIBRARY	DESIGN SYSTEM
Written rules for the look and feel	●	◖	○	●
Ready-made parts for designers	○	●	○	●
Ready-made parts for engineers	○	○	●	●
Design and code use the same names and values	○	◖	◖	●
Guidance on when and why to use each part	◖	○	◖	●
Accessibility built in	○	◖	◖	●
A way to request and announce changes	○	○	◖	●
Someone whose job it is to look after it	◖	◖	◖	●

FIG 1.1 **What each one typically gives you.** ● usually yes, ◖ sometimes, ○ usually not. Notice that the design system column isn't just "more stuff". It's the only one that joins design and code together and has a way of changing over time.

A MISTAKE WE SEE OFTEN

"We have a design system" (it's a Figma file)

A team publishes a polished Figma library and announces a design system. Leadership relaxes; the consistency problem is solved. Six months later, engineering has built its own buttons anyway, because nothing they could actually use in code was ever delivered. A UI kit is a perfectly good first step. Just call it a UI kit, so nobody plans as if the code side is covered.

Which one do you have today?

Try this quick check. Every "no" shows you where your current setup stops.

- [ ] Our colours, text sizes and spacing values have **names** (like "primary-blue" or "space-16"), not just hex codes copied around.
- [ ] Engineers **import** buttons and inputs from a shared package instead of rebuilding them for each feature.
- [ ] A designer and an engineer would use **the same name** for the same thing.
- [ ] There's **written guidance** on when to use each piece, not just what it looks like.
- [ ] Someone is **responsible** for keeping it up to date, and it's part of their actual job.
- [ ] When something changes, **people are told**, and they can choose when to update.
- [ ] People know **how to ask** for something new or suggest a change.

Seven yeses: you're running a design system. One to three: you have a UI kit or a component library, which is a fine place to be, as long as it's a deliberate stage and not where things got stuck.

01.4

What's inside a complete design system

Now let's open the box. A complete design system has a few layers that stack on top of each other, plus some things that run through all of them. The important idea is that each layer is built from the one below it. That's what lets a change at the bottom ripple safely all the way up.

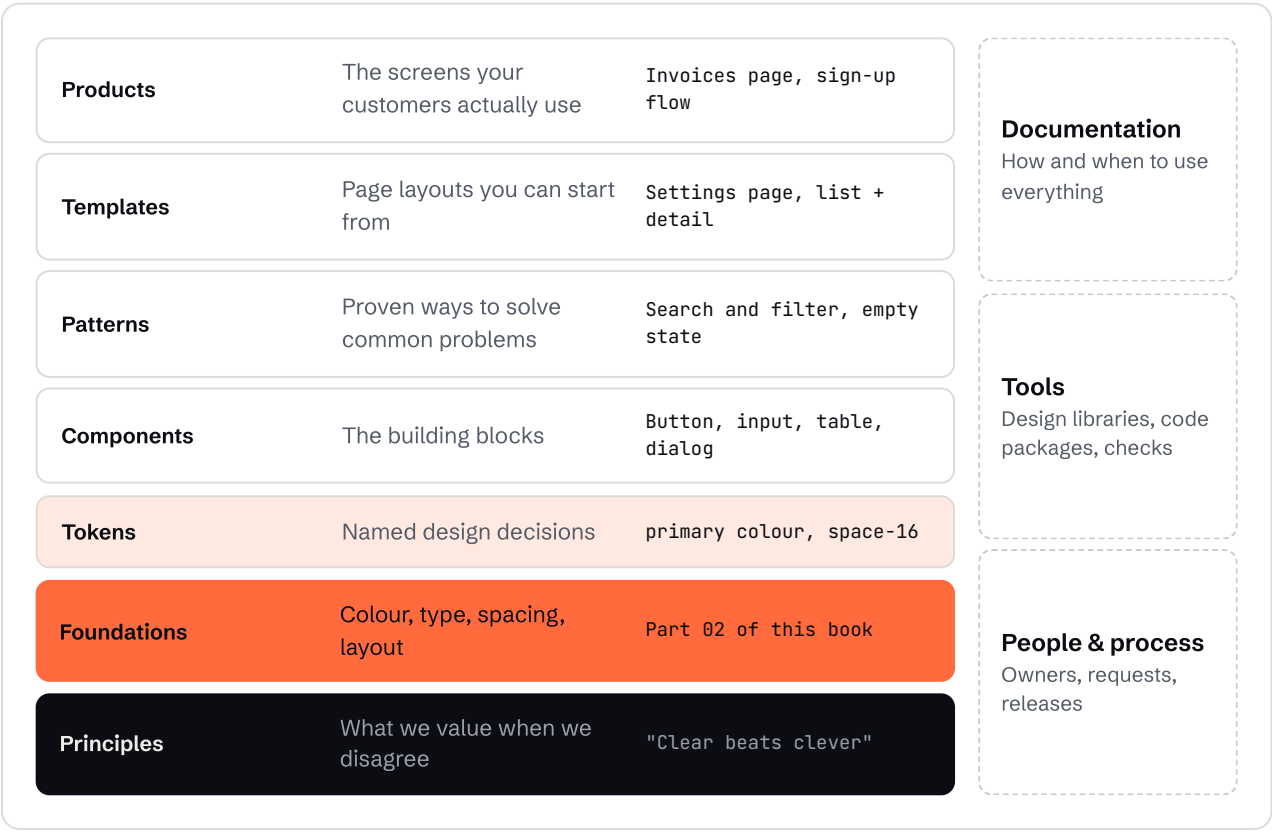


FIG 1.2 The layers of a design system. Read it from the bottom up: everything above is built from what's below. The three boxes on the right run through every layer. They're what turn a pile of parts into something people trust.

NEW WORD

Design token

A design decision with a name, like "primary-blue = #1C75D6". Design tools and code both read from the same list, so they always match.

Two of these layers deserve a plain-English introduction, because you'll meet them throughout the book.

Foundations are the basic visual decisions that everything else inherits: your colours, your text sizes, how much space goes between things, how pages are laid out. Change a foundation and you change everything built on it. Part 02 of this book is all about getting them right.

Tokens are how those decisions get written down so that both designers and engineers can use them. Instead of a designer writing "#1C75D6" on a spec and an engineer copying it into code (and maybe mistyping it), the colour gets a name, like `color-primary`. Design tools and code both read the value from the same place. If the colour ever changes, it changes in one place, and everywhere that uses the name updates.

The idea of building interfaces from small parts up to large ones was popularised by Brad Frost's *Atomic Design*,[2] and the term "design tokens" came out of Salesforce's design system team.[3] In October 2025 an industry working group published the first stable, shared standard for how tokens should be written,[4] which tells you how mainstream the idea has become.

COMPLETE DOESN'T MEAN BIG

A startup's complete design system might be a one-page list of principles, about a hundred tokens and fifteen components. It still has every layer, just a small amount of each. Completeness is about the layers being *connected*, not about how much is in them.

01.5

How it connects design, engineering, product and business

Here's something that surprises most people: the biggest benefit of a design system isn't the components. It's that everyone ends up **speaking the same language**.

Without a system, a designer says "the blue button", a product manager says "the main CTA", and an engineer writes code for "btn-submit-v2". They're all talking about the same thing, and every handover is a small act of translation. Translations go wrong. With a system, there's one "primary button". The designer drags it in, the engineer types `<Button variant="primary">`, and both get exactly the same thing.

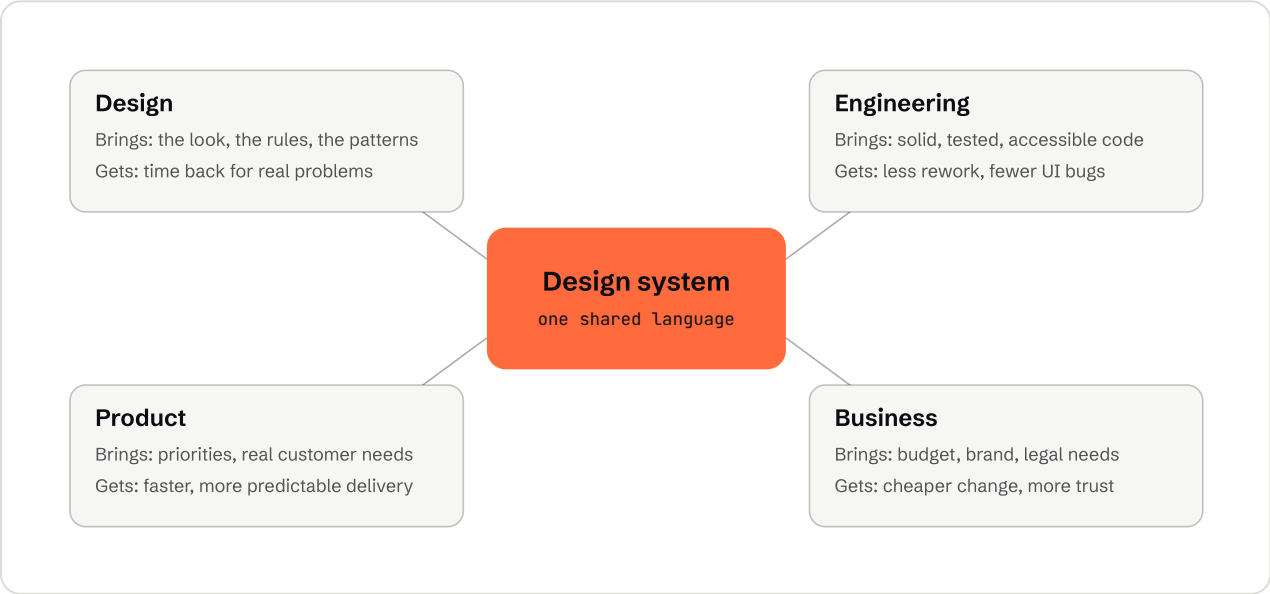


FIG 1.3 **The system sits in the middle of an exchange.** Each group puts something in and gets something back. When a system only takes (asking teams to adopt it without making them faster) or only gives (building things nobody asked for), support drains away.

Here's what that looks like day to day, before and after:

WHO	WITHOUT A SYSTEM	WITH A WORKING SYSTEM
Designers	Redesign the same table, form and empty screen on every project. Spend hours writing handover specs.	Start from known parts and spend their energy on the parts of the problem that are genuinely new.

WHO	WITHOUT A SYSTEM	WITH A WORKING SYSTEM
Engineers	Build "yet another" dropdown. Fix the same keyboard bug in five places.	Import a tested dropdown. Fix a bug once, and the fix reaches every screen on the next update.
Product managers	Estimates swing wildly because nobody knows how much UI work is hiding in a feature.	UI work is predictable, so conversations are about what the feature does, not how many pixels of padding it has.
The business	A rebrand takes quarters. Accessibility problems come back in every audit.	Brand and accessibility changes are made in one place and rolled out.

01.6

Why it matters more as you grow

A two-person team doesn't need a design system to stay consistent. They sit next to each other. So why does it suddenly become essential at thirty, sixty or three hundred people? Two simple reasons, and neither needs any industry statistics to understand.

Reason one: the number of conversations explodes

If every team that makes UI decisions has to stay in sync with every other team, the number of "have you talked to them?" conversations grows much faster than the number of teams. Four teams have six pairs that need to stay aligned. Sixteen teams have 120. This is the same arithmetic Fred Brooks described decades ago for software teams.[5] If instead every team stays in sync with one shared system, sixteen teams need sixteen relationships, not 120.

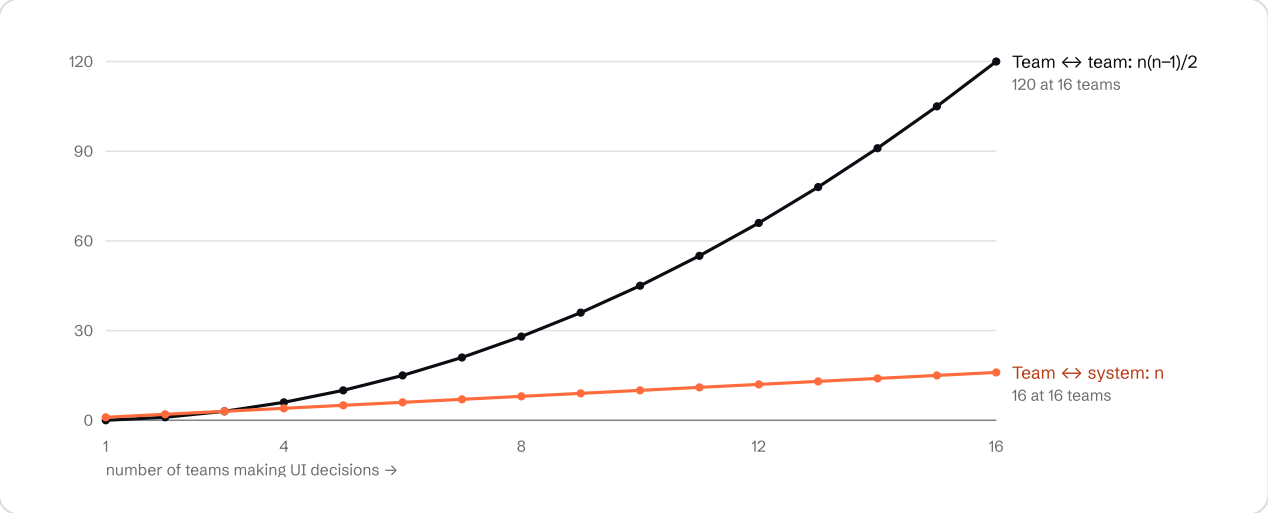


FIG 1.4 How many relationships it takes to stay consistent. ILLUSTRATIVE This is simple arithmetic, not survey data. Real teams don't all talk to each other, but the shape of the curve explains why consistency tends to fall apart somewhere between three and eight teams.

Reason two: every change costs more

Remember the five buttons? Without a system, changing the brand colour means hunting down every place it was typed by hand: roughly *products × platforms × places it appears*. With a system, you change one value and release an update. The same goes for adding dark mode, fixing a contrast problem or refreshing your typography.

That's why the payoff of a design system usually shows up at the *second* product, the *second* platform or the *first* rebrand, not when the first button is built. At CARS24, for example, the work Layer One's founder led put more than ten products in three countries onto one shared colour and token system.[1] Without a shared layer of named decisions, that kind of change would mean editing each product by hand.

PRACTICE A SIMPLE TEST

Judge a design system by what it makes cheap

A good system makes the everyday thing fast (building a normal screen) and the scary thing cheap (changing a decision everywhere at once). If yours does neither, it's a catalogue, not a system.

Ten words you'll meet in this book

Foundations	The basic visual decisions everything inherits: colour, text, spacing and layout.
Token	A design decision with a name, shared by design tools and code (`space-16 = 16px`).
Component	A reusable piece of interface: a button, an input, a dialog.
Variant	An approved version of a component, like a primary or a secondary button.
Pattern	A proven way to solve a common problem using several components, such as how to show "no results found".
Theme	A complete set of values that changes the look everywhere at once, like dark mode or another brand.
Library	The place components live: a Figma library for designers, a code package for engineers.
Adoption	How much of the real, shipped product is actually built with the system.
Contribution	When someone outside the system team adds or improves something in the system.
Release	A new version of the system that teams can update to, with notes on what changed.

THE SHORT VERSION

What to remember from this chapter

01 Inconsistency doesn't come from carelessness. It comes from many good people making reasonable decisions separately.

02 A design system is decisions, reusable parts and working agreements. Missing any one of the three is the most common reason systems fail.

03 A UI kit helps designers, a component library helps engineers, and a design system connects the two with shared names and a way to change.

04 The value grows with size: more teams, more platforms and more change all make a shared system more valuable.

Up next → **Chapter 02: Who needs a design system, and when**

CHAPTER 02

02

Who needs a design system, and when

Here's a secret: every product already has a design system. It's just usually unwritten, inconsistent and living in a few people's heads. The real question isn't "do we need one?" It's "how much of it should we write down, and when?"

READ THIS IF

You're trying to decide whether to invest now, later or not at all, or someone is telling you it's too early.

WHAT YOU'LL LEAVE WITH

01 Straight answers to the seven questions founders and leaders ask most.

02 A simple way to tell where your company is today and what the next step looks like.

03 A decision tree and a checklist you can bring to a planning meeting.

IN THIS CHAPTER

02.1 The seven questions everyone asks

02.2 Where are you today? A five-stage map

02.3 A decision tree for your next planning meeting

02.4 Signs you've outgrown what you have

02.5 Four companies, four different answers

02.6 Are you ready? A checklist

02.1

The seven questions everyone asks

When we sit down with founders and product leaders, the same seven questions come up almost every time. Here are our honest answers, including when the answer would change.

Q

Does every company need a design system?

SHORT ANSWER — **Every company needs consistent design decisions. Not every company needs a formal system with its own team.**

If your product has more than one screen, you've already made decisions about colours, text and spacing. The question is whether those decisions are written down and shared, or re-made by whoever builds the next screen. Two people sitting side by side can stay consistent just by talking. Once decisions are being made by people who don't talk every day, the unwritten system starts to split. PRACTICE

Q

Isn't this only for big enterprises?

SHORT ANSWER — **No. Small companies run them too, just smaller.**

zeroheight's 2025 survey of people who work on design systems found teams averaging about three people at companies with fewer than 100 employees, and about nine at companies with 500 or more.[6] DOCUMENTED (Keep in mind that people who answer a design systems survey usually have one.) What big companies need more of isn't system, it's *coordination*: more teams, more rules about who decides what. The core of a small system and a large one looks surprisingly similar.

Q

Should a startup wait until it finds product-market fit?

SHORT ANSWER — **Wait on the big system. Don't wait on the basics.**

Before product-market fit, your product will change shape, so building a large custom component library is likely wasted effort. But naming your colours, text sizes and spacing takes days, not months, and it actually makes pivots *cheaper*: restyling a product built on named values is a quick change instead of a rebuild. Pair that with a good open-source component library (Radix, MUI and others) styled with your own values, and save custom components for when the product settles down. PRACTICE

Q

We have five or six designers. Is that enough to justify one?

SHORT ANSWER — **Usually yes, but count engineers and products, not just designers.**

Five designers on one product with eight engineers can stay consistent with a good UI kit and a shared list of named values. Five designers supporting forty engineers across a web app, a mobile app and an admin panel cannot. They'll spend half their week answering "which grey is it?" in Slack. What really drives the need is *how many places UI is being built*, and how far apart those places are. PRACTICE

Q

When is a UI kit enough?

SHORT ANSWER — **When designers disagreeing with each other is the main problem, rather than design and code disagreeing.**

A UI kit is often enough if you have one product, on one platform, built in one codebase, by fewer than about three squads, and engineering already uses a well-styled off-the-shelf component library. It stops designs drifting apart. It does nothing to stop code drifting away from design. When that second problem shows up, you've hit the ceiling.

Q

How do we know we've outgrown what we have?

SHORT ANSWER — **When inconsistency starts costing you time in delivery, not just debates in design reviews.**

The checklist in 02.4 is a practical test. If you tick five or more items, you're already paying for a design system. You're just paying for it in rework instead of investment.

Q

When is it too early, or unnecessary?

SHORT ANSWER — **When nobody will own it, nobody will use it, or the product won't live long enough to benefit.**

Hold off if you expect to rebuild your UI from scratch within six months; if one designer and one or two engineers build everything together; if it's a short-lived marketing site; or if engineering leadership hasn't agreed to use it. A system that only designers use is just an expensive UI kit. And be honest about ownership. A system nobody looks after decays faster than having no system at all, because people stop trusting it and start working around it.

02.2

Where are you today? A five-stage map

Design systems grow in stages, like a staircase. The most useful thing you can do is figure out which step you're on and aim for the *next* one, not the top. Jumping straight to the top usually means building approval processes for components nobody uses yet.

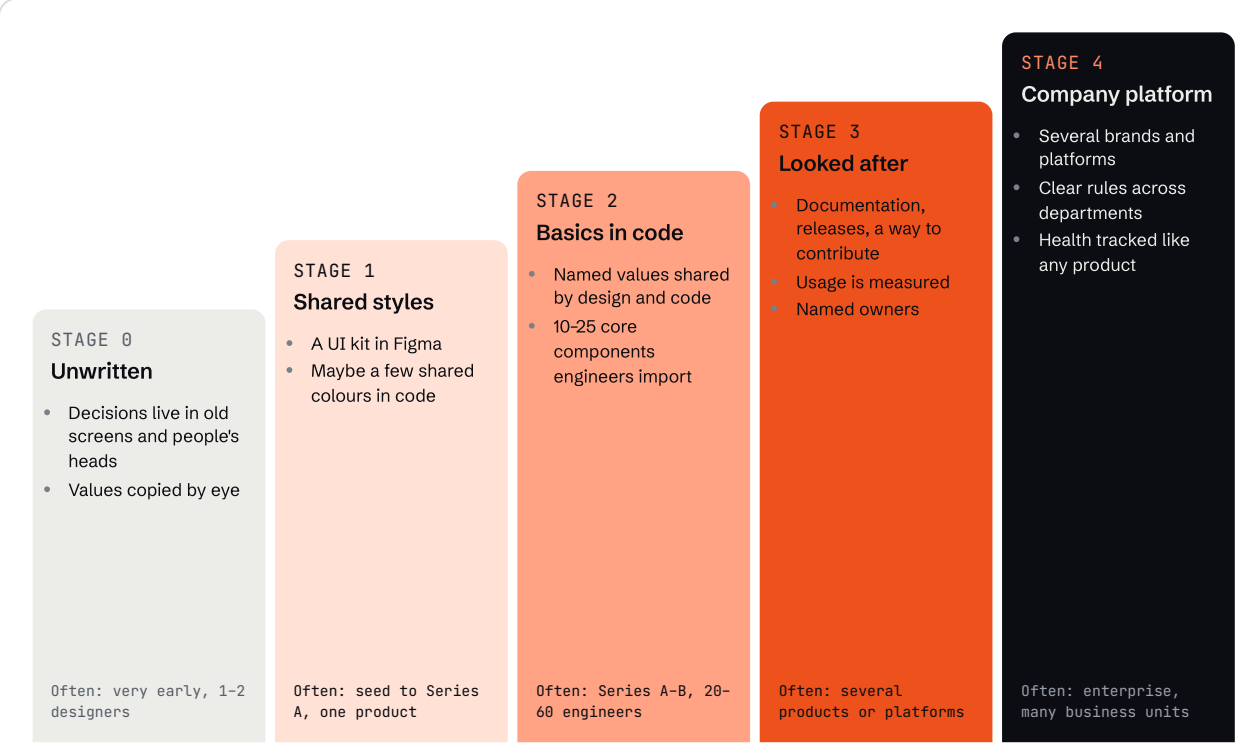


FIG 2.1 Five stages, from unwritten to company platform. PRACTICE The "often" lines are patterns we see, not rules. A 30-person company selling a white-labelled product to banks might need Stage 3; a 300-person company with one simple product may be perfectly served by Stage 2.

STAGE	YOU'LL RECOGNISE IT WHEN…	THE NEXT STEP	WHAT IT TAKES
0 Unwritten	Designers ask each other for hex codes; engineers measure screenshots.	Write down the basics; publish a UI kit.	1–3 weeks of one designer
1 Shared styles	The designs look consistent, but the code is full of colours that "look about right".	Name the values once and share them with code; build the most-used components.	1–3 months, a designer and an engineer, part-time
2 Basics in code	New screens use the package, older ones don't, and nobody owns incoming requests.	Give it owners, documentation, releases and a way to contribute.	A small dedicated team (1.5–3 people)
3 Looked after	Several teams contribute; updates ship on a rhythm; usage is tracked.	Themes, multiple platforms, decision-making across the company.	A dedicated team plus contributors in product teams
4 Platform	The system is treated like infrastructure, with its own roadmap and targets.	Keep it healthy; make change cheaper; automate checks.	An ongoing product budget

These stages describe your *company's* relationship with its design system. Ben Callahan at Sparkbox describes a complementary maturity model for the *system itself*, which starts with "building version one" and then "growing adoption" before the system settles into a long-lived product.[7] The two views fit together well. Whatever stage your company is at, the system goes through a hard early phase where it has to prove it's worth using. Chapter 05 covers that phase in detail.

02.3

A decision tree for your next planning meeting

If you only have ten minutes with your leadership team, walk through these four questions. Notice that they ask about engineering and products before they ask about design. That's on purpose.

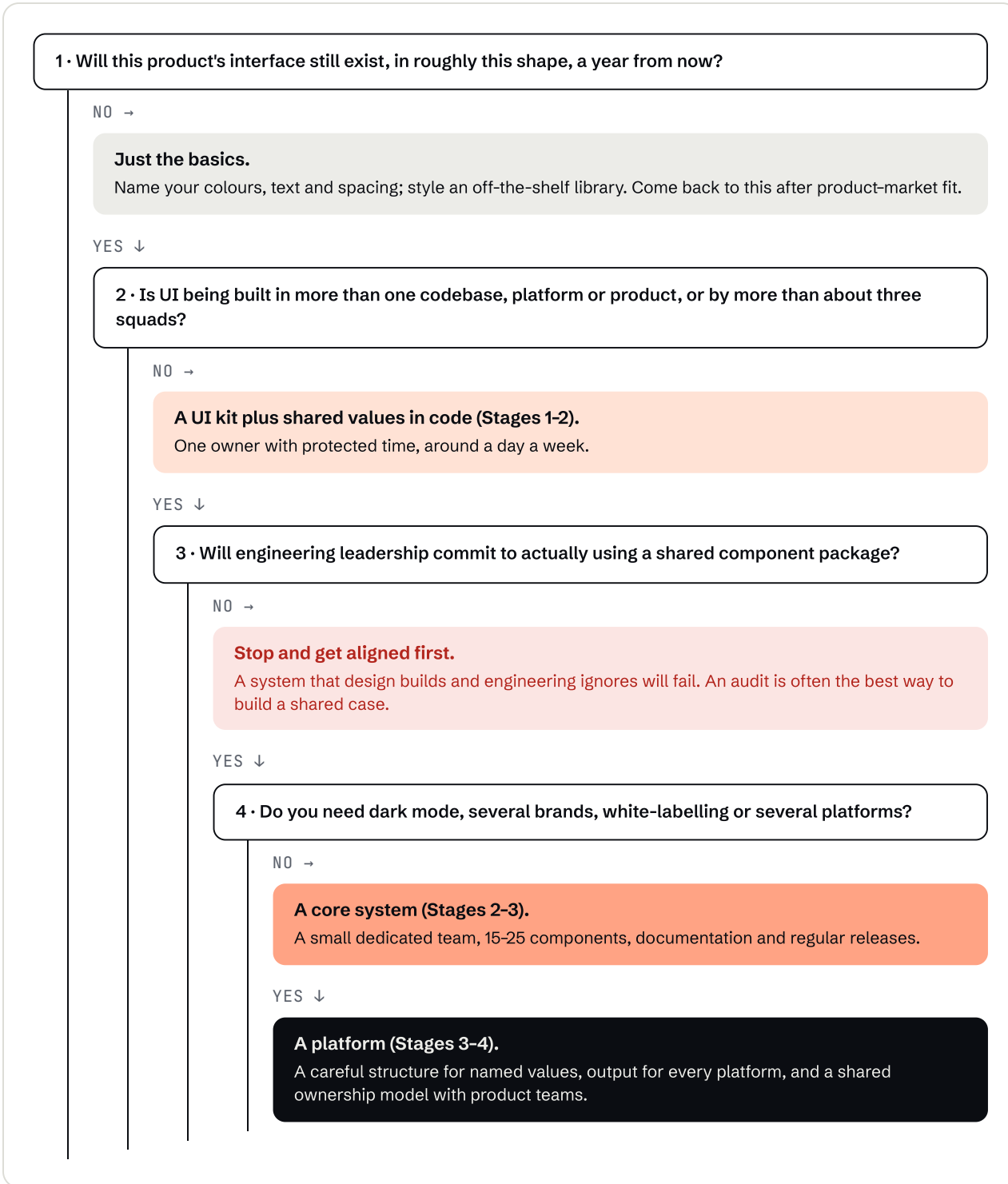


FIG 2.2 Should we invest, and how much? PRACTICE Question 3 is the one teams most often skip, and it's the most common root cause of failed systems we see in audits.

02.4

Signs you've outgrown what you have

You don't need a consultant to tell you whether you have a consistency problem. Most of the evidence is sitting in your code and your calendar. Here's what to look for, and every item can be counted or observed, which is useful because these become your "before" numbers later.

In the code

- [ ] A search for colour codes finds **hundreds of different colours**.
- [ ] There's **more than one button** component (or dialog, or table) in the codebase.
- [ ] The same **accessibility bug** has been fixed in several places.
- [ ] Engineers think a brand colour change would take **weeks**, not hours.
- [ ] Code reviews regularly turn into **debates about spacing** and styling.

In the organisation

- [ ] Design review has become a **bottleneck** before every release.
- [ ] New designers or engineers take **weeks** to learn "how we do UI here".
- [ ] Customers or sales say your products **feel like different companies**.
- [ ] A **second product, platform, brand or acquisition** is on the roadmap.
- [ ] Enterprise customers now ask for **accessibility compliance**.

SCENARIO

The dark mode request that exposed everything

THE SITUATION

A B2B product gets its most-requested feature for the third quarter running: dark mode. The PM estimates two sprints.

WHAT GOES WRONG

Engineering finds 340 different colour values typed directly into the code, many of them nearly identical greys. There's no way to "switch" them, because nothing says which grey is for text and which is for borders. The estimate becomes two quarters.

WHAT A DESIGN SYSTEM CHANGES

With colours named by their job ("text", "border", "background"), dark mode is a second set of values for the same names. It becomes a design task and a testing task, not a rewrite. Chapter 07 shows exactly how.

PRACTICE HOW TO READ YOUR SCORE

0–2 ticks: you're fine for now; write down your basics. **3–5:** you're paying for inconsistency in rework, so plan the next stage. **6 or more:** not having a system is already costing you more than having one would. Start with an audit to see the size of the problem.

02.5

Four companies, four different answers

To make this concrete, here are four situations we see again and again. They're composites, not real clients, but you'll probably recognise one of them. ILLUSTRATIVE

COMPANY A — A fintech startup: 3 designers, 4 engineers, still finding its feet

The founder has seen Stripe's design and wants "a proper design system". The product changes direction every month or two. Everyone sits in one room.

WHAT THEY NEED

The basics only: about a week to name colours, text sizes and spacing, plus a good open-source component library styled to match.

WHY NOT MORE?

Custom components would be redesigned with each pivot. The named values survive pivots; the components wouldn't.

WHEN TO REVISIT

When a second squad forms, or when they start building a mobile app.

COMPANY B — A Series B SaaS: 7 designers, 45 engineers, 2 products

They have their main web app and a recently acquired analytics product built with completely different code. There are three button styles in production, and customers keep asking for dark mode.

WHAT THEY NEED

A two-week audit, then a shared set of named values that can style both products, and 15-20 shared components.

WHO DOES IT

A small core team of about two people, plus a named designer and engineer in each squad who contribute.

THE PAYOFF

Dark mode and visual consistency across both products, and a path to merge the acquired product's UI over time.

COMPANY C — An enterprise with 30+ internal tools built by different vendors

Every tool looks different. Accessibility is now a legal requirement. Nobody owns the interface centrally, and each vendor uses its own technology.

WHAT THEY NEED

Start with named values and accessibility rules, not components. Those can be added to old tools; new components can't be dropped into every old technology.

WHO DOES IT

A central team that sets the rules, with a clear process vendors must follow to contribute and get approval.

THE PAYOFF

A consistent, accessible baseline across tools, without rewriting them all at once.

COMPANY D

A marketing website, rebuilt every two or three years

The marketing team asks for "a design system for the website". An agency builds and maintains it.

WHAT THEY NEED

A style guide and a set of reusable page sections in the CMS.

THE SMART MOVE

Share the same named colours and fonts with the product, so the brand stays aligned between website and app.

WHAT TO AVOID

Funding a product-grade system for a site that will be thrown away in two years.

02.6

Are you ready? A checklist

Run through this before you commit budget. A "no" doesn't mean stop. It means that item becomes the first thing to work on.

- [ ] **A sponsor.** A leader with budget who will defend the system in planning meetings.
- [ ] **Engineering on board.** Engineering leadership agrees teams will use the shared code.
- [ ] **An owner.** A named person with protected time, written into their role.
- [ ] **An audit.** You know what exists today: styles, components, values, accessibility gaps.
- [ ] **A pilot team.** One product team volunteers to build with it first.
- [ ] **"Before" numbers.** You've counted the signs in 02.4, so you can show progress.
- [ ] **The right scope.** You've picked the next stage, not the last one.
- [ ] **Known technology.** You know which platforms and frameworks it has to support.

> ## Audit before you build.
>
> Rule #2 — Layer One

THE SHORT VERSION

What to remember from this chapter

01 Every product has a design system; the choice is how much of it to write down and share.

02 What drives the need is how many places UI gets built, more than how many designers you have.

03 Before product-market fit, name your basic values and use an off-the-shelf library. Hold off on the big custom system.

04 Aim for the next stage, not the final one, and don't start without engineering leadership on board.

Up next → **Chapter 03: What a design system is worth**

CHAPTER 03

03

What a design system is worth

"It'll make us more consistent and efficient" won't get a budget approved. This chapter explains where the value really comes from, what has actually been measured, and how to measure it in your own company.

READ THIS IF

You need to convince a CFO, a board or a sceptical engineering leader, or prove that your current system is earning its keep.

WHAT YOU'LL LEAVE WITH

01 The three ways a design system creates value, in plain terms.

02 An honest view of the published evidence, including where it's weak.

03 A short list of numbers you can start tracking this quarter.

IN THIS CHAPTER

03.1 Three ways a design system saves you money

03.2 What's actually been measured (and what hasn't)

03.3 Nine outcomes, and why each one happens

03.4 Numbers you can start tracking this quarter

03.5 When the value shows up, and when it doesn't

03.1

Three ways a design system saves you money

Every real benefit of a design system comes back to one of three simple ideas. If someone promises you a benefit that doesn't fit one of these, be a little suspicious.

01

Decide once

Teams stop re-deciding the same things: which grey, how much padding, how errors should look. That time goes to the problems that are actually new.

02

Build once

A component is built, tested and made accessible once, then reused everywhere. A bug fixed in it is fixed everywhere.

03

Change once

Because products point to shared parts instead of copies, changing the source changes every product. Rebrands, themes and accessibility fixes become updates, not projects.

Here's an everyday example of each.

Decide once. A designer starts a new settings page. Without a system, she spends the morning choosing toggle styles, spacing and section headings, then an hour in review defending those choices. With a system, those choices already exist. She spends the morning on what's actually hard: how to explain a confusing billing option to customers.

Build once. An engineer finds that dropdown menus can't be closed with the Escape key. Without a system, that bug exists in each team's own dropdown, so it gets fixed (or not) five separate times. With a system, it's fixed once and every team gets the fix with their next update.

Change once. Marketing refreshes the brand colour. Without a system, it's a project touching every product. With a system, it's one change and a release.

The first two save time steadily, a little every day. The third saves time in big lumps, whenever something changes, and over a product's life it's often the biggest saving of all. That's worth remembering when you build your business case: if you only count the first two, you're underselling the system.

03.2

What's actually been measured (and what hasn't)

Let's be straight with you: the published research on design system value is **thinner than the marketing suggests.** Many of the impressive percentages you'll see online trace back to a handful of small studies, several run by companies that sell design tools. That doesn't mean the benefits aren't real. It means you should trust the mechanisms above, and measure the results in your own company rather than borrowing someone else's number.

Here are the three numbers worth knowing, along with how far to trust each one.

34%

faster to complete a design task when designers had a relevant, up-to-date design system.

DOCUMENTED Figma's internal experiment.[8] Run by a tool vendor, with limited detail on the sample.

47%

faster to code a simple form using IBM's Carbon system than from scratch (2 hours vs 4.2).

DOCUMENTED Sparkbox.[9] Only 8 developers, and they always built from scratch first.

79.1%

of the world's top million home pages had text that was too low in contrast to read comfortably.

DOCUMENTED WebAIM Million 2025.[10] A problem a design system can fix at the source.

Why mention the weaknesses? Because the first engineer who reads the Sparkbox study will notice that the developers built the form from scratch first, so by the second attempt they already understood the design. If your business case leans on that 47%, it falls over at the first question. If it leans on the mechanism ("we'll stop building dropdowns five times") plus your own measurements, it holds.

Two more findings help explain why consistency is a business issue, not a cosmetic one. A 1995 study by Kurosu and Kashimura found that people rated better-looking interfaces as easier to use, even when they weren't.[11] And a 2003 Stanford study with more than 2,500 participants found that a website's overall look was the factor people mentioned most when judging whether they could trust it.[12] DOCUMENTED Neither study is about design systems specifically. But they show that a product which looks stitched together pays a real price in trust, and a design system is one of the most reliable ways to avoid looking stitched together.

A MISTAKE WE SEE OFTEN

Borrowing someone else's percentage

"Design systems make teams 47% faster" sounds great in a slide deck and collapses under one good question. Use published studies to show the idea is sound, then build your case on your own "before" numbers (see 03.4). Chapter 04 walks through a cautious way to do this.

03.3

Nine outcomes, and why each one happens

When people say a design system "improves quality" or "speeds up delivery", it helps to know *why*, so you can explain it and later prove it. Here's each outcome with the reason behind it and something you can actually measure.

OUTCOME	WHY IT HAPPENS	HOW YOU'D NOTICE
Faster design work	Designers start from known parts and write fewer handover specs.	Time from brief to ready-for-development design; fewer "how should this look?" questions.
Faster engineering	Standard UI is imported rather than rebuilt; less styling code to write and review.	Shorter time to finish UI-heavy tickets; fewer review comments about styling.
Less duplication	One version of each component instead of one per team.	The number of different dropdowns (or tables, or dialogs) in your code goes down.
Higher quality	Components are tested in every state; named values remove "close enough" colours.	Fewer visual bugs per release; fewer one-off colour values in code.
Accessibility	Contrast, keyboard use and screen-reader support are solved once in the system and inherited.	Fewer automated accessibility errors; fewer findings in audits.
Easier maintenance	A fix or update in the system reaches every product through one update.	Hours spent on a cross-product change, like a colour update.
Better collaboration	Design and engineering share one vocabulary.	Fewer back-and-forth messages to clarify designs.
Faster launches	New products and features start from a working kit instead of a blank page.	Time from project start to first usable version of a new product.
More trust	A coherent, polished product is seen as more usable and more credible.	Customer survey answers about quality; sales feedback from demos.

Accessibility deserves its own line in the budget

Accessibility is where the business case has changed fastest in recent years. The Web Content Accessibility Guidelines (WCAG) 2.2 set out testable rules for things like text contrast, visible keyboard focus and the size of tap targets.[13] STANDARD More and more regulation points to these rules. The European Accessibility Act, for example, applies to many digital products and services sold in the EU from 28 June 2025.[14]

A design system is the cheapest place to meet these rules. When the colour your whole product uses for body text is checked for contrast once, every screen passes. When the system's dialog handles keyboard focus properly, every dialog in every product does. Fixing accessibility screen by screen fixes it once, and only until the next screen gets built.

03.4

Numbers you can start tracking this quarter

Layer One has a rule about this:

> ## Measure time saved, not usage.
>
> Rule #8 — Layer One

Here's what we mean. It's tempting to report "our button has been used 12,000 times!" That tells you the system is being *used*. It doesn't tell you it's *worth it*. Track usage, by all means, because it's useful for the system team, but report outcomes to leadership.

You don't need fancy tooling to start. Most of these can be pulled with a few scripts and your existing project tracker.

WHAT TO TRACK	IN PLAIN WORDS	HOW TO GET IT	HOW OFTEN
Adoption	Of all the screens and files that show UI, what share use the system's components?	A script that scans the code for imports from the system package	Monthly
Hard-coded values	How many colours and spacing values are typed directly into code instead of using names?	A code search or lint rule that counts raw values	Monthly
Duplicates	How many different versions of the same component exist?	Search the code for known component patterns	Quarterly
Time to build UI	How long do UI tickets take, with and without the system?	Your issue tracker, with UI tickets tagged	Quarterly
Visual and accessibility bugs	How many design or accessibility bugs are found after development, per feature?	Bug tracker labels	Each release
Cost of change	How many hours did the last cross-product visual change take?	Log the effort the next time it happens	Each time
Request turnaround	How long from someone asking for something to it being available?	Your request board	Quarterly
Satisfaction	Do designers and engineers agree that "the system makes my work faster"?	A two-question survey	Twice a year

■ PRACTICE THE ONE THING TO DO FIRST

Count before you build

Count your duplicates, hard-coded colours and accessibility errors *before* any system work ships. Without a starting point, every later number is just an opinion. With one, a single slide that says "different colours in our code: 412 → 61" does more for next year's budget than any industry statistic.

03.5

When the value shows up, and when it doesn't

Design systems are a bit like planting a tree. There's a cost up front, the first year looks unimpressive, and the real payoff comes later. Knowing the shape of that curve helps you set honest expectations with leadership.

MONTHS 0–3

New work looks consistent

The basics and the first components arrive. New screens look coherent. The system is still mostly a cost.

MONTHS 3–12

Teams stop rebuilding things

Enough teams use it that standard UI stops being rebuilt; design bugs drop. Savings start to catch up with cost.

MONTHS 12+

Big changes get cheap

Themes, rebrands, new products and accessibility fixes become updates. This is where most systems pay for themselves.

FIG 3.1 How value builds over time. PRACTICE The timing varies with company size and how quickly teams adopt the system, but the shape is consistent: cost first, then steady savings, then large savings whenever something big changes.

Four ways a design system can lose money

It's only fair to say that a badly run system can cost more than it saves. These are the four patterns we see most:

- **Nobody uses it.** A system that only 10% of the product uses has nearly all the cost and almost none of the benefit. Adoption multiplies every outcome above.
- **It tries to do everything.** A button with forty options is slower to use than just building the thing yourself.
- **It's a bottleneck.** If every request waits weeks for the system team, product teams will work around it, and honestly, they'll be right to.
- **It stops changing.** A system that doesn't keep up with the product becomes something teams fight against instead of something that helps them.

All four are problems of *how the system is run*, not of what's in it. That's what Chapter 05 is about.

THE SHORT VERSION

What to remember from this chapter

01 Value comes from three things: deciding once, building once and changing once. The third is often the biggest.

02 The published evidence is real but thin. Use it to show the idea works, not as your headline number.

03 Count your "before" numbers now: duplicates, hard-coded values, accessibility errors.

04 Report time and money saved to leadership, not how many times a button was used.

Up next → **Chapter 04: The economics of design systems**

CHAPTER 04

04

The economics of design systems

"Isn't a design system a huge investment?" It can be, but it doesn't have to be. Spending on a design system works more like a volume knob than an on/off switch. This chapter shows you how to set the knob, and when to turn it up.

READ THIS IF

You need a budget, a staffing plan or a return-on-investment argument, or someone has quoted you a frightening number and you want to know where it came from.

WHAT YOU'LL LEAVE WITH

01 The handful of things that make a design system cheap or expensive.

02 Three realistic budgets, with every assumption out in the open.

03 A simple, honest way to work out whether it pays off for you.

IN THIS CHAPTER

04.1 A quick word on the numbers

04.2 What makes a design system cheap or expensive

04.3 Who you need

04.4 How long it takes

04.5 Three realistic budgets

04.6 A dedicated team, or your existing team?

04.7 The cost nobody budgets for: keeping it running

04.8 Does it pay off? A simple way to work it out

04.9 Start small, and earn the next step

04.1

A quick word on the numbers

NEW WORD	
FTE-month	One person working full-time for one month. "6 FTE-months" could be one person for six months or three people for two.

Salaries vary enormously between countries and cities. A budget written in dollars for San Francisco means nothing in Bangalore, and the other way round. So in this chapter we count effort in **FTE-months** first, which works anywhere, and only then turn it into money using example monthly costs that you should swap for your own.

When we say "monthly cost per person", we mean the *loaded* cost: salary plus everything else a company pays for that person, such as benefits, equipment, office space and taxes. That's typically noticeably more than salary alone.

ILLUSTRATIVE EVERY FIGURE IN THIS CHAPTER IS AN ILLUSTRATION

The numbers here come from clearly stated assumptions, not from measuring real projects. The *method* is the valuable part: change the assumptions to match your company and the answers change with them. The full model is in `scripts/generate-data.mjs` in this playbook's source files.

04.2

What makes a design system cheap or expensive

When two companies get wildly different quotes for "a design system", the difference almost always comes down to the same few factors. The biggest two are platforms and technology: every extra platform (web, iPhone, Android) means building every component again, in a different language, by different engineers.

FACTOR	CHEAPER WHEN…	MORE EXPENSIVE WHEN…	IMPACT
Platforms	You only build for the web	You build for web, iPhone and Android (and maybe desktop)	**Very high**
Technology	Everyone uses the same front-end framework	Some teams use React, some Angular, some an old in-house stack	**Very high**
Brands and themes	One brand, light mode only	Dark mode, several brands, or white-labelled versions for customers	**High**
Existing mess	The product is young	Years of inconsistent UI need to be moved over	**High**
Number of components	15–25 everyday components	60+, including complex ones like data grids and charts	**High**
Accessibility	Built in from the start	Added afterwards to components that weren't designed for it	**Medium**
Documentation	Clear usage notes and code examples	A full public documentation site with tutorials	**Medium**
How spread out teams are	One office, one time zone	Many teams, outside agencies, many time zones	**Medium**
Tooling	Values copied over by hand	Automated pipelines, visual testing, automatic checks	**Medium** (pays back)

This also explains why we always start with named values (tokens). They're the one part of the system that's written once and then automatically turned into code for every platform. A component has to be rebuilt per platform; a colour name doesn't.

04.3

Who you need

A design system is built and run by a small mix of people. In a startup these might all be part-time hats worn by people with other jobs. In an enterprise they're full-time roles.

ROLE	WHAT THEY ACTUALLY DO	WHEN YOU NEED IT
System lead	Decides what gets built next, keeps leadership on side, tracks whether teams are using it. Treats the system like a product.	From Stage 2. Often half of a design or engineering lead's time.
System designer	Designs the foundations and components, keeps the Figma library tidy, writes usage guidance.	From day one
System engineer	Builds components in code, sets up the pipeline that turns named values into code, publishes updates.	From day one. This is the role most often missing.
Mobile engineers	Build the iPhone and Android versions of components from the same named values.	When you have native apps
Accessibility specialist	Checks components against the rules and tests with screen readers.	Part-time from Stage 2
Writer / docs	Organises the documentation and writes guidance on wording and tone.	Part-time from Stage 3
Contributors	Designers and engineers in product teams who suggest and build improvements.	From Stage 3, with time set aside in their team's plans

A MISTAKE WE SEE OFTEN

A design system team made only of designers

You end up with a beautiful Figma library and very little real-world use, because the product is built in code, not in Figma. As a rule of thumb, have at least as much engineering time on the core team as design time, and more once you support several platforms. PRACTICE

04.4

How long it takes

For a single web product, a first usable version usually takes four to six months, and the phases overlap. You don't wait for every colour to be perfect before starting on buttons. Here's what a typical first half-year looks like.

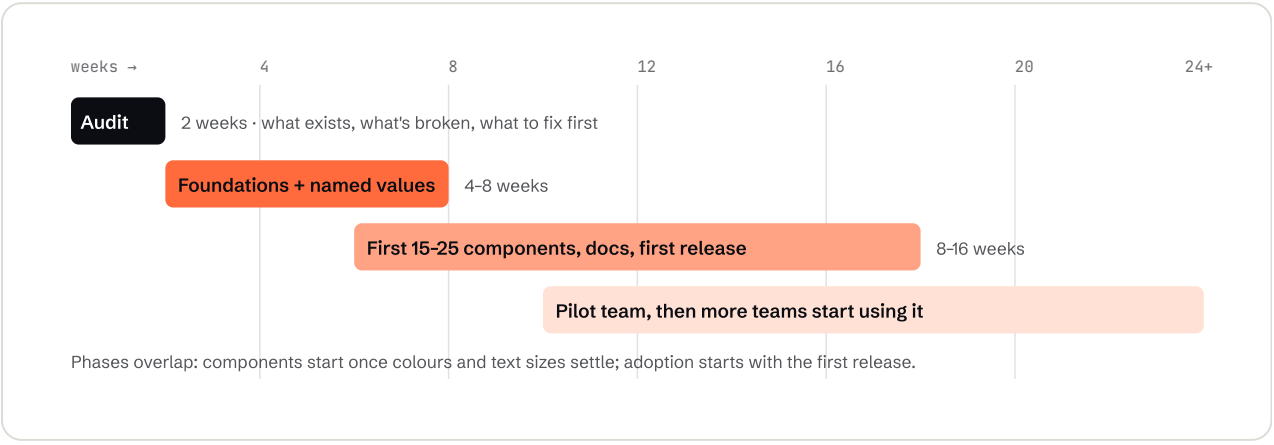


FIG 4.1 A typical first six months for one web product. PRACTICE After the first release, the main job shifts from building to helping teams adopt it. Each extra platform adds component work on top.

04.5

Three realistic budgets

Here's what the first year might look like for three very different companies. Each budget is built from roles and months, then converted to money at three example monthly costs per person: $6,000, $12,000 and $20,000. Those are just example inputs to show the range. Use your own numbers. ILLUSTRATIVE

YEAR-ONE INVESTMENT

SCENARIO	PROFILE	BUILD FTE-MO	RUN FTE-MO	TOTAL	@ $6,000/MO	@ $12,000/MO	@ $20,000/MO
Startup Stage 1 → 2	1 product, web, 2–3 designers, 8–15 engineers	3.0	3.6	**6.6**	$39,600	$79,200	$132,000
Mid-sized SaaS Stage 2 → 3	2–3 products, web + mobile web, 6–12 designers, 40–80 engineers	22.2	22.8	**45.0**	$270,000	$540,000	$900,000
Enterprise Stage 3 → 4	5+ products, web + iOS + Android, multiple brands, 30+ designers, 200+ engineers	108.0	0.0	**108.0**	$648,000	$1,296,000	$2,160,000

HOW EACH TOTAL IS BUILT UP

SCENARIO	PHASE	ROLE	ALLOCATION	FTE-MONTHS
Startup	Build	Design system designer	0.5 FTE × 3 mo	1.5
		Frontend engineer	0.5 FTE × 3 mo	1.5
	Run	Designer (embedded owner)	0.2 FTE × 9 mo	1.8
		Engineer (embedded owner)	0.2 FTE × 9 mo	1.8
Mid-sized SaaS	Build	Design system lead (0.5)	0.5 FTE × 6 mo	3.0
		Design system designer	1 FTE × 6 mo	6.0
		Design system engineers	2 FTE × 6 mo	12.0
		Accessibility review	0.2 FTE × 6 mo	1.2
	Run	Design system lead (0.5)	0.5 FTE × 6 mo	3.0
		Design system designer	1 FTE × 6 mo	6.0
		Design system engineers	1.5 FTE × 6 mo	9.0
		Product-team contributions	0.8 FTE × 6 mo	4.8
Enterprise	Build	Design system lead / PM	1 FTE × 12 mo	12.0
		Design system designers	2 FTE × 12 mo	24.0
		Web engineers	3 FTE × 12 mo	36.0
		iOS + Android engineers	2 FTE × 12 mo	24.0
		Accessibility specialist	0.5 FTE × 12 mo	6.0
		Docs / content	0.5 FTE × 12 mo	6.0

FIG 4.2 Year-one effort for three types of company. ILLUSTRATIVE "Build" is the initial push; "run" is keeping it going for the rest of the year. The startup uses part-time owners rather than new hires. Not included: the time product teams spend switching their screens over to the system, which is often bigger than the system team's own cost and should be planned with each team.

A few things jump out. The startup's system costs less than one engineer for a year, and that buys enough to stay consistent through its next funding round. The mid-sized company needs a small team of two or three people, not a department. And the enterprise budget is driven mostly by platforms: native iPhone and Android work alone is more than a fifth of it.

04.6

A dedicated team, or your existing team?

This is one of the most common questions we get. The honest answer is "it depends on your size", so here's how to think about it.

	OWNED BY AN EXISTING PRODUCT TEAM	A DEDICATED DESIGN SYSTEM TEAM
How it works	A designer and an engineer in a product team get protected time, often one or two days a week, to look after the system.	A team whose whole job is the system, treating the product teams as its customers.
Good at	Cheap, and stays close to real product needs.	Focus, quality, and serving many teams and platforms.
Watch out for	Deadlines eat the protected time; the system quietly bends towards that one team's needs.	Drifting away from real product work; becoming a bottleneck if others can't contribute.
Usually fits	One product, fewer than about 30 engineers.	Two or more products or platforms, roughly 40+ engineers.

For a reality check, zeroheight's 2025 survey found dedicated teams averaging about three people at companies under 100 employees, four at 100–499, and nine at 500 or more.[6] DOCUMENTED Even in very large companies, teams rarely grow past 20 to 25 people. At that size, the extra capacity comes from product teams contributing rather than from a bigger central team.

04.7

The cost nobody budgets for: keeping it running

The most expensive budgeting mistake isn't overspending on the build. It's spending nothing after launch. Picture a team that builds a great system, celebrates the launch, and then gets reassigned to "real" product work. Within a year the components are behind what the products need, teams quietly make their own copies, and people stop using the system. All that investment slowly leaks away.

What "keeping it running" involves

- [ ] Fixing bugs and accessibility problems in components
- [ ] Adding new versions of components as products change
- [ ] Upgrading when the underlying technology moves on
- [ ] Keeping the Figma library tidy and in step with the code
- [ ] Documentation, office hours and onboarding new people
- [ ] Reviewing and accepting contributions from other teams
- [ ] Checking whether teams are actually using it

PRACTICE

PLAN FOR A TEAM, NOT A PERCENTAGE

People often ask, "what percentage of the build cost is maintenance?" We think that's the wrong question. Plan for the people who built the system to stay with it, at a similar or slightly smaller size, with their time shifting from building new things to helping teams use what exists. In our experience, systems whose builders move on right after launch rarely stay in use.

04.8

Does it pay off? A simple way to work it out

Return on investment (ROI) sounds like finance jargon, but the idea is simple: *is the time and money the system saves bigger than what it costs?* There are three pieces to add up.

01 **Time saved from reuse**

How many people build UI, how much of their time goes on UI, how much of that work the system covers, and how much faster it makes them. Multiply those together to get hours, then multiply by an hourly cost.

02 **Big changes made cheaper**

For each planned change, like a rebrand, dark mode or an accessibility clean-up, estimate the cost with and without a system. Only count changes that are actually on the roadmap.

03 **What the system costs**

The system team's cost for the year.

```
// The same thing, written as a formula
hours_saved = people × working_hours × share_of_time_on_UI × adoption × speed_up
value       = hours_saved × hourly_cost + savings_from_planned_big_changes
ROI         = (value - system_cost) ÷ system_cost
```

A worked example: Ledgerly

Ledgerly is the fictional finance product we use throughout this book. It has 40 engineers and 6 designers. We'll assume the system makes work it covers 20% faster. That's deliberately well below the 34% and 47% from Chapter 03, because those came from best-case tasks. We'll let adoption grow over three years, and we'll count *only* time saved from reuse, ignoring every big change, to see the most cautious picture.

YEAR	ADOPTION	HOURS SAVED	VALUE	TEAM COST	NET	ROI
Year 1 Build year: foundations + core components, adoption ramps	40%	1,664 h	$149,760	$288,000	-$138,240	-48%
Year 2 Most new work uses the system	70%	2,912 h	$262,080	$288,000	-$25,920	-9%
Year 3 Migration of older screens largely complete	85%	3,536 h	$318,240	$288,000	$30,240	11%

Assumptions: 40 engineers, 6 designers · 1600 focused h/FTE-year · 25% of engineering and 50% of design time is UI production · efficiency gain on system-covered work 20% · team of 2 FTE at $12,000/FTE-month ($90/h). Excludes change-once events, accessibility remediation and quality effects.

FIG 4.3 Ledgerly with a two-person system team, counting only time saved. ILLUSTRATIVE On reuse alone, the system starts making more than it costs in year three, and takes a little longer to win back the first two years. Any single big change on the roadmap (dark mode, a rebrand, an accessibility programme) usually brings that point forward.

We'll admit that's not the flashy answer people expect, and we think that's exactly why it's useful. It shows that at 40 engineers, a two-person team doesn't pay for itself on speed alone in year one. The case depends on two things: getting teams to adopt the system quickly, and the big changes that are coming. That tells you exactly where to focus.

LEDGERLY — ANNUAL NET VALUE VS A 2-FTE TEAM ($288,000/YR)

EFFICIENCY GAIN ↓ · ADOPTION →	40%	70%	90%
10%	-$213,120	-$156,960	-$119,520
20%	-$138,240	-$25,920	$48,960
34% (Figma figure)	-$33,408	$157,536	$284,832

FIG 4.4 What matters more: speed or adoption? ILLUSTRATIVE Adoption, as it turns out. A system used for 90% of the work with a modest 10% speed-up beats one used for 40% of the work with a 34% speed-up. That's why Chapter 05 treats adoption as the system team's main job.

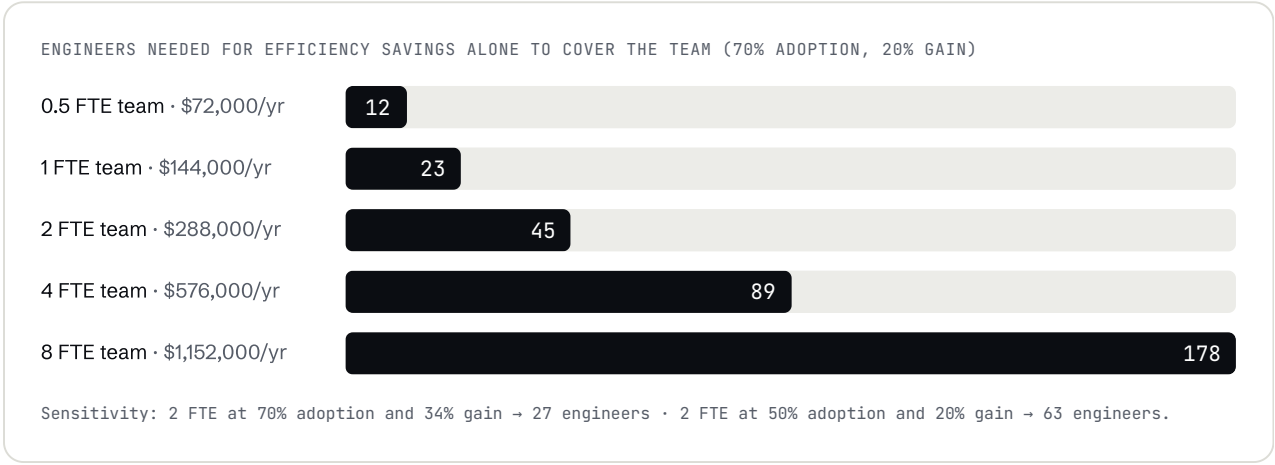


FIG 4.5 How big does the company need to be? ILLUSTRATIVE With these assumptions, each full-time system person needs roughly 22 to 23 engineers' worth of UI work to pay for themselves on reuse alone. It's a handy sanity check: an eight-person system team for 60 engineers needs a very strong reason beyond speed.

04.9

Start small, and earn the next step

You don't have to commit to the whole thing up front. The approach we recommend is to fund the system one step at a time, and let each step earn the next. Each step has a clear "gate": something you can point to that justifies the next round of investment.

01 **Audit — two weeks**

Find out what you have in design and code, and count your duplicates, hard-coded values and accessibility problems. **Gate:** leadership agrees on the top problems.

02 **Foundations and named values — one to two months, part-time**

Colours, text, spacing and layout, named and shared by design and code. **Gate:** one product team ships new work using only the named values.

03 **The ten most-used components — two to three months**

Pick them by how often they appear in your audit, not by which ones are fun to design. **Gate:** the pilot team is measurably faster, or has fewer UI bugs.

04 **Documentation, releases and contribution — ongoing**

Make it usable by people who didn't build it. **Gate:** a second team adopts it without hand-holding, and someone outside the core team contributes something.

05 **Scale up — themes, platforms, governance**

Only now invest in multiple brands, native platforms and formal decision-making. **Gate:** most new work uses the system, and there's a real upcoming change that needs it.

THE SHORT VERSION

What to remember from this chapter

01 Cost is driven mostly by platforms, technology and brands. Named values are the cheapest part and pay back first.

02 Staff the system with engineers as well as designers, and keep the team after launch.

03 Be honest in your ROI: on speed alone, payback takes time; adoption and big upcoming changes make the case.

04 Fund it in steps, and let each step earn the next.

Up next → **Chapter 05: Running a design system day to day**

CHAPTER 05

05

Running a design system day to day

Building a design system is the easy part. Running it is where most of them succeed or fail. Who owns it? Who decides what goes in? What happens when two teams want different things, or an update breaks someone's screen? This chapter is about the everyday operation.

READ THIS IF

You're setting up a system team, your system isn't being used as much as you hoped, or product teams have started calling the system team "the bottleneck".

WHAT YOU'LL LEAVE WITH

01 A clear picture of the ways companies organise design system ownership, and which fits you.

02 A practical process for requests, contributions, updates and breaking changes.

03 Ways to get people using the system, and to keep it healthy for years.

IN THIS CHAPTER

05.1 It's a product, not a project

05.2 Who owns it? Four ways to organise

05.3 Who decides what goes in?

05.4 From "we need this" to "it's in the system"

05.5 Contributions: letting product teams help

05.6 Shipping changes without breaking anything

05.7 Getting people to actually use it

05.8 Keeping it healthy for years

05.1

It's a product, not a project

A project has an end date. You build it, you launch it, you move on. A lot of design systems are treated this way, and that's exactly why they fail. Nathan Curtis, one of the most influential voices in the field, put it in a phrase that has stuck for a decade: a design system isn't a project, it's *a product, serving products*.[15]

What does "product" mean in practice? It means the system has **users**, who are the designers and engineers in your product teams. It has a **roadmap** of what's coming next. It ships **updates** on a regular rhythm. It offers **support** when people get stuck. And somebody checks whether it's actually **working** for the people who use it. Ben Callahan of Sparkbox describes the people who build with a system as its "subscribers", and that's a useful way to think about them. They chose to rely on you. Keep earning it.[7]

PRODUCT HABIT	WHAT IT LOOKS LIKE FOR A DESIGN SYSTEM
Know your users	Regularly sit with product teams. Look at what they build *outside* the system, and ask why.
Keep a roadmap	A public list of what's coming, ordered by how many teams need it and how painful the gap is.
Ship on a rhythm	Regular updates with clear notes on what changed, so teams can plan.
Offer support	A Slack channel with someone answering, weekly office hours, and a short onboarding session for new joiners.
Measure what matters	Is it being used? Is it saving time? Do people like working with it? (See 05.8.)

■ PRACTICE **A SIMPLE TEST**

Ask a designer in a product team: "If the system doesn't have what you need, what do you do?" If they can answer confidently, you have an operating model. If they shrug and say "I just make my own", you have a library.

05.2

Who owns it? Four ways to organise

Every company that runs a design system has to answer one question early: whose job is it? In his 2015 essay on team models, Nathan Curtis described the options that most of the industry still uses today.[16] Here they are, in plain terms.

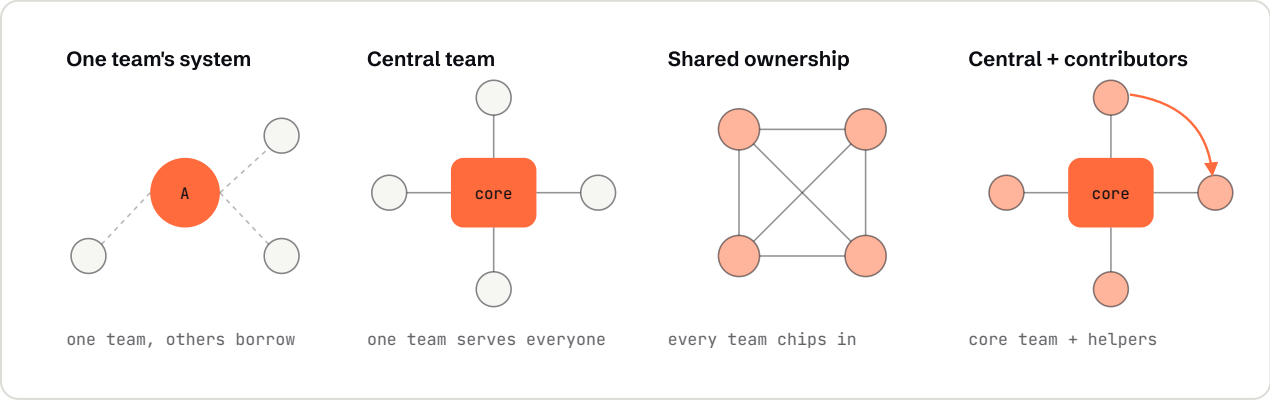


FIG 5.1 **Four ways to organise ownership.** Orange shows who owns the system; tinted circles are product teams that contribute. In the last model, improvements flow from product teams into the core, and updates flow back out. Based on Nathan Curtis's team models.[16]

One team's system (Curtis calls it "solitary"). One product team builds a system for itself, and others borrow it. It's fast and grounded in real work, but it naturally bends towards that one team's needs, so other teams often don't adopt it.

A central team ("centralised"). A dedicated team builds and runs the system for everyone. Quality and focus are high, but the team can drift away from what products actually need, and it can become a queue everyone is stuck in.

Shared ownership ("federated"). Designers and engineers across product teams share the work, with no central team. It sounds fair and efficient. In practice it's often nobody's real job, so it quietly stalls. Curtis himself later concluded that successful systems always have a central team, and also seek participation from a wider community.[17]

Central team plus contributors (a hybrid). A core team owns quality and direction, and product teams contribute improvements back. Salesforce described their version as "cyclical": the core and the contributors constantly feed each other.[18] This is where most mature systems end up.

REAL-WORLD EXAMPLE

Spotify: from 22 systems to one family

WHAT HAPPENED

After Spotify's original central design system team was disbanded, teams started building their own. By the time the company looked again, it had 22 separate design systems.

THE PROBLEM

Each one made sense locally, but together they made the product inconsistent and duplicated a lot of work.

WHAT THEY DID

They created Encore, a "system of systems": shared foundations at the core, with local systems for specific products or platforms on top.[19]

PRACTICE **OUR RECOMMENDATION**

Start with one team or a central team while you get the basics right. As soon as two or more teams depend on the system, move to "central team plus contributors", and do it on purpose: set aside contributor time in product teams' plans instead of hoping people will help in their spare time.

05.3

Who decides what goes in?

Imagine two product teams. The payments team needs a date picker that shows bank holidays. The reporting team needs a date picker that selects ranges of months. Both ask the system team for "a date picker". Who decides what gets built, and does it go into the shared system at all?

This is the everyday reality of running a system, and the answer starts with a simple idea: **not everything belongs in the core**.

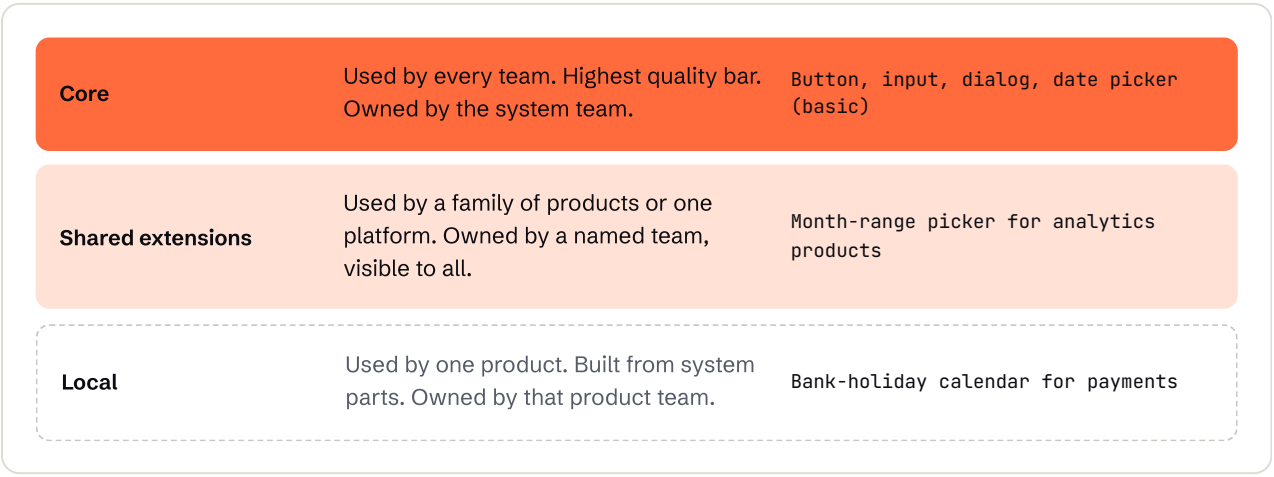


FIG 5.2 Three levels of "belonging". Nathan Curtis calls these tiers: a high-quality core everyone uses, with lower tiers for things that are specific to a product or platform.[20] Brad Frost describes product-specific combinations as "recipes" that live outside the core so it doesn't fill up with one-offs.[21]

In our date picker example, the system team builds (or improves) a basic, accessible date picker in the core. The reporting team builds a month-range picker as an extension, using core parts. The payments team adds bank holidays locally. Everyone gets what they need, the core stays clean, and if a third team later needs month ranges, the extension is a strong candidate to move into the core.

Write down who decides what

Most arguments about design systems are really arguments about who gets to decide. You can prevent most of them by writing it down in advance. Here's a starting point for a central team with contributors:

DECISION	WHO MAKES THE CALL	WHO SHOULD BE ASKED FIRST
Changing a foundation (colours, text sizes)	System lead	Product designers, brand, engineering
Adding a new component to the core	System lead	The teams that asked for it
Adding a variation to an existing component	System designer	The requesting team
How a component works in code, and changes that would break things	System engineers	Product engineers

DECISION	WHO MAKES THE CALL	WHO SHOULD BE ASKED FIRST
Whether a component meets accessibility rules	System lead (with an accessibility specialist)	—
Building a local, product-only component	**The product team**	System designer (a quick heads-up)
Removing an old component	System lead	Every team still using it
The system's roadmap and budget	Design and engineering leadership	System lead, product teams

Look at the sixth row. Product teams are allowed to build their own components when the system doesn't cover what they need. A healthy system doesn't forbid that. It just asks teams to say so, so the system can learn from it.

05.4

From "we need this" to "it's in the system"

Let's follow a real request through the system. Priya is a product designer on the billing team. She's designing a new invoice screen and needs a way to show a short list of key facts: amount, due date, status. Nothing in the system quite fits.

What happens next decides whether Priya trusts the system or quietly gives up on it. Brad Frost has described a governance process that many teams use as a starting point,[22] and here's our version of it, in plain steps.

01 **Check the system first**

Priya searches the system and its documentation. If something fits, she uses it. Done.

02 **Talk to the system team early**

Nothing fits, so she posts in the system's Slack channel with a screenshot and a sentence about the problem. Not a finished design, just the need. Early is cheap; late is expensive.

03 **Decide: one-off or shared?**

The system designer replies within a day or two. Have other teams needed this? It turns out the subscriptions team built something very similar last month. Two teams need it, so it's a candidate for the system.

04 **Agree what "good" looks like**

They agree on a rough scope together: which states it needs, how it behaves on small screens, and accessibility requirements.

05 **Build it once, properly**

Either the system team builds it, or Priya's team builds it with the system team's help (that's a contribution; see 05.5). Either way it gets designed for every state, built in code, tested and documented.

06 **Review, release and announce**

A final check, then it ships in the next update with a short note: what it is, when to use it, and a thank-you to Priya's team.

07 **Teams pick it up**

Billing and subscriptions both update and use the shared version. The subscriptions team's one-off copy is retired.

Meanwhile, Priya doesn't wait. If her deadline is before step 6, she builds a local version from system parts and swaps it for the official one later. The system should never be the reason a product team misses a deadline.

How do you decide what makes the cut?

The UK government's GOV.UK Design System publishes its criteria openly, and they're a great model. To be accepted, a proposed component or pattern must be **useful** (there's evidence other teams need it) and **unique** (it doesn't duplicate something that exists). Before it's published, it's checked to be **usable** (tested with real users, including disabled users), **consistent** (built from existing styles and parts) and **versatile** (works across browsers, devices and assistive technology).[23] DOCUMENTED You can adopt those five words almost as they are.

> PRACTICE **THE RULE OF TWO**
>
> Our simplest test: something goes into the core when **at least two teams** need it. Until then, it lives locally with the team that built it, made from system parts so it's easy to promote later. This keeps the core free of one-offs and gives the system a steady supply of tried-and-tested candidates.

05.5

Contributions: letting product teams help

A central team of three can't keep up with the needs of fifteen product teams. Contribution, meaning product teams improving the system themselves, is how small system teams serve big organisations. But it only works if contributing is easy and the effort matches the size of the change.

Nathan Curtis makes a useful distinction between **small** and **large** contributions.[24] Small ones are things like fixing a bug, improving an accessibility issue or correcting documentation: changes that are clearly improvements, which should go through a normal, quick review. Large ones change how something works or add something new, and they deserve a proper proposal and discussion before anyone builds them.



FIG 5.3 Four sizes of contribution. PRACTICE Effort scales with the size of the change. The most common mistake is putting a two-line bug fix through a month-long approval process, which teaches people never to contribute again.

SCENARIO

The engineer who fixed a bug and never contributed again

THE SITUATION

Arjun, a product engineer, finds that the system's dropdown can't be closed with the Escape key. He fixes it in an afternoon and submits the change.

WHAT GOES WRONG

The change sits for three weeks waiting for a design review and a "contribution committee" meeting. By the time it's merged, Arjun has worked around it in his own code. He doesn't bother next time.

WHAT SHOULD HAPPEN

A clear bug fix is a small contribution: one engineer reviews it within a few days, it ships in the next patch, and Arjun gets a thank-you in the release notes.

Proposals for bigger changes

NEW WORD	
RFC	"Request for comments": a short written proposal shared for feedback before a big decision is made.

For larger changes, many teams use a short written proposal, often called an RFC. Keep it to a page: what problem are we solving, what are the options, what do we recommend, and what would teams have to change? Share it for comments for a fixed time, say a week, and then the person responsible decides. Written proposals sound bureaucratic, but they're actually faster than meetings, because people can comment when it suits them and the reasoning is saved for later.

One last thought from Curtis that's worth holding on to: he has said he dislikes the word "contribution" because it makes teams focus on the *output* (a pull request) instead of the *outcome* (a better system for everyone).[25] The point isn't to collect contributions. It's to make the system better and to make product teams feel it's theirs.

05.6

Shipping changes without breaking anything

Here's a situation that happens in almost every design system eventually. The system team decides buttons should have slightly more rounded corners and a little more padding. It's a small visual tweak. They ship it on a Friday. On Monday, three product teams report broken screens: buttons no longer fit in a table row, a toolbar wraps onto two lines, and a checkout button now pushes the price off the screen on small phones.

Nothing in the code "broke". But the products did. Nathan Curtis wrote a whole article about exactly this: changes to colour, size and spacing can break a product as surely as a change to the code.[26]

Version numbers, in plain words

NEW WORD

Breaking change

An update that could make something stop working or look wrong in a product, so teams need to check or change their own work.

Most systems label their updates with a three-part version number, like `4.2.1`, following a widely used convention called semantic versioning.[27] It's simpler than it sounds. Each number tells teams how careful they need to be when updating:

WHICH NUMBER CHANGES	WHAT IT MEANS FOR TEAMS	DESIGN SYSTEM EXAMPLES
First (4 → 5)	"Careful: you'll need to check or change your work."	A component renamed or removed; a named colour removed; button sizes changed in a way that affects layouts.
Middle (4.2 → 4.3)	"New things added; nothing you use should change."	A new component, a new variation, a new icon.
Last (4.2.1 → 4.2.2)	"Fixes only. Safe to update."	A bug fix; an accessibility fix that doesn't change size or layout; a docs correction.

Curtis's series on releasing design systems goes into this in depth: how often to release, how to number versions, and how to handle breaking changes.[28] The headline advice is simple. Decide in advance which kinds of *visual* change count as breaking, write it down, and stick to it. In our button story, a change to padding that affects layouts should have been a "first number" release, announced in advance, with screenshots and a test period.

Retiring old things gracefully

NEW WORD

Deprecation

Marking something as "old, please stop using it" before actually removing it, so teams have time to switch.

Sooner or later you'll need to replace a component. Doing it suddenly breaks things; never doing it leaves a mess. The healthy middle path looks like this. First, mark the old component as deprecated: it still works, but the documentation and a warning in the code say "please use the new one". Then give teams a guide to switching, and, where the change is mechanical, an automated script to do it for them. Track who's still using the old version. Only remove it in the next "first number" release, after a notice period everyone knows about in advance.

Telling people what changed

Release notes are the system's most-read documentation, so write them for the people using the system, not for the people who built it. Compare "Refactored Button padding tokens" with "Buttons are now 4px taller. If you've placed buttons inside table rows, check they still fit." The second one actually helps someone. Add before-and-after screenshots for anything visual. For bigger releases, a ten-minute demo in a team meeting goes a long way.

05.7

Getting people to actually use it

Here's an uncomfortable truth: a design system nobody uses is just an expensive side project. Adoption is the system team's most important job, and Ben Callahan's maturity model puts "growing adoption" right after "building version one" for good reason.[7] It's the stage where many systems stall.

The good news is that adoption is rarely about persuasion. It's about making the system the *easiest* way to get work done. Teams take the path of least resistance. Your job is to make sure that path runs through the system.

Start with a pilot

Dan Mall, in his book *Design That Scales*, recommends building the system through **pilots**: real product projects where the system is used, tested and improved at the same time as the product work gets done.[29] Like a TV pilot, it's a low-risk way to find out what works before you roll it out widely. It also gives you your first success story: a real team that shipped faster, which is far more convincing than any slide.

SCENARIO **"I couldn't find it, so I made my own"**

THE SITUATION

A new designer joins and needs a warning banner. The Figma library has 400 components with names like "Alert_v2_final" and "Banner/New/Warning (old)".

WHAT GOES WRONG

She can't tell which one is current, so she draws her own. Engineering builds it from scratch. The product now has a fourth kind of warning.

WHAT FIXES IT

Clear names, removing or hiding old versions, a search-friendly description on every component, and a "start here" page. Findability is adoption. Part 03 covers library organisation in detail.

What works

Great documentation

Teams use what they can understand in five minutes before a deadline. Layer One's rule: *docs decide adoption.*

Champions in each team

One designer and one engineer per product team who get early previews and a direct line to the system team.

Office hours

A weekly slot where anyone can bring a screen and get help building it with the system.

Starter templates

New projects begin with the system already set up, so using it is the default, not an extra step.

Automatic nudges

Code checks that flag hard-coded colours or old components during code review.

Share the credit

Celebrate contributors and pilot teams in release notes and all-hands meetings.

A MISTAKE WE SEE OFTEN

Mandating adoption before earning it

"From next quarter, all teams must use the design system." If the system isn't ready to cover their needs, teams will comply on paper and work around it in practice, and they'll resent it. Make it genuinely useful first; mandates work best when they confirm something teams already want.

05.8

Keeping it healthy for years

Design systems rarely die suddenly. They fade. Updates get less frequent, requests pile up, the Figma library and the code drift apart, and one day someone says "we should build a new design system", and the cycle starts again.

You can see it coming if you check a few simple signals regularly. Review them with the team monthly, and with leadership every quarter. Look at which way they're moving, not at hitting exact targets.

SIGNAL	WHAT TO LOOK AT	HEALTHY	WARNING SIGN
Usage	Share of UI built with the system; how many products are on a recent version	Rising; most products within one major version	Flat; products stuck on old versions
Coverage	Hard-coded values in code; local components per product	Hard-coded values falling	Local components growing faster than the core
Quality	Open bugs; accessibility errors	Bugs fixed faster than they arrive	The same kind of bug keeps coming back
Speed	Time from request to release; number of contributions	Days to weeks; contributions growing	Months; contributions dropping to zero
Feeling	"The system makes my work faster" (% who agree)	Most agree, and rising	People call it a blocker

Tidy as you go

NEW WORD

Design debt — Shortcuts and inconsistencies that pile up over time and make future work slower, like clutter in a garage.

Every system builds up clutter: components that no longer quite fit, workarounds that became permanent, copies that teams made and modified, old versions still in use, and places where the Figma library and the code no longer match. That's normal. The problem is clutter nobody can see. Keep a simple, visible list of it (a "debt register"), and set aside a fixed slice of every cycle, around a fifth of the team's time, to clear some of it. PRACTICE Teams that tidy a little all the time rarely need a big rewrite.

Watch your own capacity

Finally, look after the team. In zeroheight's 2025 survey, 41% of respondents said they didn't have enough people to run their system, and that was most common at smaller companies.[6] DOCUMENTED An overstretched system team becomes slow, a slow team becomes a bottleneck, and a bottleneck gets worked around. Protecting the team's capacity is part of protecting the system.

Operating checklist

- [ ] Ownership model chosen and written down
- [ ] Who-decides-what table agreed (05.3)
- [ ] A clear path from request to release (05.4)
- [ ] Contribution levels published, with quick reviews for small fixes
- [ ] Core, extensions and local levels defined
- [ ] Versioning rules, including which visual changes are "breaking"
- [ ] A deprecation process with a notice period
- [ ] Release notes written for the people using the system
- [ ] A pilot team and champions in each product team
- [ ] A monthly health check and a visible debt list

> **Leave the team able to run it without us.**
>
> Rule #10 — Layer One

THE SHORT VERSION

What to remember from this chapter

01 Treat the system as a product with users, a roadmap, releases and support, not a project with an end date.

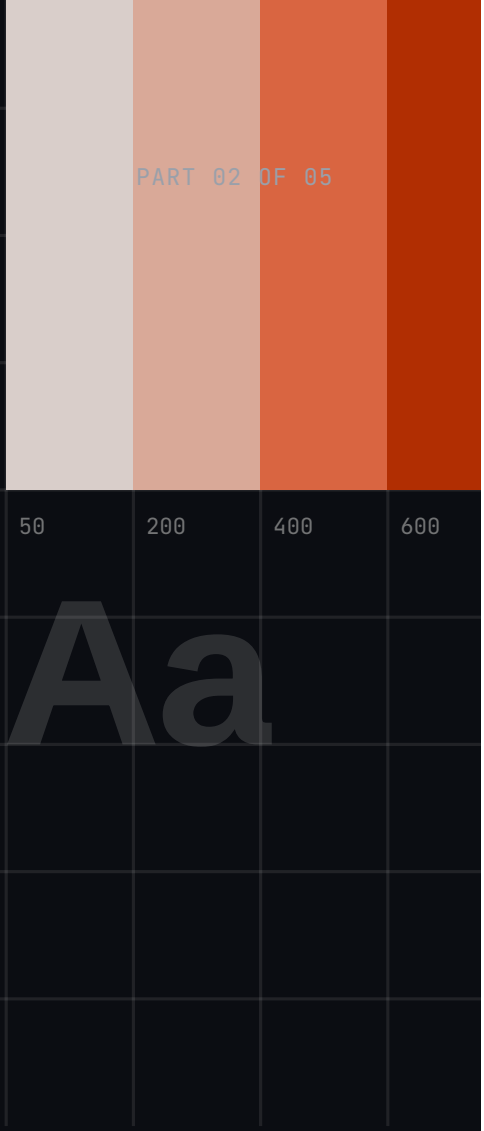
02 Most mature systems end up with a central team plus contributors from product teams. Plan contributor time on purpose.

03 Not everything belongs in the core. Use levels (core, extensions, local) and the rule of two.

04 Decide in advance which visual changes count as breaking, and announce changes in plain language.

05 Adoption is the main job: pilot, document well, and make the system the easiest path.

Up next → **Part 02: Building the foundations**



02

Building the foundations of a design system

Colour, text, spacing and layout are the decisions every button and every screen inherits. Get them right and everything built on top gets easier. Get them wrong and every component carries the mistake. This part walks through each one, step by step, and shows how a decision becomes working code.

06 Foundations architecture

07 Colour

08 Typography

09 Spacing

10 Grid and layout

11 How foundations work together

CHAPTER 06

06

Understanding design foundations

Before buttons and menus, there's a smaller set of decisions that everything else inherits: colours, text, spacing and layout. They're the cheapest part of a design system to set up, and the most expensive part to get wrong.

READ THIS IF

You're about to decide your product's colours, type or spacing, or you want to understand how a design decision turns into code without anyone retyping it.

WHAT YOU'LL LEAVE WITH

01 What counts as a "foundation", and what doesn't.

02 Why changing a foundation is a big deal, explained through a simple idea called blast radius.

03 How a decision travels from a designer's head into the product's code.

IN THIS CHAPTER

06.1 What foundations are

06.2 Why foundations matter: blast radius

06.3 From a designer's decision to the product's code

06.1

What foundations are

Think of building a house. Before anyone chooses kitchen cabinets, someone has decided the ceiling height, the size of a standard door and where the plumbing runs. Every room inherits those decisions. Get them right and everything fits. Get them wrong and every room fights the house.

Foundations are the same thing for a digital product. They're the basic visual decisions that every button, card and page inherits. They aren't components themselves, but every component is built out of them. Different design systems give them slightly different names (IBM's Carbon calls them "elements"; Google's Material calls them "styles" and "foundations"), but the list is remarkably consistent.[30][31]

CHAPTER 07

Colour

Which colours you use, for what, and how they work in dark mode.

CHAPTER 08

Typography

Fonts, text sizes, and how headings and body text relate.

CHAPTER 09

Spacing

How much room goes between and inside things.

CHAPTER 10

Layout

Grids, screen sizes and how pages are arranged.

ALSO

Elevation

Shadows, and what sits on top of what.

ALSO

Shape

How rounded corners are, and how thick borders are.

ALSO

Motion

How long animations take and how they move.

ALSO

Icons

Sizes, line thickness and style.

This edition covers the four with the biggest effect on how good your product feels: colour, typography, spacing and layout. The others follow exactly the same thinking.

06.2

Why foundations matter: blast radius

Here's a useful way to think about any design decision: how many places does it affect? We call that its **blast radius**. Changing the label on one button affects one button. Changing the colour used for all secondary text affects every screen in every product. Foundations have the biggest blast radius of anything in a design system.

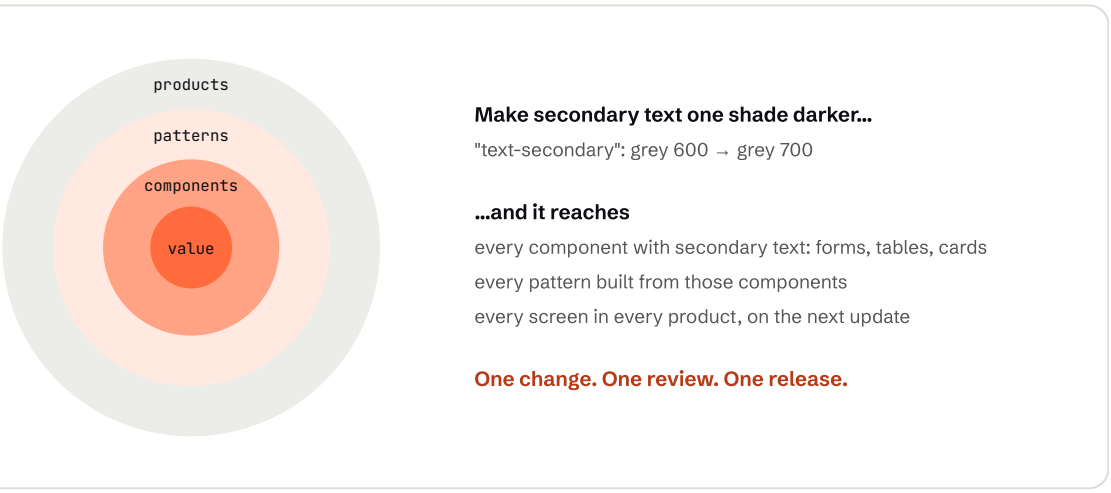


FIG 6.1 Blast radius. Here, a readability fix made once at the foundation level reaches every product. The same mechanism spreads a bad decision just as efficiently, which is why foundations deserve the most careful thought in the whole system.

That double edge is the whole point. A wide blast radius is what makes foundations powerful: fix a contrast problem once and it's fixed everywhere. It's also what makes them risky: pick a text size that's slightly too small and every screen inherits it. Four things in particular are decided at this level:

Quality	Whether text is readable and whether headings stand out from body text is decided here. Muddy text sizes make every screen muddy.
Consistency	Offering twelve spacing values instead of "whatever looks right" makes consistency the default instead of something checked in review.
Flexibility	Naming colours by their job (07.4) is what later makes dark mode and new brands possible without touching any components.
Speed	Fewer choices mean faster decisions. "Which of these twelve?" is a much quicker question than "how many pixels?"

06.3

From a designer's decision to the product's code

Here's the problem foundations have to solve. A designer decides the primary button should be a particular shade of orange. In a traditional handover, she writes `#AB2800` on a spec. An engineer copies it into the code. Another engineer, on another team, copies it from a screenshot and gets `#AA2900`. A year later, nobody knows which is right, and changing it means finding every copy.

Design tokens fix this by making sure a decision is **written down once** and everything else reads from that one place. The steps look like this:



FIG 6.2 How a decision becomes code. Only step 01 needs a human judgement. Everything after it is automatic, so changing the value in step 02 or 03 updates every platform in one release.

Notice that there are two names in the middle. The first, `brand-700`, describes the colour itself: it's the 700 shade of our brand orange. The second, `action-primary`, describes its *job*: it's the colour for main actions. Buttons use the job name, not the colour name. That small extra step is what makes dark mode possible later. In dark mode, "the colour for main actions" can point to a lighter orange, and no button needs to change. Chapter 07 explains this properly.

A shared format for tokens

For years, every design tool and code library stored tokens in its own format, which made moving between them painful. In October 2025 the W3C Design Tokens Community Group, an industry group that includes people from Adobe, Google, Microsoft and others, published the first stable version of a shared format.[4][32] DOCUMENTED It's a simple structured text file. Each token has a type (colour, size and so on) and a value, and tokens can point to each other using curly brackets, like `{color.brand.700}`. (Strictly speaking it's a community group report rather than an official W3C standard, but it's the format most token tools are now converging on.)

Here's what a small piece of Ledgerly's tokens looks like in that format. You don't need to be able to read code to get the idea: raw colours at the top, job names that point to them in the middle, and a button that points to a job name at the bottom.

```
{
  "color": {
    "brand": {
      "500": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.645, 0.19, 37], "hex":
"#e95728"}},
      "600": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.565, 0.189, 37], "hex":
"#cc3c00"}},
      "700": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.486, 0.172, 35], "hex":
"#ab2800"}},
      "800": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.406, 0.146, 34], "hex":
"#861b00"}}
    },
    "neutral": {
      "50": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.975, 0.011, 260], "hex":
"#f3f7fe"}},
      "100": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.945, 0.018, 260], "hex":
"#e6edf9"}},
      "700": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.485, 0.018, 260], "hex":
"#595f69"}},
      "950": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.255, 0.018, 260], "hex":
"#1e232c"}}
    }
  },
  "semantic": {
    "action": {
      "primary": {
        "bg": {"$type": "color", "$value": "{color.brand.700}"},
        "text": {"$type": "color", "$value": "{color.neutral.50}"}
      }
    },
    "text": {
      "primary": {"$type": "color", "$value": "{color.neutral.950}"}
    }
  },
  "button": {
    "primary": {
      "background": {"$type": "color", "$value": "{semantic.action.primary.bg}"}
    }
  }
}
```

FIG 6.3 Tokens in the shared format. Three levels: raw colours (with their exact values), job names that point to raw colours, and a button setting that points to a job name. These values were calculated by this playbook's build script, not typed by hand.

PRACTICE **TOKENS FIRST, COMPONENTS SECOND**

This is Rule #4 of how we work, and the most important sequencing decision in a design system. Components built before tokens exist end up with colour codes and sizes typed directly into them, which someone has to find and replace later. Tokens set up first make every component ready for themes and changes from day one.

THE SHORT VERSION

What to remember from this chapter

01 Foundations are the basic visual decisions everything else inherits: colour, type, spacing and layout, plus a few more.

02 They have the biggest blast radius in the system, so they deserve the most careful thought.

03 Tokens let a decision be written once and read everywhere: by design tools and by the code for every platform.

04 Name the raw value, then name its job. Components use the job names.

Up next → **Chapter 07: Colour systems**

CHAPTER 07

07

Colour systems

Colour is the foundation teams get wrong most often, and the one where getting it right pays off fastest. We'll go step by step, from how colours are described, to building a palette, to making dark mode a simple switch. Every value in this chapter is calculated, not guessed.

READ THIS IF

You're creating or fixing a colour palette, planning dark mode or a second brand, or your accessibility audits keep flagging text that's hard to read.

WHAT YOU'LL LEAVE WITH

01 Why the way you describe colour matters, and why we recommend a model called OKLCH.

02 A repeatable method for building a palette, with a complete worked example.

03 A way of naming colours that makes dark mode and new brands a settings change.

IN THIS CHAPTER

07.1 Why colour goes wrong
07.2 How colour is described
07.3 Building a palette, step by step
07.4 Making sure people can read it
07.5 Naming colours by their job
07.6 Colours that respond: hover, press and focus
07.7 Dark mode, brands and white-labelling
07.8 Questions teams always ask about colour

07.1

Why colour goes wrong

Let's start with a story we've seen in some form at almost every company we've audited.

SCENARIO **340 greys and no way to switch them**

THE SITUATION	WHAT THEY FIND	WHY IT MATTERS
A five-year-old product finally commits to dark mode. An engineer searches the code for colour values to see how big the job is.	340 different colour values, most of them greys that are almost, but not quite, the same. Some are for text, some for borders, some for backgrounds. Nothing in the code says which is which.	To build dark mode, someone has to look at every one of those 340 values in context and decide what it should become. A two-sprint feature turns into a two-quarter project.

Nobody set out to create 340 greys. A designer picked a grey that looked right for a divider. Another picked a slightly lighter one for a caption. An engineer approximated one from a screenshot. Each choice was reasonable. Together they're unmanageable.

There's a second, quieter problem too: **text that's hard to read**. Pale grey text looks elegant in a design file on a bright monitor. On a laptop in sunlight, or for someone with less-than-perfect eyesight, it disappears. WebAIM's 2025 scan of the world's top million home pages found text with too little contrast on 79.1% of them, making it the single most common accessibility problem on the web. [10] DOCUMENTED

A colour system solves both problems. It gives you a small, deliberate set of colours, each checked for readability, each named by the job it does. The rest of this chapter shows you how to build one.

07.2

How colour is described

To a computer, a colour is just numbers, and there are several ways of writing those numbers down. They're called **colour models**. You've probably seen some of them: `#CC3C00` or `rgb(204, 60, 0)`. The model you use matters more than you'd think, because it decides which questions are easy to answer. For a design system, the most important question is: *is this colour lighter or darker than that one, and by how much?* Readability, hierarchy and hover effects all depend on it.

MODEL	DESCRIBES COLOUR AS	GOOD FOR	THE CATCH
RGB / HEX	Amounts of red, green and blue light	How screens actually work; every tool understands it	Unreadable to humans. Nobody can look at `#CC3C00` and know how to make it 10% lighter.
HSL / HSV	Hue (which colour), saturation, lightness	Picking colours by feel; most colour pickers use it	Its "lightness" isn't what your eyes see. A yellow and a blue with the same HSL lightness look completely different.
OKLCH	Lightness (0 to 1), chroma (how colourful), hue (which colour)	Building palettes and themes	Some combinations can't be shown on normal screens, and its lightness isn't exactly the same as the official readability measure. Both are manageable (see below).

"Lightness" that matches your eyes

The big difference is that OKLCH is a **perceptual** model. It was designed so that the numbers match what people actually see. It's based on a colour space called Oklab, published by Björn Ottosson in 2020,[33] and it's now built into CSS, the language that styles websites, as `oklch()`.[34] STANDARD

The easiest way to see why this matters is to look. Below, the top row shows six colours that HSL says are all equally light. The grey strip under each one shows how light it really looks.

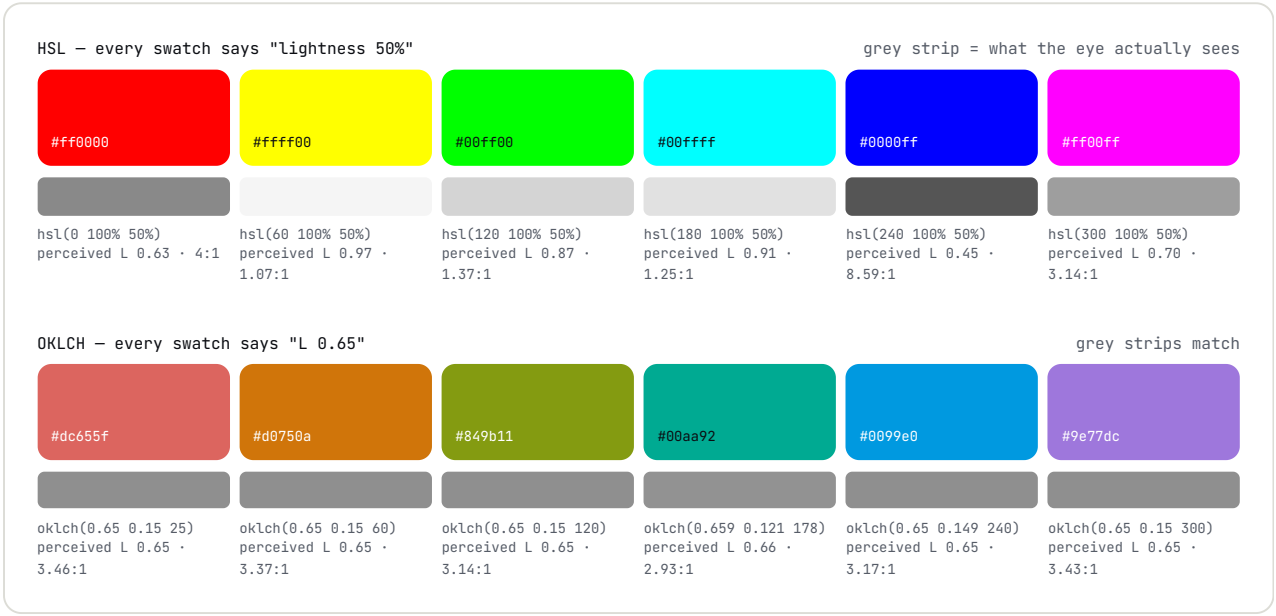


FIG 7.1 Same number, very different result. Top: six colours with the same HSL lightness. How light they actually look ranges from 0.45 to 0.97, and their readability on white ranges from 1.07:1 to 8.59:1. Bottom: six colours with the same OKLCH lightness. The grey strips match. That, in one picture, is why we build palettes in OKLCH.

The three dials of OKLCH

You can think of OKLCH as three dials. Turn one and the other two stay put.

Lightness `L 0 → 1`



How light it looks, from black (0) to white (1). This is the dial that controls readability.

Chroma `C 0 → ~0.37`



How colourful it is. Zero is grey. The maximum depends on the hue and the lightness.

Hue `H 0° → 360°`



Which colour family, as an angle around a wheel. Every swatch in this row looks equally light.

Why OKLCH is so useful for a design system

Because the dials are independent and lightness matches your eyes, a lot of hard problems get easy. You can say "shade 700 of every colour is this light" and it will be true for blue, red and green alike. You can swap one brand colour for another by turning only the hue dial, without changing how light anything is, so readability doesn't break. Palettes become a recipe rather than a pile of hand-picked swatches. Gradients look cleaner. And OKLCH can describe the more vivid colours modern screens can show. It's supported in CSS and in the shared token format, [34][32] and Tailwind CSS, one of the most widely used styling frameworks, switched its whole default palette to OKLCH in version 4.0.[35] DOCUMENTED

TWO THINGS TO WATCH OUT FOR

1. Some combinations can't be displayed. Screens can only show a certain range of colours. Very colourful shades that are also very light or very dark often fall outside that range. Good tools quietly turn down the chroma dial until the colour fits. Our palette generator does this and marks each colour where it happened.

2. OKLCH lightness isn't the official readability score. The accessibility guidelines measure contrast with a different formula.[36] Equal OKLCH lightness gives *similar* readability across colours, not identical. You still check every pairing; 07.4 shows how close it gets.

07.3

Building a palette, step by step

Step 1: start from your brand colour, but don't use it for everything

Most palettes start from a brand colour, and the first surprise is usually that the exact brand colour can't be used for text or for buttons with white text on them. Layer One's own orange, `#ff6b3d`, is a perfect example. On white it has a contrast ratio of just **2.83 to 1**, well below the 4.5 to 1 the accessibility guidelines require for normal text. On the dark ink background, it reaches a comfortable 6.87 to 1. STANDARD

The fix isn't to change the brand. It's to treat the brand colour as a *starting point* for a family of shades, then use different shades for different jobs. The exact brand orange stays for big brand moments like the logo and illustrations. A deeper shade from the same family is used for buttons and links on light backgrounds. In the palette below, the brand family is generated from `#ff6b3d`, and shade 700 becomes the main action colour.

Step 2: add supporting colours for specific jobs

A product needs far fewer colours than a brand guideline suggests. Most products need one brand colour, a family of greys, and a few colours that signal status:

COLOUR	ITS JOB	RULE OF THUMB
Brand	Main actions, selected items, brand moments	Never reused to mean "error" or "success"
Neutral (greys)	Text, backgrounds, borders: most of the screen	Very slightly tinted, so it doesn't feel lifeless
Red	Errors, destructive actions like "Delete"	Clearly different from the brand colour
Green	Success, confirmation	Always paired with an icon or words
Amber	Warnings, pending states	Use dark text on amber, not white
Blue	Links, information, keyboard focus outlines	Can double as the brand colour if your brand is blue

One important rule: colour should never be the *only* way to tell people something. Colour blindness is common, screens and lighting vary, and WCAG 2.2 requires that colour isn't the only visual signal.[13] STANDARD So a red error message also gets an icon and clear words.

Watch for clashes between your brand and your status colours, too. Ledgerly's orange sits at hue 37° and its error red at 25°, only 12° apart. That's common with warm brands, and it means colour alone could never tell "main action" from "delete". The answer isn't a different colour. It's design: delete buttons get clear labels and a confirmation step, and errors always come with an icon.

Step 3: build each colour into a family of shades

NEW WORD	
Ramp	A family of shades of one colour, from very light to very dark, numbered like 50, 100, 200 … 950.

Each colour becomes a **ramp**: a set of shades numbered from very light (50) to very dark (950). The clever part, and the reason we use OKLCH, is that each shade number has the same lightness in every colour. Blue 700, red 700 and grey 700 are all equally light (L ≈ 0.485). So the number itself tells you something useful: "anything 700 or darker is safe for text on white".

SHADE	50	100	200	300	400	500	600	700	800	900	950
Lightness	0.975	0.945	0.885	0.815	0.730	0.645	0.565	0.485	0.405	0.330	0.255
Colourfulness	12%	26%	50%	74%	92%	100%	100%	92%	78%	60%	45%
Typical use	Backgrounds, tints		Soft fills, borders		Strong borders, icons		Text-safe accents, buttons		Text, dark-mode surfaces		

Notice that the shades are bunched up at both ends. That's on purpose. Interfaces need lots of very light tints (backgrounds, hover states, selected rows) and several very dark shades (text, dark-mode surfaces), but only a few mid-tones. Other well-known systems take a similar purpose-driven approach: Radix Colors uses 12 steps, each with a stated job from "app background" to "high-contrast text",[37] and Google's Material Design generates families of tones from a key colour.[31] DOCUMENTED

Step 4: shape the colourfulness

Colourfulness should peak in the middle of the ramp and fade towards both ends. Very light shades that are very colourful look neon; very dark shades that are very colourful look muddy. That's the "colourfulness" row in the table above. Where a shade would be too colourful for screens to show, the generator turns the chroma dial down until it fits, keeping lightness the same so readability stays predictable.

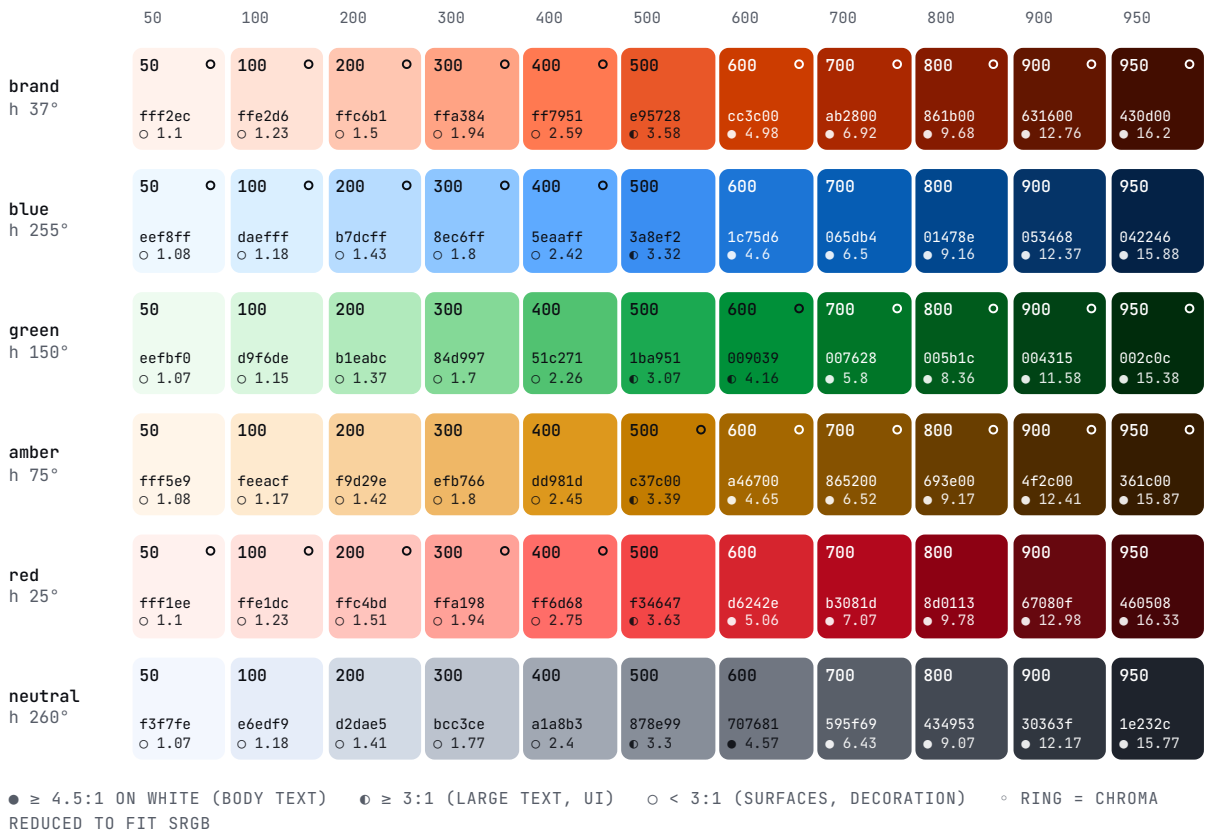


FIG 7.2 Ledgerly's complete palette: six ramps, 66 colours. Each chip shows its shade number, the colour code, and its contrast against white. Everything is generated from three inputs: the lightness curve, the colourfulness curve and one hue angle per colour. A small ring marks shades whose colourfulness was turned down to fit what screens can display.

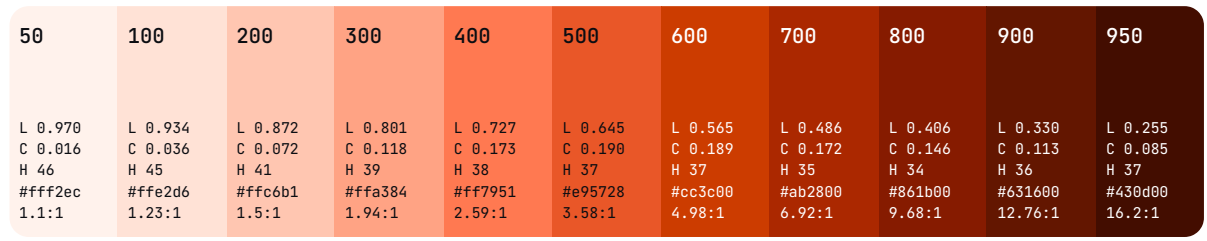


FIG 7.3 The brand ramp up close. Lightness (L), colourfulness (C) and hue (H) for each shade, plus its colour code and contrast on white. Lightness climbs in even, predictable steps while colourfulness rises and falls.

07.4

Making sure people can read it

NEW WORD

Contrast ratio

How different two colours are in brightness, from 1:1 (identical) to 21:1 (black on white). Higher is easier to read.

Readability is measured with a **contrast ratio**: a number that compares how bright the text is with how bright the background is. The Web Content Accessibility Guidelines (WCAG) 2.2 set three thresholds you'll use all the time:[36][38] STANDARD

4.5:1

The minimum for normal text. (7:1 for the stricter AAA level.)

3:1

The minimum for large text: about 24px, or about 19px if bold.

3:1

The minimum for the edges of buttons and inputs, focus outlines and meaningful icons.

Because every ramp uses the same lightness curve, contrast behaves almost identically across colours. The chart below proves it: six colours, six lines that nearly sit on top of each other.

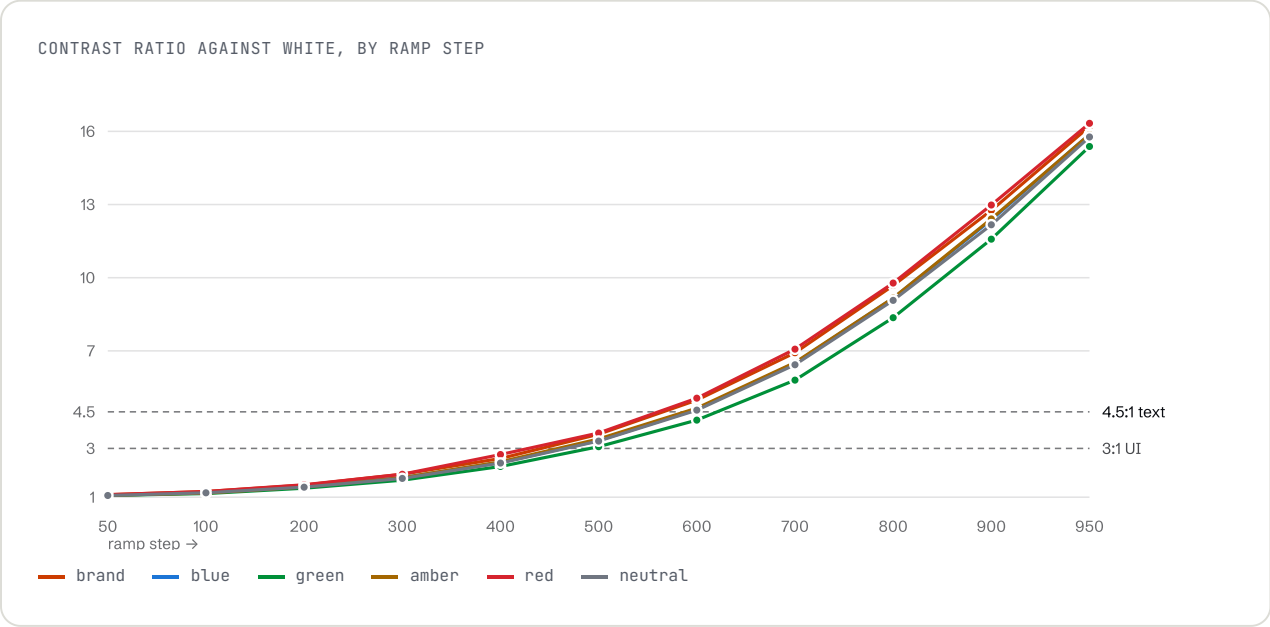


FIG 7.4 Equal lightness gives near-equal readability. It isn't perfect: at shade 600, green reaches 4.16:1 while red reaches 5.06:1, because the official formula weights green more heavily than OKLCH does. So the simple, safe rule is "shade 700 and darker passes 4.5:1 on white, in every colour", which this palette guarantees. Anything else, check individually.

This also gives designers **pairing rules** they can use without a contrast checker. Because lightness is fixed per shade, readability between two shades depends mostly on how many steps apart they are:

	50	100	200	300	400	500	600	700	800	900	950
50	—	1.1	1.3	1.6	2.2	3.1	4.3	6.0	8.4	11.3	14.7
100	1.1	—	1.2	1.5	2.0	2.8	3.9	5.5	7.7	10.3	13.4
200	1.3	1.2	—	1.3	1.7	2.3	3.2	4.6	6.4	8.6	11.2
300	1.6	1.5	1.3	—	1.4	1.9	2.6	3.6	5.1	6.9	8.9
400	2.2	2.0	1.7	1.4	—	1.4	1.9	2.7	3.8	5.1	6.6
500	3.1	2.8	2.3	1.9	1.4	—	1.4	1.9	2.8	3.7	4.8
600	4.3	3.9	3.2	2.6	1.9	1.4	—	1.4	2.0	2.7	3.5
700	6.0	5.5	4.6	3.6	2.7	1.9	1.4	—	1.4	1.9	2.5
800	8.4	7.7	6.4	5.1	3.8	2.8	2.0	1.4	—	1.3	1.7
900	11.3	10.3	8.6	6.9	5.1	3.7	2.7	1.9	1.3	—	1.3
950	14.7	13.4	11.2	8.9	6.6	4.8	3.5	2.5	1.7	1.3	—

● ≥ 7:1 (AAA TEXT) ● ≥ 4.5:1 (AA TEXT) ● ≥ 3:1 (LARGE TEXT, UI BOUNDARIES) BLANK < 3:1

FIG 7.5 Contrast between every pair of greys. Each cell is the contrast ratio between two shades. In this palette, shades **five or more steps apart** always reach 3:1 (enough for borders and large text), and shades **seven or more steps apart** always reach 4.5:1 (enough for body text). Rules this simple only work on an evenly built ramp.

The light grey text trap

The most common readability mistake is "secondary" text, like captions, timestamps and helper text, set in a pale grey because it looks refined. It's the main reason so many websites fail contrast checks. Fix it once, at the source: Ledgerly's secondary text colour points to grey 700, which reaches 5.99:1 on the background. Nothing lighter is allowed for text, and because every component uses that one named colour, every screen passes.

WHAT ABOUT APCA?

You may hear about APCA, a newer way of measuring contrast that takes text size and weight into account and has been explored for future versions of the guidelines.[39] Radix Colors, for example, tunes its text shades with it.[37] It's promising, but today the WCAG 2 ratio is the official requirement. Use WCAG 2 to pass, and APCA as extra guidance if you like.

07.5

Naming colours by their job

A palette is a box of paints. A colour *system* is the set of labels that says what each paint is for. This is the single most important idea in this chapter, so let's take it slowly.

There are two kinds of name. The first kind describes the colour: "blue 600", "grey 950". The second kind describes the job: "link text", "page background", "error message". Components only ever use the second kind. The job names point to the colour names.

NEW WORD	
Semantic token	A colour named by its job ("text-secondary") rather than its appearance ("grey-700"). It points to a raw colour.

The design systems world has formal names for these levels. Colour names are called **primitive tokens** (the raw ingredients). Job names are called **semantic tokens** ("semantic" just means "about meaning"). Some systems add a third level of **component tokens** for settings specific to one component. Every mature public design system we know of separates raw values from their jobs, even if they use different words for it.[40][41][42][3] DOCUMENTED

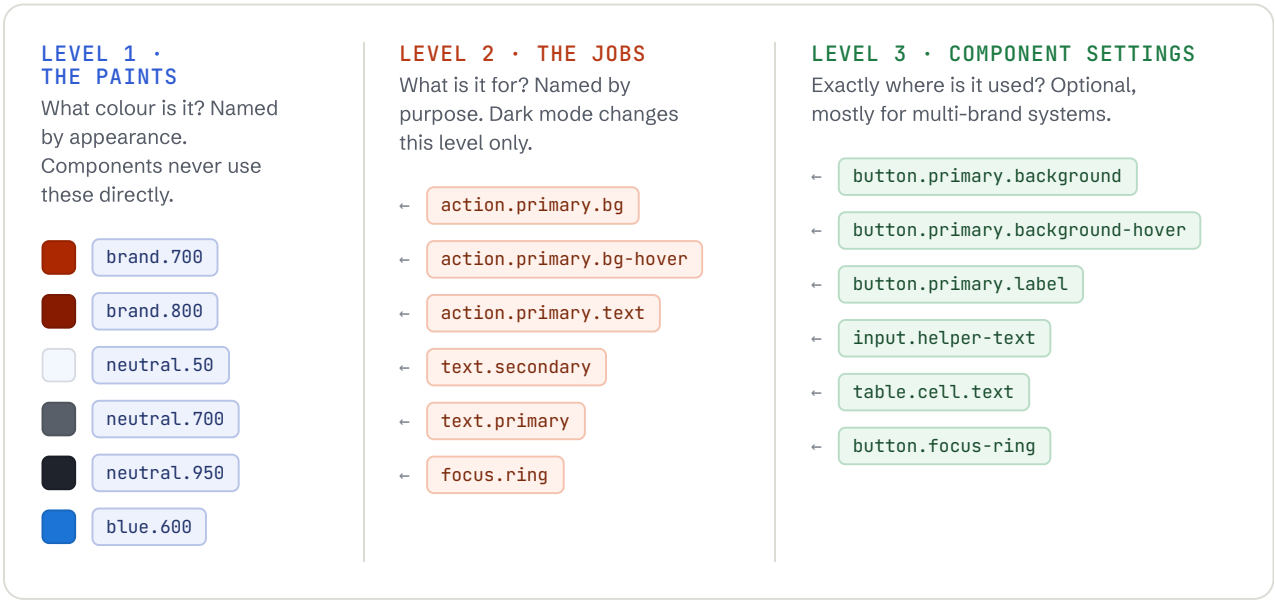


FIG 7.6 **Three levels of colour naming.** Read each row from right to left: the button's background uses the "main action" colour, which uses brand orange 700. The actual colour code lives in exactly one place.

Why bother with two names?

Dark mode becomes a switch

Dark mode just changes which paint each job points to. No component changes.

Fixes happen in one place

If error text needs to be darker, you change one pointer, not every red in the product.

Checks are simple

"Is secondary text readable?" is one check, not an audit of every grey ever used for text.

The palette stays tidy

People choose from a short list of jobs, not 66 swatches, so new random colours stop appearing.

Do you need the third level?

Component settings add another layer to name, document and maintain. Use them if you support several brands or white-labelled versions that need to restyle specific components, or if one component genuinely needs to differ from the normal rule. Otherwise, components can use job names directly. Many teams start with two levels and add the third only when they need it. PRACTICE

Naming the jobs

Pick a simple naming pattern so that people can guess names they haven't seen. That's Rule #3 of how we work: *name things so people can guess them*. This pattern works for most products: what kind of thing, then its role, then (if needed) a variation and a state.

```
color.bg.surface                    // backgrounds: canvas, surface, subtle
color.text.secondary                 // text: primary, secondary, disabled, link
color.border.strong                  // borders: default, strong
color.action.primary.bg.hover       // interactive: role, part, state
color.feedback.danger.text          // status: danger, warning, success, info
```

SEMANTIC TOKEN	LIGHT THEME →	DARK THEME →
color.bg.canvas	neutral.50	neutral.950
color.bg.surface	neutral.50	neutral.900
color.bg.subtle	neutral.100	neutral.800
color.border.default	neutral.200	neutral.700
color.border.strong	neutral.500	neutral.500
color.text.primary	neutral.950	neutral.50
color.text.secondary	neutral.700	neutral.300
color.text.disabled	neutral.400	neutral.600
color.text.link	blue.700	blue.300
color.action.primary.bg	brand.700	brand.400
color.action.primary.bg-hover	brand.800	brand.300
color.action.primary.text	neutral.50	neutral.950
color.feedback.danger.text	red.700	red.300
color.feedback.danger.bg	red.100	red.900
color.feedback.success.text	green.700	green.300
color.feedback.success.bg	green.100	green.900
color.focus.ring	blue.600	blue.400

FIG 7.7 Ledgerly's colour jobs, and what each points to in light and dark mode. This table *is* the theme. No component code is involved.

07.6

Colours that respond: hover, press and focus

Buttons, links and other interactive elements need to change colour when you hover over them, press them, or reach them with the keyboard. There are two common ways to handle this.

APPROACH A — STEP ALONG THE RAMP

Move one shade darker

Hover goes one shade darker in light mode (brand 700 → 800) and one lighter in dark mode. Simple, easy to check for readability, and every state has a name.

Default | Hover | Pressed | Focus

APPROACH B — ADD A SEE-THROUGH LAYER

Lay a tint on top

A faint, see-through layer is placed over the button on hover or press. Material Design uses this approach. It works on any colour and needs fewer names.

Default | + hover layer | + press layer

Focus deserves special attention. When someone moves through a page with the keyboard (many people do, and some have to), the focus outline shows where they are. It must contrast at least 3:1 with what's around it.[38] Give it its own named colour (Ledgerly uses blue 600, which reaches 4.28:1 on the background) rather than reusing the brand colour, which may not stand out against the button it surrounds.

Disabled elements are exempt from contrast rules, but they still need to be recognisable. And never rely on a greyed-out button alone to explain why something can't be done. Tell people why.

07.7

Dark mode, brands and white-labelling

Here's the payoff for all the naming work. A theme is simply a complete list of which paint each job points to. Below is the same little interface, built from the same named jobs, shown with two different lists.

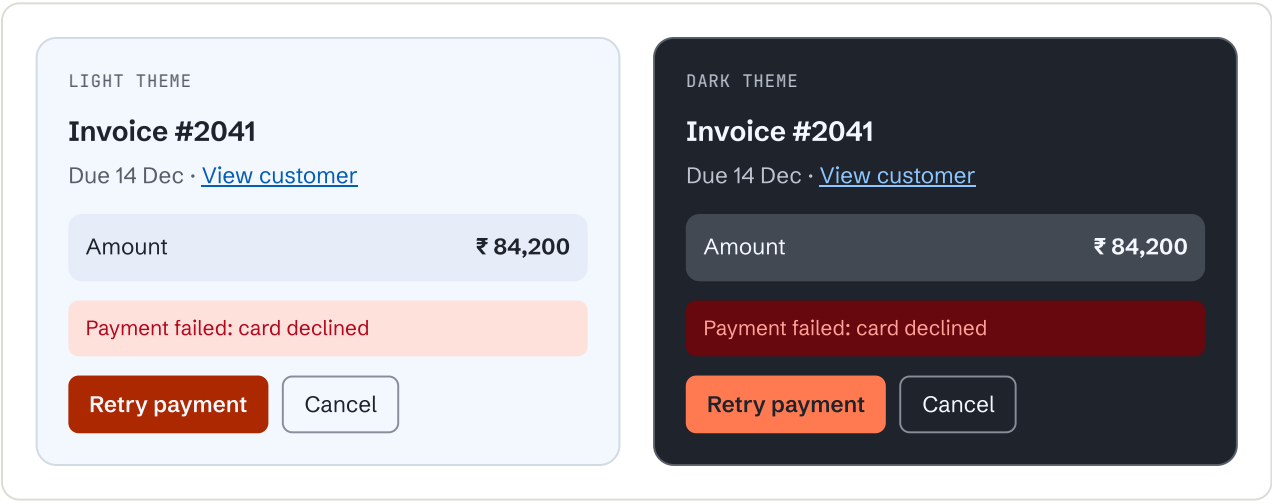


FIG 7.8 One interface, two themes. Every colour on both cards comes from the job names in the previous table. Nothing in the "component" changed between them, only the list of pointers.

USE	PAIRING	WCAG MIN	LIGHT	DARK
Body text	`text.primary` on `bg.canvas`	4.5:1	14.67 pass	14.67 pass
Secondary text	`text.secondary` on `bg.canvas`	4.5:1	5.99 pass	8.89 pass
Links	`text.link` on `bg.canvas`	4.5:1	6.05 pass	8.76 pass
Primary button label	`action.primary.text` on `action.primary.bg`	4.5:1	6.44 pass	6.09 pass
Error message	`feedback.danger.text` on `feedback.danger.bg`	4.5:1	5.74 pass	6.68 pass
Success message	`feedback.success.text` on `feedback.success.bg`	4.5:1	5.02 pass	6.81 pass
Input border (non-text)	`border.strong` on `bg.canvas`	3:1	3.07 pass	4.77 pass
Focus ring (non-text)	`focus.ring` on `bg.canvas`	3:1	4.28 pass	6.51 pass
Disabled text (exempt)	`text.disabled` on `bg.canvas`	exempt	2.23	3.45

FIG 7.9 Every pairing checked, in both themes. Calculated by the build script from the real colour values. A theme isn't finished until this table passes, so make it an automatic test that runs whenever colours change.

Dark mode isn't just "invert the colours"

- **Avoid pure black.** A very dark grey (Ledgerly uses grey 950) is easier on the eyes and leaves room to show depth.

- **Show depth with lighter surfaces.** Shadows barely show on dark backgrounds, so raised cards and menus get slightly *lighter* instead.
- **Lighten the accent colours.** Ledgerly's main action colour moves from brand 700 to brand 400 in dark mode, with dark text on top (5.96:1).
- **Check everything again.** A pairing that passes in light mode can fail in dark mode, and the other way round.

IBM's Carbon, for example, offers four themes (two light, two dark) from a single set of colour jobs. [41] DOCUMENTED

Several brands, or customers' brands

SITUATION	WHAT CHANGES	HOW TO SET IT UP
Light and dark	Which paint each job points to	Two pointer lists over one palette
Several brands you own	The brand paints, sometimes fonts and corner rounding	A brand level that swaps the brand paints, then light and dark per brand
White-label for customers	A few things the customer chooses, like one colour and a logo	Generate the customer's ramp from their colour; let them change *only* those inputs

PRACTICE **WHITE-LABELLING SAFELY**

Let the customer pick one brand colour, then generate their ramp using your lightness curve, and point your jobs at it. Because lightness is fixed per shade, their "brand 700" ends up exactly as light as yours, so readability survives a colour choice you didn't make. Run the pairing checks automatically and fall back to a darker shade if a colour falls short (as green does at 600).

07.8

Questions teams always ask about colour

Q

How many colours should a design system have?

SHORT ANSWER — **Few colour families, plenty of shades, and fewer named jobs than you'd expect.**

Most products need one brand colour, one grey family and three or four status colours: five to seven ramps. At 11 shades each, that's 55 to 77 paints, but only 30 to 60 named jobs, and the jobs are what people actually choose from. PRACTICE

Q

How many shades should each ramp have?

SHORT ANSWER — **Between 10 and 13 is the normal range.**

Tailwind uses 11, Radix 12 and Material 13.[35][37][31] DOCUMENTED Fewer than about 9 leaves gaps for hover states and dark mode. More than about 13 gives you shades nobody can tell apart.

Q

Is there a maximum?

SHORT ANSWER — **No official one. The practical limit is how many your team can tell apart and remember.**

The warning sign isn't the size of the palette. It's having several job names that mean almost the same thing: "text-muted", "text-subtle", "text-secondary", "text-tertiary". If you can't explain the difference between two names in one sentence, merge them.

Q

How do we stop new colours creeping in?

Components use job names only, and an automatic check flags any colour code typed directly into product code. New job names need a reason ("which job doesn't exist yet?") and the owner's approval. And recount the "different colours in our code" number from Chapter 03 every quarter. It should go down.

Q

How should we do gradients?

SHORT ANSWER — **Blend in OKLCH, and name the start and end colours.**

Blending in the traditional way passes through a dull, greyish middle. Blending in OKLCH keeps the middle bright and even. Modern CSS lets you choose which way to blend.



BLENDED THE TRADITIONAL WAY (SRGB): BLUE 600 → AMBER 400



BLENDED IN OKLCH: BLUE 600 → AMBER 400

Q

What about see-through colours and overlays?

SHORT ANSWER — **Fine for overlays and hover layers; never for text.**

The dark, see-through layer behind a pop-up dialog is a good use of transparency. Transparent text isn't: its readability changes depending on whatever is behind it, so you can't check it once and trust it. Give text

solid colours.

Colour checklist

- [ ] Palette built in OKLCH from one lightness curve
- [ ] Brand colour used as a starting point; buttons use a readable shade
- [ ] 5–7 colour families, 10–13 shades each, same numbering everywhere
- [ ] Colour codes generated by a tool, not hand-picked
- [ ] A deliberately tinted (or deliberately neutral) grey family
- [ ] Simple pairing rules shared with designers
- [ ] Raw colours and job names kept separate; components use job names only
- [ ] A naming pattern written down
- [ ] Hover, press, focus and disabled colours for every interactive role
- [ ] Light and dark mode as two pointer lists
- [ ] Every pairing checked automatically
- [ ] Status colours always paired with an icon or words

THE SHORT VERSION

What to remember from this chapter

01 Use OKLCH: its lightness matches what eyes see, so palettes become predictable.

02 Build each colour into a ramp where each shade number has the same lightness in every colour.

03 Check readability with the WCAG ratios: 4.5:1 for text, 3:1 for large text and UI edges.

04 Name colours by their job and let components use only job names. Dark mode and new brands then become a switch.

Up next → **Chapter 08: Typography systems**

CHAPTER 08

08

Typography systems

Most of what people look at in your product is text: labels, numbers, headings, messages. A typography system isn't just picking a nice font. It's deciding a small set of text styles, each with a clear job, that stay readable on every screen.

READ THIS IF

You're choosing a font, setting up text sizes, or your screens feel cluttered and it's hard to tell what's important.

WHAT YOU'LL LEAVE WITH

01 How to choose a font for a product (not a poster).

02 A complete, named set of text styles you can adapt.

03 How text should differ between dashboards, mobile apps, enterprise tools and more.

IN THIS CHAPTER

08.1 Choosing a font
08.2 Text sizes that work together
08.3 Naming text by its job
08.4 Text on different screen sizes
08.5 Readable for everyone
08.6 Different products need different type
08.7 Turning text styles into tokens

08.1

Choosing a font

When a brand team picks a font, they're usually thinking about a logo, a billboard or a headline. A product font has a harder job. It has to be readable at 12 pixels in a crowded table, show numbers that line up in columns, and work in every language you sell in. So for products, choosing a font is mostly a practical decision. Personality matters, but it comes second.

WHAT TO CHECK	HOW TO CHECK IT	WHY IT MATTERS
Readable when small	Look at 12-14px samples on a real laptop screen. Can you tell `I`, `l` and `1` apart? `O` and `0`?	Labels, tables and captions are all small.
Enough weights	At least regular, medium and semibold, or a "variable" font with a weight range.	Bold and regular are how you show what's important.
Numbers that line up	Check for "tabular figures": numbers that are all the same width.	So columns of amounts align in tables and dashboards.
Your languages	Every script you use now or might soon: accented Latin, Cyrillic, Arabic, Devanagari…	A fallback font for one market breaks consistency.
Licence	Can you use it on the web, in apps, across all your products? Open-source fonts make this easy.	A licence problem can block a launch.
File size	How much it adds to page load.	Fonts are often among the largest files a page loads first.

THE "DO NOTHING" OPTION

Every operating system has a built-in interface font: San Francisco on Apple devices, Roboto on Android, Segoe UI on Windows. They're very readable, free, load instantly and feel familiar. For internal tools they're often the best choice. The trade-off is that your product looks less distinctive.

HOW MANY FONTS?

One for the interface is usually enough. A second, like a monospaced font for codes and IDs or a display font for marketing, is common. A third is rarely needed. This book uses one font for everything, plus a monospaced font for values a computer would read.

Variable fonts: one file, every weight

NEW WORD

Variable font

A single font file that can produce any weight (and sometimes width) in a range, instead of separate files for each.

Traditionally, regular, medium and bold were separate font files. A **variable font** stores a whole range of weights in one file.[43] For a design system that means fewer files to load and exact weights to choose from. Systems still name a small set of weights, so "any weight" doesn't turn into "every weight".



FIG 8.1 One file, any weight. The font used in this book, Schibsted Grotesk, can produce any weight from 400 to 900. The system names four of them so everyone uses the same ones.

08.2

Text sizes that work together

NEW WORD

Type scale The short, fixed list of text sizes a product uses, like 12, 14, 16, 18, 22, 28 and 36 pixels.

If anyone can pick any text size, you end up with 13, 14, 14.5, 15 and 16 pixels all over the product, and nothing quite looks related. A **type scale** fixes this: a short list of sizes that everyone chooses from.

A classic way to create one is to start from a base size (usually 16 pixels, the browser's default) and multiply by the same number each step up. That number is called the ratio. A small ratio gives you lots of sizes close together; a large ratio jumps quickly to big headlines.

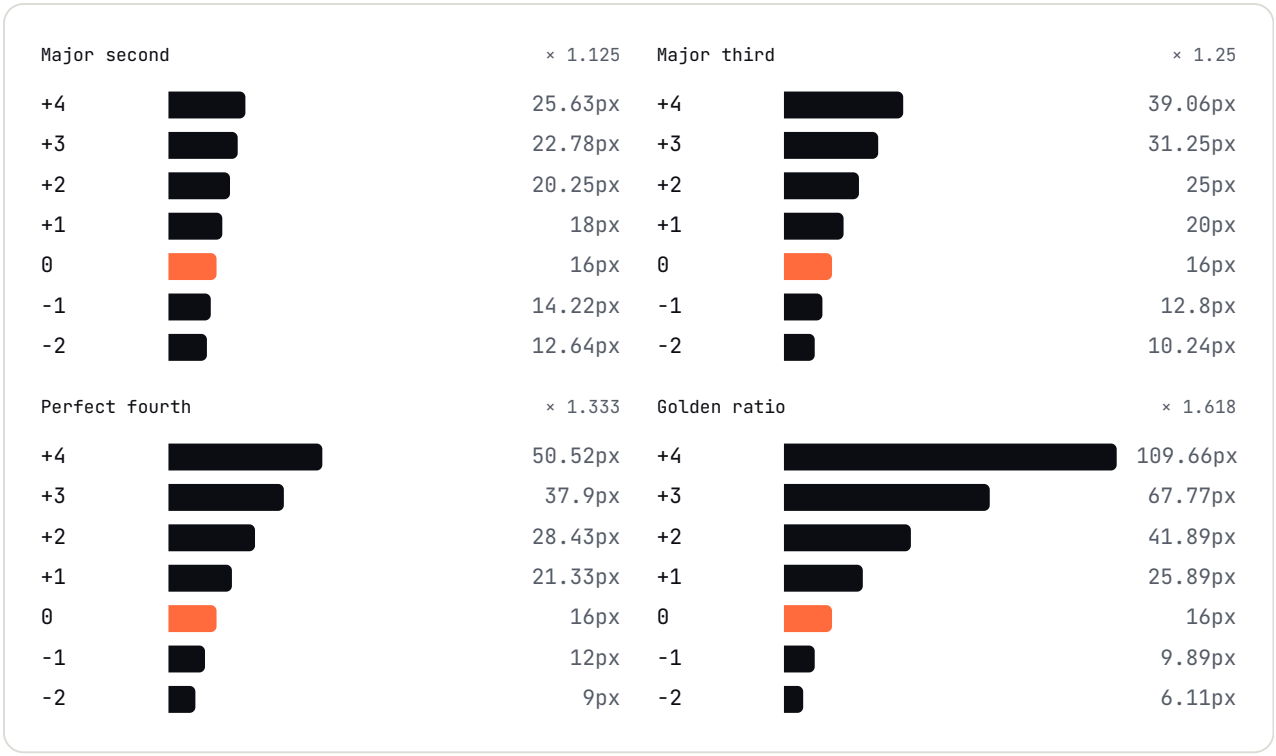


FIG 8.2 Four scales from the same 16px starting point. A small ratio (1.125) suits busy, information-dense screens. A big one (1.618, the "golden ratio") suits magazines and marketing pages, but jumps so fast it leaves few useful sizes for an app.

In practice, a pure mathematical scale is a starting point, not the answer. Apps need several sizes clustered between 12 and 20 pixels (labels, body text, small headings) and only a few large ones. So most mature systems start from a ratio, then adjust by hand and round to whole numbers. Google's Material Design 3, for example, uses a hand-tuned set of 15 named styles rather than a strict formula. [44] DOCUMENTED

Line spacing: tighter as text gets bigger

The space between lines (designers call it line-height or leading) should get proportionally *tighter* as text gets bigger. Body text reads comfortably at about 1.4 to 1.6 times its size. Big headlines look loose and disconnected above about 1.2. We also round every line-height to a multiple of 4 pixels, so text lines up neatly with everything else on the page (Chapter 09 explains why 4).

TEXT SIZE	LINE SPACING (× SIZE)	HOW LEDGERLY'S SCALE WAS CALCULATED
Up to 14px	about 1.45	Multiply the size by the target, then round to the nearest multiple of 4px. Small text ends up a little looser and large text a little tighter, which is the direction you want. Headings use the tighter targets at every size, so an 18px heading gets 24px, not the 28px of 18px body text.
15–20px	about 1.5	
21–32px	about 1.3	
33–48px	about 1.2	
Over 48px	about 1.1	

Letter spacing

Fonts are spaced for reading sizes. Blown up to headline size, the gaps between letters start to look too wide, so big headings usually get slightly *tighter* letter spacing. Very small text and ALL-CAPS labels go the other way and get slightly *looser* spacing, which makes them easier to read.

08.3

Naming text by its job

Just like colours, text styles should be named by what they're for, not by their size. "Heading large" survives a redesign; "28px" doesn't. Five families of text cover almost every product:

DISPLAY
Big numbers and headlines: a dashboard total, a welcome message.

HEADING
Page and section titles: the structure of a screen.

BODY
Paragraphs, descriptions, table cells, form values.

LABEL
Buttons, tabs, form labels, menu items. Slightly bolder.

CAPTION
Timestamps, helper text, small print.

display.lg	Ledgerly	57/64 · 600 · -0.02em
display.md	Ledgerly	45/56 · 600 · -0.02em
heading.xl	Quarterly revenue	36/44 · 600 · -0.015em
heading.lg	Quarterly revenue	28/36 · 600 · -0.01em
heading.md	Quarterly revenue	22/28 · 600 · -0.005em
heading.sm	Quarterly revenue	18/24 · 600
body.lg	Invoices are reconciled nightly against the le…	18/28 · 400
body.md	Invoices are reconciled nightly against the ledger.	16/24 · 400
body.sm	Invoices are reconciled nightly against the ledger.	14/20 · 400
label.md	Export CSV	14/20 · 500 · 0.005em
label.sm	Export CSV	12/16 · 500 · 0.01em
caption	Updated 4 minutes ago	12/16 · 400 · 0.01em

FIG 8.3 Ledgerly's complete set of text styles. Twelve styles in five families, shown at their real sizes. The grey notes give size/line spacing in pixels, then weight, then letter spacing. Every line spacing is a multiple of 4.

TOKEN	SIZE PX	LINE PX	RATIO	WEIGHT	TRACKING EM
`display.lg`	57	64	1.12	600	-0.02
`display.md`	45	56	1.24	600	-0.02
`heading.xl`	36	44	1.22	600	-0.015
`heading.lg`	28	36	1.29	600	-0.01
`heading.md`	22	28	1.27	600	-0.005
`heading.sm`	18	24	1.33	600	0
`body.lg`	18	28	1.56	400	0
`body.md`	16	24	1.5	400	0
`body.sm`	14	20	1.43	400	0
`label.md`	14	20	1.43	500	0.005
`label.sm`	12	16	1.33	500	0.01
`caption`	12	16	1.33	400	0.01

FIG 8.4 The same set as named values. Each style bundles its font, size, line spacing, weight and letter spacing under one name. The ratio column shows line spacing getting tighter as text gets bigger.

Making things stand out takes more than size

A common mistake is trying to show importance with size alone. You end up with a dozen sizes and a page that still feels flat. Weight (bold or regular), colour (dark or grey), capital letters and space around text are just as effective, and they don't add new sizes.

× SIZE IS DOING ALL THE WORK

Invoices

Overdue

3 invoices totalling ₹2,40,000

Last reminder sent 2 days ago

✓ SIZE, WEIGHT, COLOUR AND SPACE TOGETHER

Invoices

OVERDUE

3 invoices totalling ₹2,40,000

Last reminder sent 2 days ago

The version on the right uses fewer sizes but is much clearer, because each level differs from the next in two or more ways at once.

08.4

Text on different screen sizes

A 36-pixel heading looks great on a laptop and takes up half the screen on a phone. There are two ways to handle this.

STEPPED

Different sizes per screen size

"Heading XL" is 28px on phones and 36px on larger screens. Simple, predictable, and easy to show in design files.

FLUID

Sizes that grow smoothly

Text grows gradually between a minimum and a maximum as the screen widens, using a CSS feature called `clamp()`. Tools like Utopia popularised this.[45]

For the engineers reading, here's what a fluid heading looks like in code, with the maths spelled out:

```
/* Heading XL: 28px on a 360px-wide phone → 36px on a 1280px-wide laptop */
--type-heading-xl-size: clamp(1.75rem, 1.554rem + 0.87vw, 2.25rem);

/* growth  = (36 - 28) ÷ (1280 - 360) = 0.0087 → 0.87vw
   start   = 28 - 0.0087 × 360       = 24.87px → 1.554rem */
```

A MISTAKE WE SEE OFTEN

Fluid text that ignores zoom

If text size is based only on screen width, it won't get bigger when someone zooms in, which can fail the accessibility requirement that text can be enlarged to 200%.[13] STANDARD Always include a fixed part (the `rem` part above) in the formula, and only make headings fluid. Keep body text at a steady, readable size.

On phones and tablets, there's another layer. iPhone and Android both let people choose a larger text size for every app on their device. Apple's guidelines encourage apps to support this, which it calls Dynamic Type.[46] A design system for mobile apps should connect each of its text styles to the platform's built-in, adjustable text styles rather than fixing sizes in place.

08.5

Readable for everyone

Accessibility for text isn't a separate project. Most of it is decided by the type system. Here's what to build in from the start:

REQUIREMENT	WHAT IT MEANS FOR YOUR TEXT STYLES	SOURCE
Enough contrast	4.5:1 for normal text, 3:1 for large text. Each text style uses a colour that passes (see Chapter 07).	STANDARD WCAG 1.4.3 [36]
Can be enlarged	Text can be zoomed to 200% without anything being cut off. Use flexible units and let boxes grow.	STANDARD WCAG 1.4.4
Survives spacing changes	Some readers apply their own, wider line and letter spacing. Layouts mustn't break when they do, so avoid boxes with a fixed height around text.	STANDARD WCAG 1.4.12 [47]
Works on narrow screens	Content fits a 320-pixel-wide screen without scrolling sideways (tables and maps excepted).	STANDARD WCAG 1.4.10 [48]
Comfortable line length	Aim for about 45–75 characters per line in long text. Very long lines are tiring to read.	DOCUMENTED Bringhurst [49]
Not too small	Avoid body text below 14px in dense web apps or 16px on mobile web. Keep 11–12px for captions and labels.	PRACTICE

08.6

Different products need different type

The same principles produce quite different results depending on what you're building. A trading dashboard and a meditation app shouldn't have the same text system. Here's how we'd tune typography for common kinds of product. Platform defaults are documented; the rest comes from practice.

PRODUCT	BODY TEXT	RATIO	WHAT MATTERS MOST	WATCH OUT FOR
SaaS web app	14–16px	about 1.2	Clear headings to navigate by; comfortable forms; 10–14 styles in total.	Big marketing-site headings leaking into the app.
Enterprise software	14px (13px dense)	1.125–1.2	Fitting a lot on screen; easy scanning; long working sessions.	Text too small for people with weaker eyesight; offer a roomier mode.
Mobile apps	iOS: 17pt; Android: 16sp	Platform's own	Use the platform's text styles; support users' larger-text settings.[46][44] DOCUMENTED	Fixed-height rows that cut text off when users enlarge it.
Desktop apps	13–15px	1.125–1.2	Fit in with the operating system; dense menus and panels.	Mixing system and brand fonts inconsistently.
Websites that resize	16px	1.2 → 1.25–1.33	Headings that grow with the screen; steady body text; comfortable line length.	Headings that wrap to five lines on phones.
Data-heavy dashboards	13–14px	about 1.125	Numbers that line up; a big-number style for key figures; a monospaced font for IDs.	Everything the same size; use weight and colour to show importance.

IBM's Carbon formalises a similar idea with two sets of text styles: a "productive" set for dense, task-focused screens, and an "expressive" set for marketing and storytelling pages.[50] DOCUMENTED If your company has both a busy app and a marketing site, consider two sets rather than one compromise.

08.7

Turning text styles into tokens

Text uses two levels of named values, just like colour. The first level names individual values: the font family, each weight, each size. The second level bundles them into complete styles named by job, like "heading large". The shared token format has a built-in "typography" type for exactly this kind of bundle. [32] Here's what that looks like:

```
// individual values
"font": {
  "family": { "sans": { "$type": "fontFamily", "$value": ["Schibsted Grotesk", "system-ui", "sans-serif"] } },
  "weight": { "semibold": { "$type": "fontWeight", "$value": 600 } }
},
// a complete style, named by its job
"type": { "heading": { "lg": {
  "$type": "typography",
  "$value": {
    "fontFamily": "{font.family.sans}", "fontWeight": "{font.weight.semibold}",
    "fontSize": { "value": 28, "unit": "px" }, "lineHeight": 1.29,
    "letterSpacing": { "value": -0.01, "unit": "em" }
  } } } }
```

Typography checklist

- [ ] One interface font (plus a monospaced one if needed), tested at small sizes
- [ ] Numbers that line up in tables
- [ ] 10–15 styles named by job, not size
- [ ] Line spacing rounded to multiples of 4px
- [ ] Letter spacing set for headings and labels
- [ ] A plan for screen sizes: stepped or fluid (with zoom support)
- [ ] Layouts survive 200% zoom and readers' own spacing
- [ ] Long text kept to about 45–75 characters per line
- [ ] Mobile styles connected to users' text-size settings
- [ ] The same named styles in Figma and in code

THE SHORT VERSION

What to remember from this chapter

01 Choose a product font for readability at small sizes, aligned numbers and language support first.

02 Use a short, fixed list of sizes, with line spacing that gets tighter as text gets bigger.

03 Name text styles by their job, and show importance with weight, colour and space as well as size.

04 Build accessibility into the styles themselves: contrast, zoom and flexible boxes.

Up next → **Chapter 09: Spacing systems**

CHAPTER 09

09

Spacing systems

Spacing is the foundation people notice least and feel most. It's what makes a screen feel calm or cluttered, organised or confusing. And because everyone decides it "by eye", it's usually the first thing to drift.

READ THIS IF

Your screens feel slightly "off" but nobody can say why, design reviews keep turning into debates about padding, or you need both a compact and a roomy layout.

WHAT YOU'LL LEAVE WITH

01 A base unit and a short list of spacing values, and why they work.

02 A simple way to think about space inside and between things.

03 How to offer compact and comfortable versions of the same screen.

IN THIS CHAPTER

09.1 Why spacing matters so much

09.2 Picking a base unit

09.3 The spacing scale

09.4 Space inside things, space between things

09.5 Compact and comfortable

09.6 Spacing on different screens, and common mistakes

09.1

Why spacing matters so much

Space does three jobs on a screen. It **groups** things: we automatically read items that sit close together as belonging together. It shows **importance**: things with more room around them feel more important. And it creates **rhythm**: a repeating beat that makes a page feel calm and intentional.

When spacing is chosen screen by screen, each value can be defended on its own, but together they're noise: 13 pixels here, 14 there, 22 somewhere else. People using the product don't notice the numbers. They notice that it feels a bit untidy, and they trust it a little less. Look at the same form below, spaced two ways.

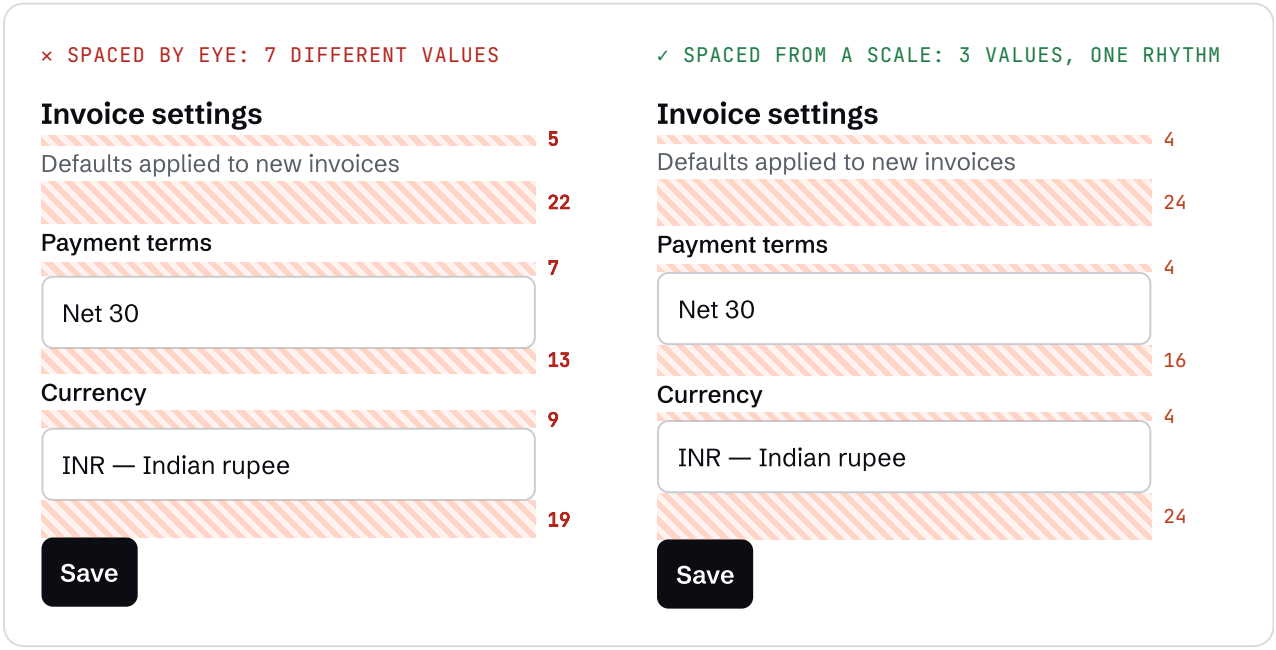


FIG 9.1 The same form, spaced two ways. On the left, the gap between "Payment terms" and its field (7) is almost the same as the gap between fields (13), so labels seem to float between inputs. On the right, three values create three clear relationships: *belongs to* (4), *next item* (16) and *new section* (24).

09.2

Picking a base unit

A spacing system starts with one small number, the **base unit**, and every spacing value is a multiple of it. Think of it like the smallest Lego stud: everything you build is a whole number of studs. The two common choices are 8 pixels and 4 pixels.

8PX BASE

Fewer, clearer choices

Easy to think about, and most screen sizes divide neatly by 8. Atlassian's design system, for example, builds its spacing on an 8px base, with a few smaller steps for tight spots.[51] DOCUMENTED

4PX BASE

Finer control for busy screens

Lets you use 12, 20 and 28, which matter for small components, icons and line spacing. Google's Material uses a 4dp grid for text and icons alongside an 8dp grid for layout.[52] DOCUMENTED

PRACTICE OUR RECOMMENDATION

Use a **4px base with an 8px rhythm**. Every value is a multiple of 4, and most of the values people actually reach for are multiples of 8. You get the precision small components need without the endless choice of "any number will do". Set line spacing for text (Chapter 08) to the same 4px grid, so text and boxes share one rhythm.

09.3

The spacing scale

The scale is the fixed list of spacing values people can choose from. It should have lots of options at the small end, where 4 pixels is a visible difference, and only a few at the large end, where nobody can tell 64 from 68.

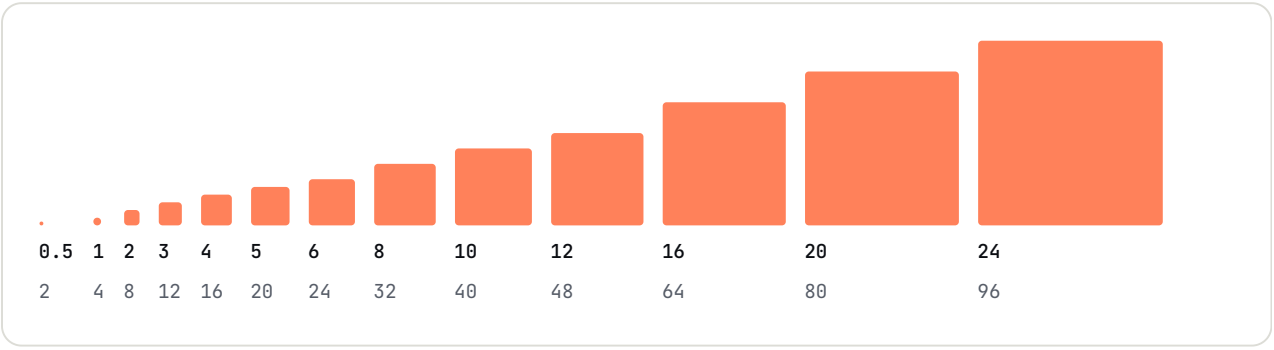


FIG 9.2 Ledgerly's spacing scale. Thirteen values from 2 to 96 pixels, drawn at actual size. Each one's name is just how many 4-pixel units it contains: "space-6" is 6 × 4 = 24px. If you know the name you know the size, and the other way round.

Naming spacing values

Different systems name spacing in different ways. Any of them works, as long as you pick one and stick to it:

STYLE	EXAMPLE	GOOD	NOT SO GOOD
Count of base units	`space-6` = 24px	Easy to guess; easy maths	Changing the base unit renames everything
Multiplier × 100	`space.300` = 3 × 8px (Atlassian) [51]	Room for in-between steps	Less obvious at a glance
Numbered steps	`spacing-06` = 24px (IBM Carbon) [30]	Short; stable order	You have to look up every value
T-shirt sizes	`space-md`	Friendly	Runs out of names (xxxl?); hard to add a size in between

Our advice: choose the style your engineers can read without a lookup table.

09.4

Space inside things, space between things

It helps to think about three kinds of space. Nathan Curtis gave them simple names that have stuck: **inset** (padding inside a box), **stack** (space between things stacked vertically) and **inline** (space between things side by side).[53] Naming the *kind* of space, not just the amount, makes design specs and code reviews much clearer.

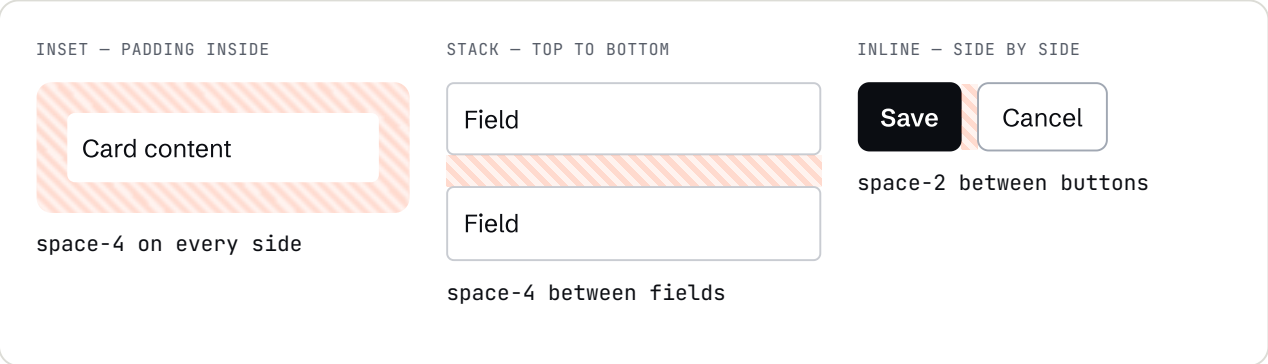


FIG 9.3 Inset, stack and inline. Three kinds of space, each with its own job.

A useful rule of thumb is to use smaller values inside things and larger values between bigger groups:

WHERE THE SPACE IS	TYPICAL VALUES	EXAMPLES
Inside a component	4–16px	Gap between an icon and its label; padding inside a button or input
Between related items	8–16px	A label and its field; one field and the next; list items
Between groups	24–32px	Sections of a form; gaps between cards
Between page sections	40–64px	Major sections of a page
Page edges	16–80px	Grows with screen size (see Chapter 10)

The golden rule: space *within* a group must always be clearly smaller than space *between* groups. A handy way to guarantee it is to jump at least two steps on the scale between levels: 8 → 16 → 24 or 32. That keeps relationships obvious, even when different designers make their own sensible choices. PRACTICE

09.5

Compact and comfortable

Think about two very different people using the same invoices table. An accountant at month-end wants to see 200 rows without scrolling. A small business owner checking one invoice on a Sunday wants things roomy and easy to tap. **Density** is how much fits in a given space, and good systems let you offer both, by switching a few spacing values rather than building separate components.

Compact
32px rows

INV-2041	Acme Ltd	₹84,200
INV-2042	Globex	₹12,900
INV-2043	Initech	₹3,15,000

Default
40px rows

INV-2041	Acme Ltd	₹84,200
INV-2042	Globex	₹12,900
INV-2043	Initech	₹3,15,000

Comfortable
48px rows

INV-2041	Acme Ltd	₹84,200
INV-2042	Globex	₹12,900
INV-2043	Initech	₹3,15,000

FIG 9.4 One table, three densities. Same text, same columns; only the row height and padding change. The compact version fits 50% more rows in the same space as the comfortable one.

There's a floor to how compact you can go, set by people's fingers and by accessibility rules. WCAG 2.2 requires click and tap targets of at least 24 by 24 pixels, with some exceptions.[54] STANDARD For touchscreens, platform guidelines go further: Apple recommends at least 44 by 44 points,[55] and Google's Material recommends 48 by 48.[52] DOCUMENTED So compact modes are for mouse-and-keyboard use on desktop. Don't make them the default on phones.

SETTING	COMPACT	DEFAULT	COMFORTABLE
Height of buttons and inputs	32px	40px	48px
Padding inside buttons and inputs	8px	12px	16px
Padding inside table rows	4px	8px	12px
Space between form fields	12px	16px	24px

09.6

Spacing on different screens, and common mistakes

Here's a simple principle. Spacing *inside* components shouldn't change with screen size: a button has the same padding on a phone and a laptop. Spacing in the *layout* should: the margins around the page and the gaps between sections grow when there's more room. You can build that split into your named values. For engineers, it looks like this:

```
/* The scale itself never changes */
--space-2: 8px;  --space-4: 16px;  --space-6: 24px;  --space-10: 40px;  --space-16: 64px;

/* Layout spacing grows on bigger screens */
--space-page-margin:  var(--space-4);     /* 16px on phones          */
--space-section-gap:  var(--space-10);
@media (min-width: 840px) {
  --space-page-margin: var(--space-6);   /* 24px on larger screens  */
  --space-section-gap: var(--space-16);
}

/* Component spacing changes with density, not screen size */
--space-control-inset-x: var(--space-3);
```

Four mistakes we see again and again

MISTAKE

"Just this once" values

Each one seems harmless. Fifty of them add up to a second, invisible spacing system. Set up an automatic check for spacing values that aren't on the scale.

MISTAKE

The same spacing everywhere

Using 16px for everything removes the grouping signal, and groups disappear. Vary space on purpose.

MISTAKE

Components with built-in outside margins

A component that adds its own space around itself collides unpredictably with others. Let the container decide the gaps between its contents.

MISTAKE

Forgetting about line spacing

A 16px gap between two blocks of loosely spaced text looks bigger than 16px. Keep text on the same 4px rhythm so measured and visible space match.

Spacing checklist

- [ ] A base unit chosen (we recommend 4px with an 8px rhythm)
- [ ] Space within groups clearly smaller than space between them

- [ ] A fixed list of about 12-14 values
- [ ] Names that tell you the size
- [ ] Inset, stack and inline used in specs
- [ ] Compact and comfortable modes; touch targets big enough
- [ ] Component spacing fixed; layout spacing grows with screen size
- [ ] An automatic check for off-scale values

THE SHORT VERSION

What to remember from this chapter

01 Space groups things, shows importance and sets rhythm. It's never just "empty".

02 Pick a base unit (4px is our recommendation) and a short, fixed list of values.

03 Keep space inside groups clearly smaller than space between them.

04 Offer density modes by changing a few values, and respect minimum tap sizes on touchscreens.

Up next → **Chapter 10: Grid and layout systems**

CHAPTER 10

10

Grid and layout systems

A layout system decides where things go on every screen size before anyone designs a single page. Done well, nobody notices it. Done badly, every screen breaks in its own special way on someone's phone.

READ THIS IF

You're deciding how your product adapts to different screens, designing a dashboard, or your pages look fine on your laptop and fall apart elsewhere.

WHAT YOU'LL LEAVE WITH

01 The parts of a layout grid, in plain language, with real numbers.

02 How to choose the screen widths where your layout changes.

03 A practical way to lay out dashboards and complex business software.

IN THIS CHAPTER

10.1 What a grid is

10.2 Where the layout changes: breakpoints

10.3 Stretchy, fixed, or both

10.4 Laying out a dashboard

10.5 Phones, desktops and what lines up with what

10.6 Layouts for complex business software

10.1

What a grid is

Look at a newspaper. The text sits in columns, photos span one, two or three of them, and everything lines up even though dozens of people worked on the page. That's a grid at work. On screens, grids do the same job: they make things line up without anyone measuring.

A layout grid splits the width of the screen into **columns**, with fixed gaps between them called **gutters**. Around the outside there are **margins**, and the whole thing sits inside a **container**. Content spans whole columns (plus the gutters between them), so edges line up across the page.

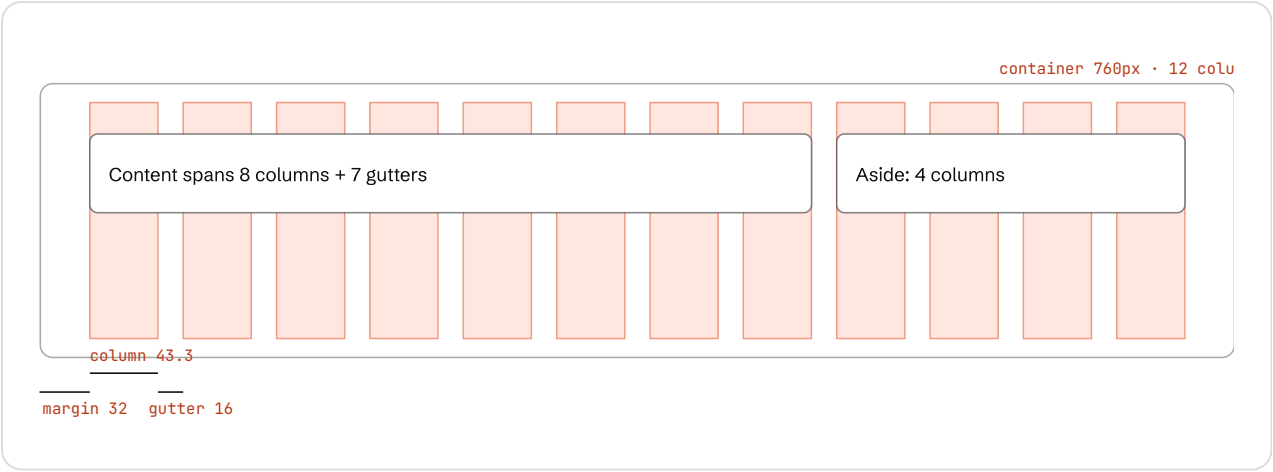


FIG 10.1 The parts of a 12-column grid. A 760px container with 32px margins and 16px gutters leaves columns about 41px wide. Twelve is popular because it divides evenly into halves, thirds, quarters and sixths.

PART	WHAT IT IS	RULE OF THUMB
Columns	The vertical strips content lines up to.	4 on phones, 8 on tablets, 12 on laptops and desktops.
Gutters	The fixed gaps between columns.	Use a value from the spacing scale (16-24px). Gutters stay fixed; columns stretch.
Margins	The space between the grid and the edge of the screen.	Grows with screen size: 16px on phones, up to 32-80px on big screens.
Container	The box the grid lives in, sometimes with a maximum width.	Cap the width of reading pages (around 1200-1440px); let busy apps use the full width.

10.2

Where the layout changes: breakpoints

NEW WORD

Breakpoint

A screen width at which the layout changes, for example from one column to two.

A **breakpoint** is a screen width where your layout rearranges itself: the side menu tucks away, two columns become one, and so on. The trick is to set breakpoints where *your* layout needs to change, guided by common types of device, rather than chasing specific phone models, which change every year. Here's how two of the most mature design systems do it:

Google Material 3 DOCUMENTED

SIZE CLASS	WIDTH
Compact (phones)	< 600
Medium (small tablets)	600–839
Expanded (tablets, small laptops)	840–1199
Large	1200–1599
Extra-large	≥ 1600

Source: Material 3 layout guidance.[56]

IBM Carbon DOCUMENTED

NAME	FROM (PX)	COLUMNS
sm	320	4
md	672	8
lg	1056	16
xlg	1312	16
max	1584	16

Source: Carbon 2x Grid.[57] Its 16 columns suit dense business screens.

Two respected systems, two different sets of numbers, and both work well. The lesson isn't to copy either. Pick four or five breakpoints, give them names rather than device names ("medium", not "iPad"), and share them as named values so design and code agree. Ledgerly uses Material's size classes with 4, 8 and 12 columns.

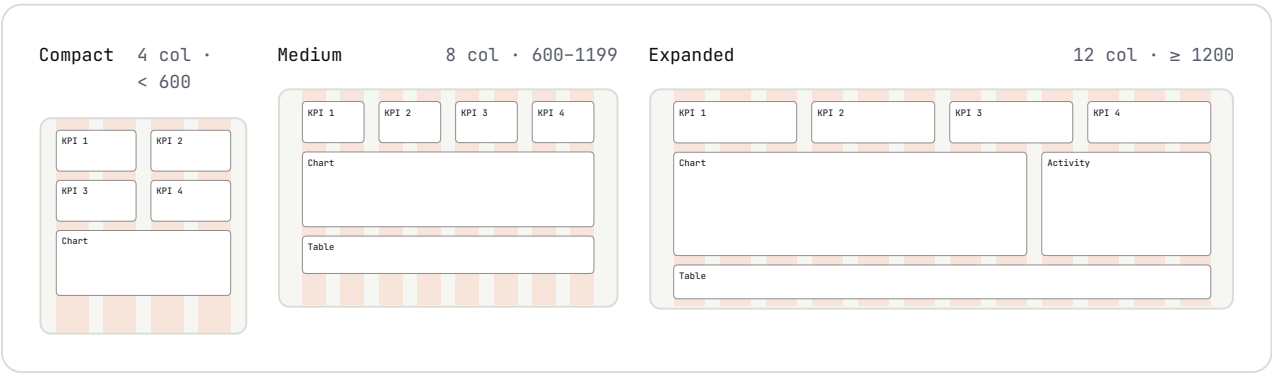


FIG 10.2 The same dashboard on three screen sizes. The content and its order stay the same; what changes is how many columns each block spans. The four key numbers go from two-per-row on a phone to four-per-row on a laptop, and the activity panel only gets its own column when there's room.

10.3

Stretchy, fixed, or both

FLUID

Stretches to fill the screen

Uses all the space. Great for busy apps, but lines of text get uncomfortably long on wide screens.

FIXED

Always the same width

Predictable and easy to design, but wastes space on big screens and needs separate layouts for small ones.

BOTH (RECOMMENDED)

Stretches, up to a limit

Columns stretch with the screen until a maximum width. Reading pages use the limit; busy apps can skip it.

How wide should content be?

The grid decides where things *can* go. Separately, you should decide how wide some things *should* be. Paragraphs are easiest to read at around 45–75 characters per line,[49] so body text should rarely stretch across all 12 columns of a big screen. Forms work best narrow, too. Set these widths as their own named values:

NAMED WIDTH	VALUE	USED FOR
`width-prose`	68 characters	Articles, help pages, long descriptions
`width-form`	560px	Single-column forms and settings
`width-dialog`	560px	Standard pop-up dialogs
`width-page`	1440px	Maximum width for reading and marketing pages
`width-app`	none	Busy app screens that use the full width

10.4

Laying out a dashboard

Most business software isn't really a set of pages on a grid. It's a **frame** (a top bar and a side menu that stay put) around a **content area** that changes. If the design system defines the frame once, every team stops reinventing it, and every product feels like part of the same family.

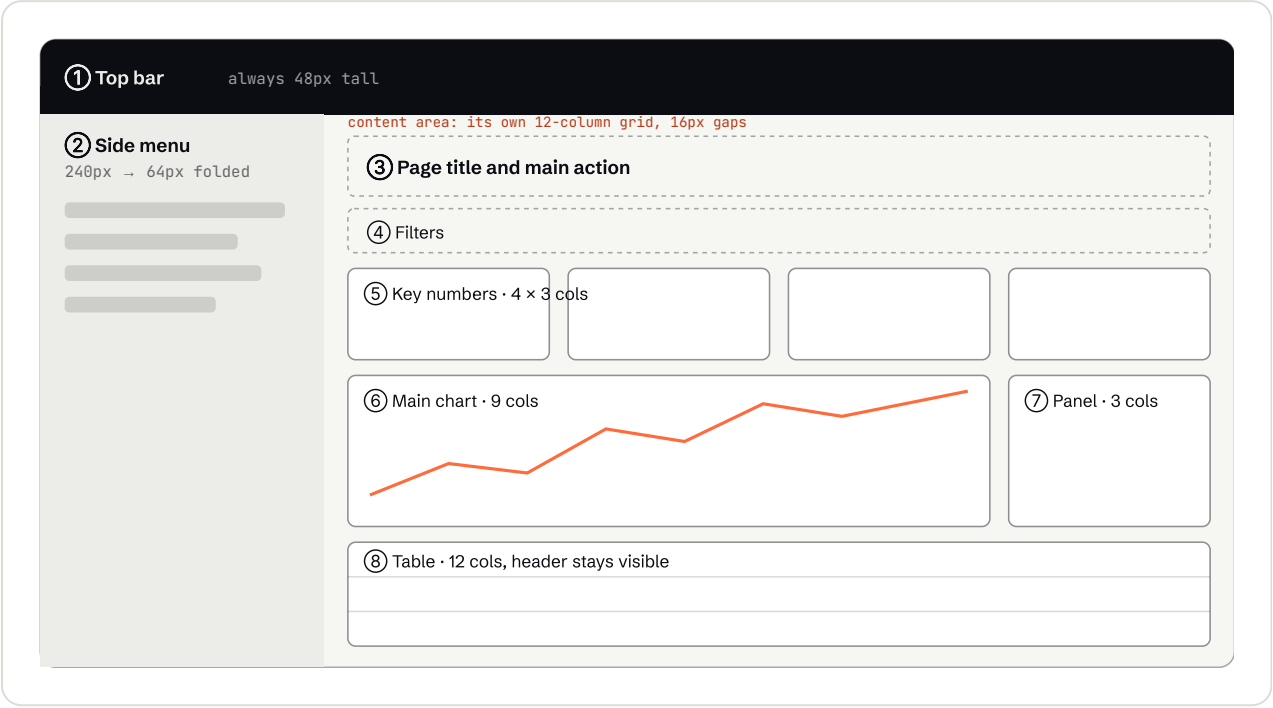


FIG 10.3 The parts of a typical business dashboard. Parts ① and ② are the frame, identical on every page. Parts ③ and ④ follow a standard page pattern. Parts ⑤ to ⑧ sit on the content area's own grid. Each part's size is a named value, not a per-page decision.

Four dashboard rules that always help

01

Most important thing at the top left

In left-to-right languages, people start reading at the top left. Put the number that answers "is everything OK?" there.

02

Cards in a row share a height

Matching heights keep the grid visible and make side-by-side comparison easy.

03

A short list of widths

Allow cards to span 3, 4, 6, 8, 9 or 12 columns. Avoid 5 and 7, which create edges that don't line up with anything.

04

Flip for right-to-left languages

Build with "start" and "end" rather than "left" and "right", so Arabic and Hebrew layouts mirror correctly from the same design.

10.5

Phones, desktops and what lines up with what

A phone layout isn't a shrunken desktop

On a phone, the side menu becomes a bottom bar or a slide-out drawer, side-by-side panels become separate screens with a back button, and wide tables become lists or cards. Decide these changes once, as patterns, and every product gets them for free.

Big things line up to columns; small things line up to each other

Page sections and cards line up to grid columns. Inside a card, elements line up with each other using spacing values. A button doesn't need to snap to a column. Trying to grid everything makes components rigid and awkward.

THE NARROW-SCREEN RULE

WCAG 2.2 requires content to fit a screen 320 pixels wide without scrolling sideways, except for things that genuinely need two dimensions, like data tables, maps and diagrams.[48] STANDARD Plan the 320px version of every pattern. Only ever scroll sideways *inside* a table, never the whole page.

10.6

Layouts for complex business software

Big enterprise tools push past a simple page grid. Think of an email app with a folder list, a message list and a reading pane side by side, or a CRM where you resize panels to fit your work. Your layout system needs words and rules for these too:

Several panes side by side

List and detail views that sit side by side on big screens and become separate screens on small ones. Material documents these as standard layouts.[56]

Panels you can resize or fold

People who use a tool all day expect to resize panels. Set minimum and maximum widths as named values, and remember each person's choice.

Components that fit their space

A component in a narrow side panel should adapt to *its panel*, not the whole screen. A newer CSS feature called container queries makes this possible.

Headers that stay put

Table headers that stay visible while scrolling, first columns that stay fixed, and sideways scrolling inside tables keep people oriented in large data sets.

Layout values, written down

Like everything else, layout decisions become named values that design and code share:

```
"breakpoint": { "medium": "600px", "expanded": "840px", "large": "1200px", "xlarge": "1600px" },
"grid":       { "columns": { "compact": 4, "medium": 8, "expanded": 12 },
               "gutter": "{space.4}", "margin": { "compact": "{space.4}", "expanded": "{space.6}" }
},
"layout":     { "sidebar": { "width": "240px", "folded": "64px" }, "topbar": { "height": "48px" },
               "Width": { "prose": "68ch", "form": "560px", "page": "1440px" } }
```

A note for engineers: CSS variables can't be used inside media query conditions, so breakpoint values are turned into fixed constants (in Sass or JavaScript, for example) rather than CSS variables.

Layout checklist

- [ ] 4-5 named breakpoints, shared by design and code
- [ ] Columns for each (for example 4 / 8 / 12)
- [ ] Gutters and margins taken from the spacing scale
- [ ] Stretchy layouts with a maximum width for reading pages
- [ ] Named widths for text, forms and dialogs
- [ ] The app frame (top bar, side menu) defined once
- [ ] Phone versions of each pattern decided once
- [ ] Everything works at 320px wide (tables excepted)
- [ ] "Start" and "end" instead of "left" and "right"
- [ ] Components that adapt to the panel they're in

THE SHORT VERSION

What to remember from this chapter

01 A grid is columns, gutters, margins and a container. It makes things line up without measuring.

02 Choose a few named breakpoints where your layout genuinely needs to change.

03 Stretch layouts to a limit, and cap the width of text and forms for readability.

04 Define the app frame and the phone versions of patterns once, so every team inherits them.

Up next → **Chapter 11: How foundations work together**

CHAPTER 11

11

How foundations work together

Colour, type, spacing and layout aren't four separate projects. They're one system. The easiest way to see that is to follow a single button from the first decision all the way to the code that ships.

READ THIS IF

You want to see the whole picture in one place, or you need to explain to engineers how design foundations become working components.

WHAT YOU'LL LEAVE WITH

01 One button, traced through every layer from decision to code.

02 What changes, and what doesn't, when a foundation changes.

03 A checklist to confirm your foundations are ready before you build components.

IN THIS CHAPTER

11.1 One button, five layers

11.2 Into the code

11.3 What happens when something changes

11.4 Are your foundations ready?

11.1

One button, five layers

The main "primary" button is the most-used interactive element in most products, which makes it the perfect example. Here's every foundation it depends on, and how each one travels from a decision to the screen.

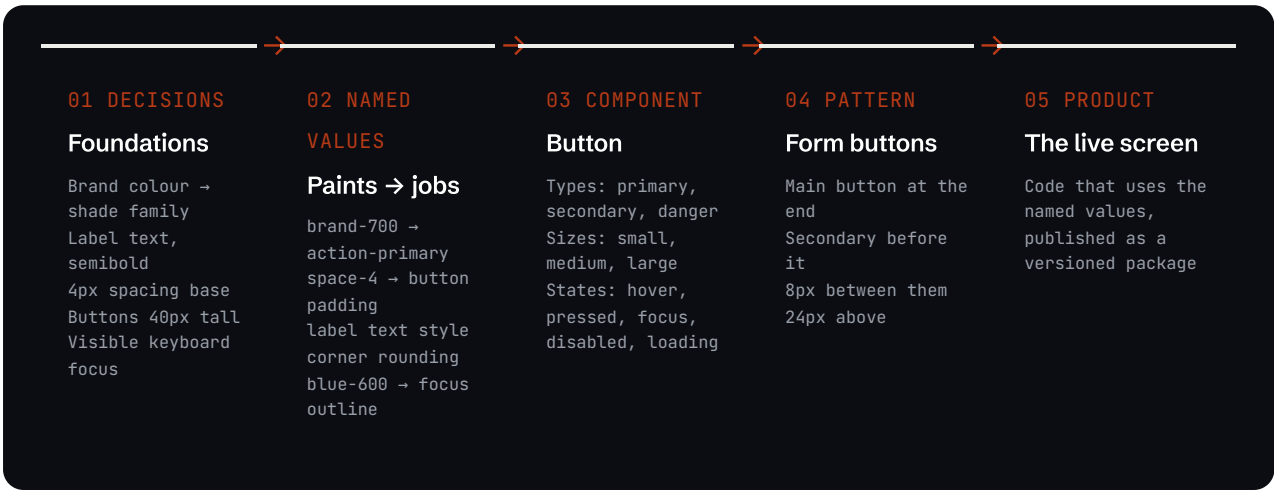
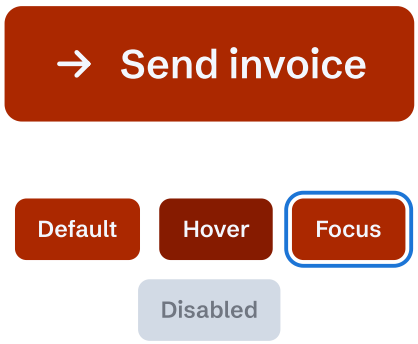


FIG 11.1 From decision to product. Each layer only knows about the layer just below it. The button doesn't know that "brand-700" exists, and the form doesn't know how much padding the button has. That's deliberate: it's what lets you change one layer without breaking the others.

The button, labelled



PART	NAMED VALUE	BECOMES (LIGHT MODE)
Background	`action.primary.bg`	brand 700
Background on hover	`action.primary.bg-hover`	brand 800
Text colour	`action.primary.text`	grey 50 · 6.48:1
Text style	`type.label.md`	14/20 · 600
Height	`size.control.height`	40px
Side padding	`space.control.inset-x`	16px
Icon gap	`space.2`	8px
Corners	`radius.control`	6px
Focus outline	`focus.ring`	blue 600 · 2px

FIG 11.2 **Every visual detail of the button is a named value.** Shown at 1.5× size. Nine details draw on all four foundations: colour (four values), text (one), spacing and size (three) and shape (one). The text contrast, 6.48:1, was calculated from the real colour values in Chapter 07.

11.2

Into the code

This section is mostly for engineers, but it's worth a look even if you don't write code, because it shows the payoff. An automatic build step turns the list of named values into whatever each platform needs. For websites, that's CSS variables. The button's own code then uses only those names. Look closely: there isn't a single colour code or pixel value typed into the button itself.

```
/* tokens.css: generated automatically, never
edited by hand */
:root {
  --color-brand-700: oklch(0.486 0.172 35);
  --color-brand-800: oklch(0.406 0.146 34);
  --color-action-primary-bg:
    var(--color-brand-700);
  --color-action-primary-bg-hover:
    var(--color-brand-800);
  --space-control-inset-x: var(--space-4);
  --size-control-height: 40px;
}
/* dark mode: only the pointers change */
[data-theme="dark"] {
  --color-action-primary-bg:
    var(--color-brand-400);
}
/* compact mode: only the sizes change */
[data-density="compact"] {
  --size-control-height: 32px;
  --space-control-inset-x: var(--space-2);
}
```

```
/* button.css: uses names only */
.button {
  height: var(--size-control-height);
  padding-inline: var(--space-control-inset-
x);
  gap: var(--space-2);
  border-radius: var(--radius-control);
  font: var(--type-label-md);
}
.button[data-variant="primary"] {
  background: var(--color-action-primary-bg);
  color: var(--color-action-primary-text);
}
.button[data-variant="primary"]:hover {
  background:
    var(--color-action-primary-bg-hover);
}
.button:focus-visible {
  outline: 2px solid var(--color-focus-ring);
  outline-offset: 2px;
}
```

iPhone and Android apps get the same values, generated from the same source file into their own languages. The source file is the agreement; each platform's components are just different ways of honouring it.

11.3

What happens when something changes

The real test of all this work is change. Here's what each common change touches, and, just as importantly, what it doesn't.

THE CHANGE	PAINTS	JOBS	COMPONENTS	PRODUCTS	THE WORK INVOLVED
Rebrand with a new colour	●	○	○	○	Change one starting colour; shades regenerate; check readability; release.
Add dark mode	○	●	○	○	One new list of pointers; check every pairing; release.
Make secondary text more readable	○	●	○	○	Change one pointer; every product is fixed when it updates.
Add a compact mode	○	●	◖	○	New spacing values; check components with fixed internal sizes.
New set of text sizes	●	◖	○	◖	Values change; products check any layouts that relied on old sizes.
White-label for a customer	●	○	○	○	Generate a brand family from the customer's colour.

● changes · ◖ worth checking · ○ untouched. These results only hold if components use named values everywhere. A single colour code typed directly into a component breaks the chain for that component.

PART 02 IN ONE SENTENCE

Decide once, name everything, and let components use only the names. Then almost every future change becomes a settings change instead of a rebuild.

11.4

Are your foundations ready?

Before you start building components at scale, run through this list. Each item points back to the chapter that explains it.

Colour ch. 07

- [ ] Shade families built in OKLCH, with colour codes generated by a tool
- [ ] Job names for backgrounds, text, borders, icons, actions, status and focus
- [ ] Light and dark lists, with every pairing checked automatically

Typography ch. 08

- [ ] Text styles named by job, saved as bundles
- [ ] Line spacing on the 4px grid; numbers that line up
- [ ] A plan for screen sizes that respects zoom

Spacing ch. 09

- [ ] A 4px-based scale with names that tell you the size
- [ ] Component spacing with compact and comfortable modes
- [ ] Tap targets big enough for fingers

Layout ch. 10

- [ ] Named breakpoints, columns, gutters and margins
- [ ] The app frame and content widths defined
- [ ] Every pattern planned for a 320px screen

Plumbing ch. 06

- [ ] One source file of named values, turned automatically into code and Figma
- [ ] An automatic check against colour codes typed into product code

> ## Tokens first, components second.
>
> Rule #4 — Layer One

THE SHORT VERSION

What to remember from this chapter

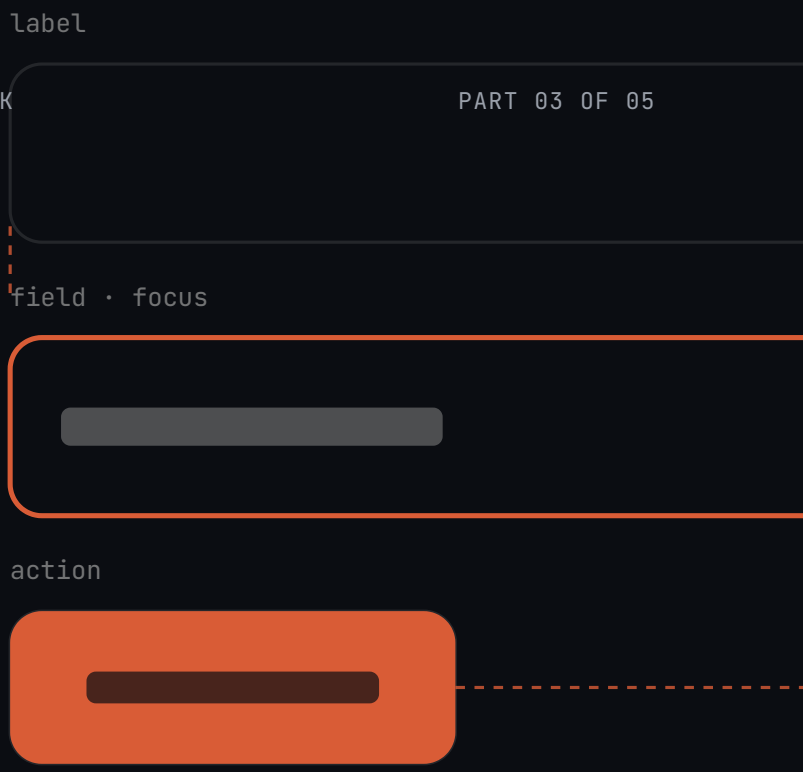
01 Every visual detail of a component should come from a named value.

02 Each layer only knows about the one below it, and that's what makes change safe.

03 Rebrands, dark mode and readability fixes become changes to names, not to components.

04 With foundations in place, you're ready for components, which is where Part 03 begins.

Up next → **Part 03: Components**



03

Components: designing, building and looking after them

Components are the part of a design system people actually touch every day: the buttons, fields, menus and tables everyone builds with. This part covers how to decide which ones to build, how to design and build them so they last, and how to keep a growing library easy to find, trust and use.

12
Components, explained

13
Deciding what to build first

14
Designing a component that lasts

15
Building components in code

16
A library people can actually use

CHAPTER 12

12

Components, explained

Buttons, text fields, menus, tables: components are what most people picture when they hear "design system". This chapter takes one apart, piece by piece, so you understand what you're actually building before you build a single one.

READ THIS IF

You're new to components, you're about to plan a component library, or you've heard words like "variant", "prop" and "slot" and want them explained simply.

WHAT YOU'LL LEAVE WITH

01 What a component really is, in design and in code.

02 The vocabulary: parts, options, states and slots.

03 A realistic sense of how many components a product needs.

IN THIS CHAPTER

12.1 What a component is

12.2 Anatomy: the parts of a component

12.3 Options: types, sizes and settings

12.4 States: one component, many situations

12.5 Small parts and big parts

12.6 How many components do you need?

12.1

What a component is

Think about the light switches in your home. You don't design a new switch for every room. You pick one model, and every room gets the same one. It works the same way everywhere, you know exactly what it does, and if a better model comes out, you can swap them all.

A component is the same idea for software: a piece of interface that's **designed once, built once and used everywhere**. A button. A text field. A dropdown menu. A date picker. A table.

The important part, and the part people often miss, is that a design system component exists *twice*, as a pair of twins. One twin lives in the design tool, where designers drag it into their screens. The other lives in code, where engineers use it to build the real product. The two twins have the same name, the same options and the same behaviour. When the twins match, a designer's screen and the shipped product match too.

THE DESIGN TWIN

In Figma (or similar)

A "master" that designers place into screens. Change the master and every copy in every design file can update.

THE CODE TWIN

In the code package

A piece of code engineers use with one line, like `<Button variant="primary">Save</Button>`. Improve it and every screen gets the improvement on the next update.

SCENARIO

When the twins don't match

THE SITUATION

The Figma button has four types: primary, secondary, tertiary and danger. The code button has three, and calls them "solid", "outline" and "ghost".

WHAT GOES WRONG

A designer uses "tertiary" in a design. The engineer guesses it means "ghost". It doesn't quite match. Design review flags it. Everyone loses half a day.

THE FIX

One list of names and options, agreed once and used in both places. Chapter 15 covers how to keep the twins in step.

12.2

Anatomy: the parts of a component

Even a simple component is made of several parts, and naming them makes design discussions much faster. "The helper text is too close to the field" is a lot clearer than "that grey bit underneath is a bit squashed". Here's an ordinary text field, taken apart.

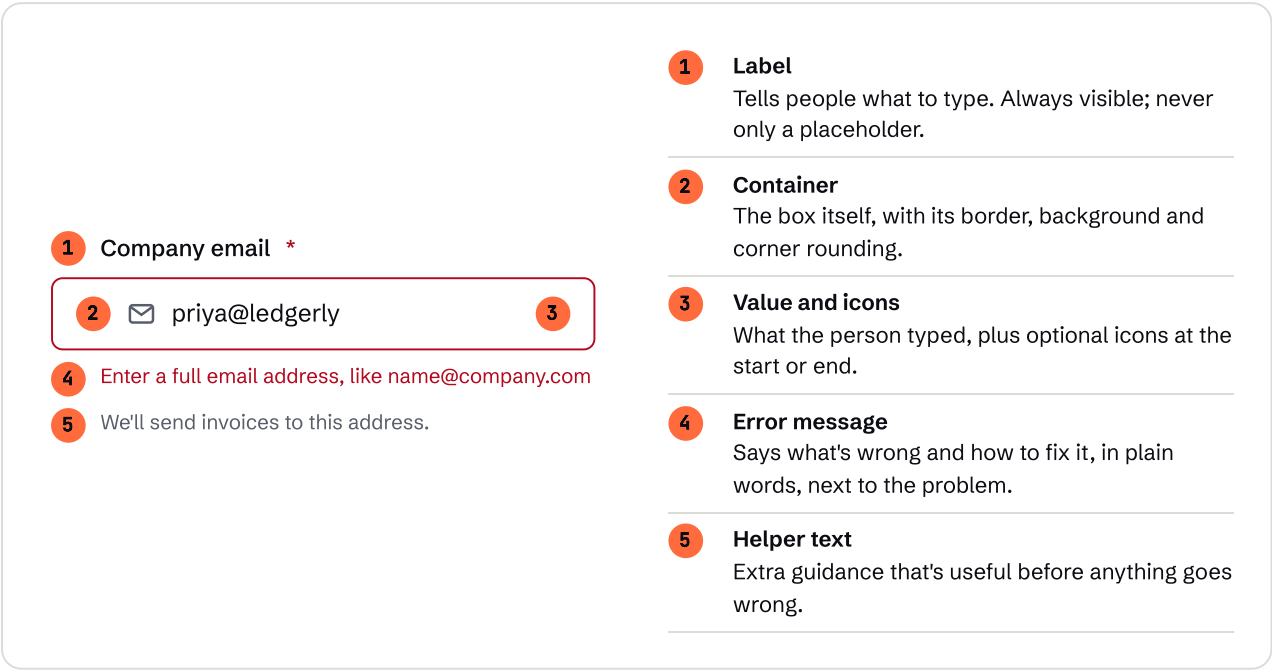


FIG 12.1 The anatomy of a text field. Five named parts. Each one has its own rules for colour, size and spacing, all taken from the foundations in Part 02. This field is shown in its error state, which is when the anatomy matters most.

Every component in your system should have an anatomy diagram like this in its documentation. It gives designers, engineers and writers a shared map, and it's where many accessibility decisions get made. In the example, the label is always visible, even while typing, because placeholder text that disappears as you type is a common usability and accessibility problem.

12.3

Options: types, sizes and settings

NEW WORD

Variant

An approved version of a component, like a primary or a secondary button. Same component, different look for a different purpose.

A single component usually comes in a few approved versions. A button might come in a **primary** version for the main action on a screen, a **secondary** version for less important actions, and a **danger** version for things like "Delete account". It might also come in three sizes. Designers call these **variants**; engineers often call the settings that control them **props** (short for "properties"). They're the same idea: the switches and dials on a component.

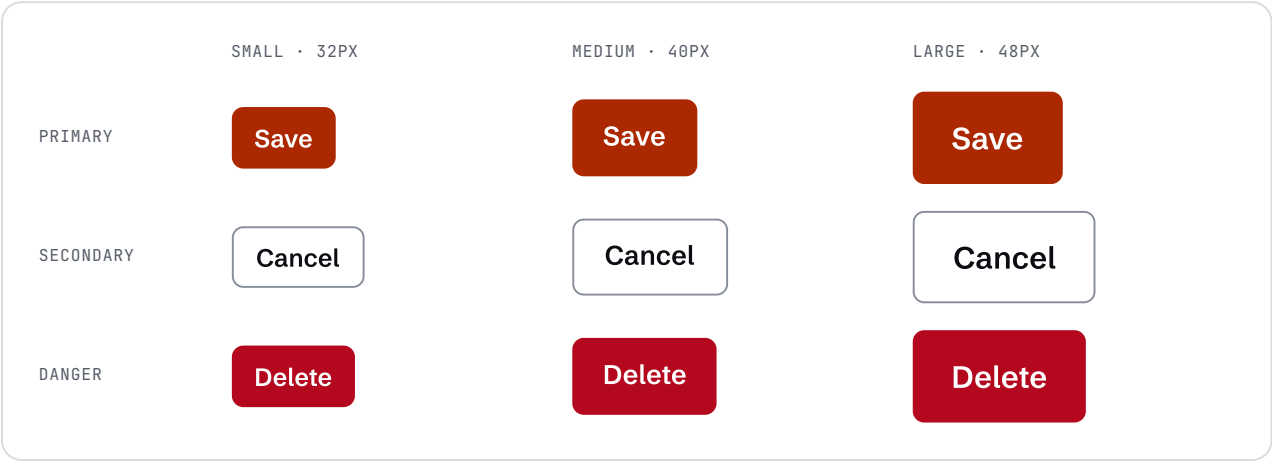


FIG 12.2 One button, nine combinations. Three types × three sizes, all from a single component. In code this is one line with two settings: `<Button variant="danger" size="sm">`.

Options are powerful, but they're also where components go wrong. Every option you add is something to design, build, test, document and maintain, for every combination. Three types and three sizes is nine combinations. Add "with icon" and "loading" and it's thirty-six. That's why mature systems are careful about adding options. A good question to ask before adding one: *is this a real, repeated need in more than one team, or a one-off for a single screen?*

A MISTAKE WE SEE OFTEN

The button with forty settings

Every team asks for one small option: a different colour, a different corner, a special icon position. Each request is reasonable, and the system team says yes each time. Two years later the button has forty settings, nobody understands them all, and it's slower to use the system button than to build your own. Saying "no" (or "build that locally") is part of looking after a component.

12.4

States: one component, many situations

A component doesn't just sit there looking nice. People hover over it, click it, tab to it with the keyboard, type into it, make mistakes in it. Each of these situations is called a **state**, and each one needs to be designed. Here's a text field in eight of them.

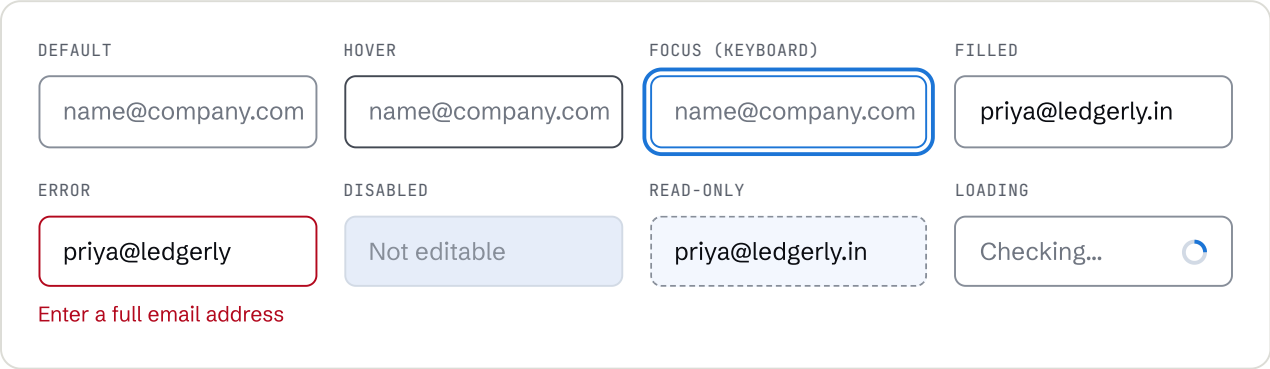


FIG 12.3 Eight states of a text field. Every state uses colours from the job names in Chapter 07. The focus state uses the dedicated focus colour, so keyboard users can always see where they are.

When a component looks broken in production, it's very often because a state was never designed. Nobody thought about what happens when the text is too long, or when the data is still loading, or when there's an error and the field is also disabled. Designing every state up front is one of the biggest quality gains a design system gives you. Chapter 14 includes a full checklist.

12.5

Small parts and big parts

Some components are small and simple: a button, a checkbox, a badge. Others are made from several smaller ones: a search bar is a text field, an icon and a button; a data table is made of headers, cells, checkboxes and pagination. Brad Frost's *Atomic Design* popularised this idea of building big things from small things, a bit like atoms making molecules.[2]

NEW WORD

Slot

An empty space inside a component where you can put other content, like a picture frame you fill yourself.

For bigger components, there's an important design choice. Do you give the component lots of settings to cover every case, or do you give it **slots**, empty spaces where teams can put their own content? A card is a good example. Instead of building "card with image", "card with avatar", "card with chart" and so on, you build one card with a few slots and let teams fill them.



One card, endless uses

The card decides the *frame*: its corners, padding, border, shadow and the spacing between slots. Teams decide what goes *inside*. The billing team puts a chart in the media slot; the customers team puts an avatar there. Both cards still look like they belong to the same product.

Fewer components, fewer settings, more flexibility. That's the trade slots give you.

FIG 12.4 **A card built with slots.** The dashed areas are slots. The card controls the frame and spacing; teams supply the content.

Slots aren't always the answer. A button shouldn't have a slot for "anything you like", because then someone puts a dropdown inside it. As a rough rule: small, frequently used components should be strict, with a few clear options; larger, container-like components should be flexible, with slots. Alla Kholmatova's book *Design Systems* describes this as a balance between strict and loose systems, and points out that more modular isn't automatically better.[58]

12.6

How many components do you need?

Fewer than you'd think, at least to start. Most products are built from a surprisingly small set of everyday components. The table below is a typical starting set for a business web app. It's not a rule, but if you're planning your first year, it's a realistic shape. PRACTICE

GROUP	COMPONENTS	COUNT
Actions	Button, icon button, link	3
Form inputs	Text field, text area, select, checkbox, radio, switch, date picker, search	8
Feedback	Alert / banner, toast, inline message, progress, spinner, empty state	6
Containers	Card, dialog, side panel, accordion, tabs	5
Navigation	Top bar, side menu, breadcrumbs, pagination, menu	5
Data display	Table, badge, tag, avatar, tooltip	5
Total		**32**

Systems grow over time, of course. At IBM, Layer One's founder worked on systems with 71 components for an airline and more than 40 for a pharmacy storefront.[1] But even those started with the everyday basics. The order you build them in matters a lot, and that's the subject of the next chapter.

THE SHORT VERSION

What to remember from this chapter

01 A component is designed once, built once and used everywhere, and it exists as twins in design and code with identical names and options.

02 Name a component's parts (its anatomy) so everyone can talk about it precisely.

03 Options multiply effort; add them only for real, repeated needs.

04 Design every state, not just the default. Use slots for big, container-like components.

Up next → **Chapter 13: Deciding what to build first**

CHAPTER 13

13

Deciding what to build first

The most common way to waste the first six months of a design system is to start with the components that are fun to design instead of the ones teams actually need. This chapter shows how to find out what you have, decide what matters, and say "not yet" to the rest.

READ THIS IF

You're planning which components to build, deciding between building your own and using an open-source library, or drowning in requests.

WHAT YOU'LL LEAVE WITH

01 A step-by-step way to take stock of the interface you already have.

02 A simple way to rank what to build first.

03 A realistic first-year plan, and the confidence to say "not yet".

IN THIS CHAPTER

13.1 Start with what you already have
13.2 Count what matters
13.3 Rank it: what to build first
13.4 Build, borrow or buy?
13.5 Saying "no" and "not yet"
13.6 A realistic first year

13.1

Start with what you already have

Before you design anything new, find out what already exists. Brad Frost calls this an **interface inventory**: you go through your product, screenshot every distinct version of every interface element, and group them by type.[59] In his original write-up he describes screenshotting the buttons from just a few pages of a bank's website and finding a startling variety of styles. That's typical. Every product we've audited has been the same.

SCENARIO **"We have one date picker." (They had three.)**

THE SITUATION

Ledgerly's team believes their product is fairly consistent. They spend two afternoons doing an inventory before starting their design system.

WHAT THEY FIND

7 kinds of primary button, 3 date pickers (one from an old library, two custom), 4 different "something went wrong" messages, 11 shades of border grey, and 2 tables that sort differently.

WHY IT MATTERS

Now they know where the pain is, they have "before" numbers to show progress, and leadership can *see* the problem on one wall. The inventory became the business case.

How to run an inventory in a week

01 **Pick your scope**

Your top 15–25 screens by traffic or importance, across every product and platform. You don't need every screen to see the pattern.

02 **Screenshot by category**

Buttons, form fields, menus, messages, tables, cards, navigation, icons, colours, text styles. One slide or board per category.

03 **Group the near-duplicates**

Put things that are *meant* to be the same next to each other. The differences will jump out.

04 **Check the code too**

Ask engineers to count how many different implementations exist for the main components, and how many raw colour values are in the code.

05 **Show it to people**

Walk leadership and product teams through the boards. It's the fastest way to build shared understanding of the problem.

PRACTICE **DO IT TOGETHER**

Run the inventory as a team exercise with designers *and* engineers, not as a solo audit. People who find the seven buttons themselves become supporters of the system. People who are shown a report about their seven buttons become defensive.

13.2

Count what matters

An inventory tells you *what* exists. To decide what to build first, you need two more pieces of information for each component: **how often it's used** and **how much trouble it's causing** today.

How often is the easy one. Engineers can count how many times each kind of component appears in the code, and designers can count in their design files. Trouble is a bit more judgement: how many different versions are there, how many bugs and accessibility problems have been reported, and how much time do teams spend rebuilding or debating it?

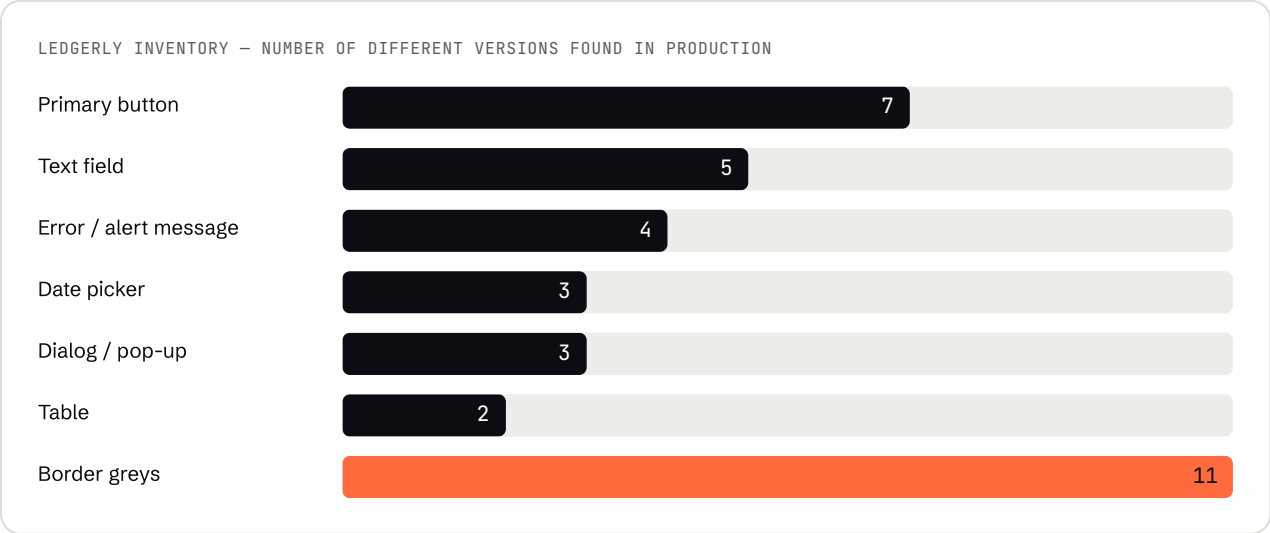


FIG 13.1 What an inventory typically reveals. ILLUSTRATIVE These are Ledgerly's numbers, from our fictional example, but the shape is typical: the most common components have the most versions, because they're built most often by the most people.

Notice something about the chart: the components with the most versions are usually the most common ones. That's not a coincidence. The more often something is built, the more chances there are for it to drift. Which is good news, because it means fixing a small number of everyday components fixes a large share of the inconsistency.

13.3

Rank it: what to build first

Put your two pieces of information together and you get a simple map. Plot each component by how often it's used and how much trouble it causes today.

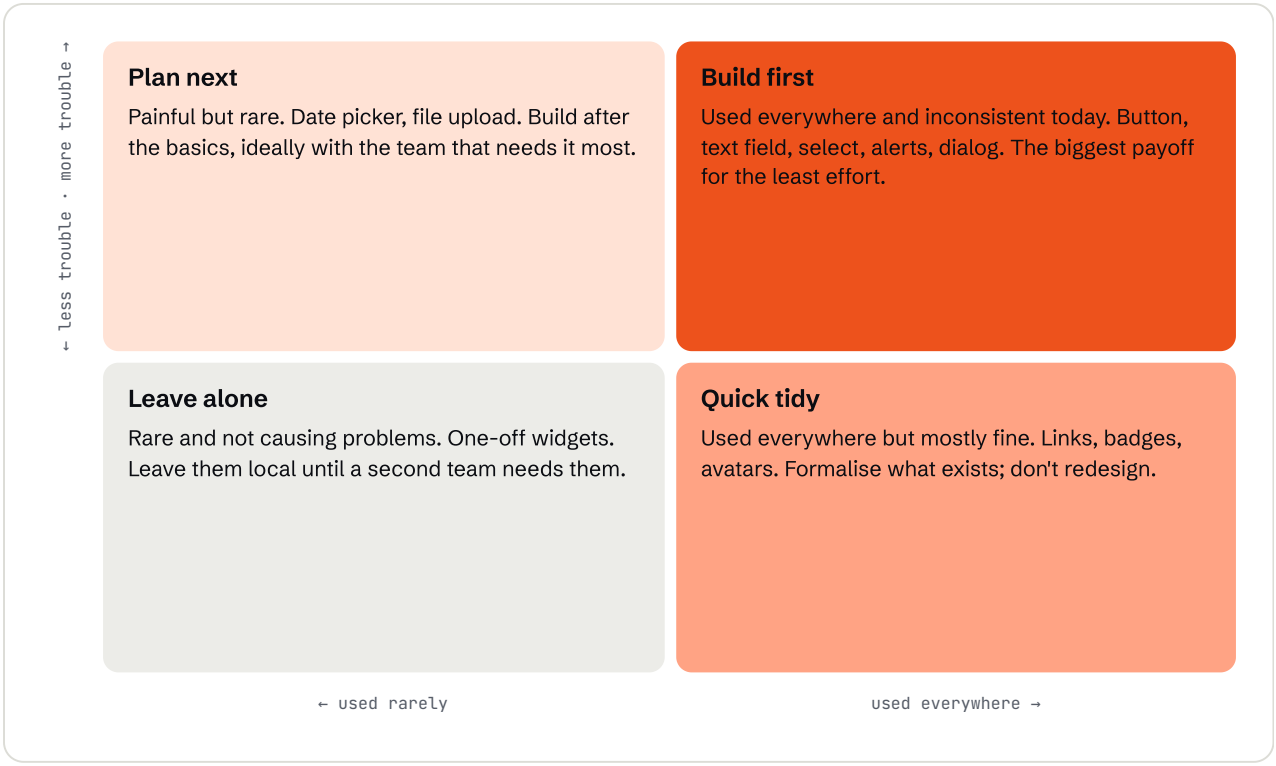


FIG 13.2 A simple way to rank components. PRACTICE Top right first. Most teams find their first ten components land there, and they're almost always the boring ones: buttons, fields, messages and dialogs.

This is less exciting than designing a fancy data visualisation component, and that's exactly the point. The boring components are where the most time is wasted and the most inconsistency lives. Nathan Curtis suggests a similar exercise for teams starting a system, which he calls "picking parts, products and people": agree on the ten most important parts, the products that will use them first, and the people who'll do the work.[60]

A MISTAKE WE SEE OFTEN

Starting with the hardest component

A team decides to begin with the data table "because it's the biggest problem". Six months later they have a sophisticated table, nothing else, and product teams have stopped waiting. Start with components you can finish in days, so teams see progress quickly, and tackle the big ones once you have momentum and a working process.

13.4

Build, borrow or buy?

You don't have to build every component from scratch. There are excellent open-source libraries, and one kind in particular is worth knowing about.

NEW WORD

Headless component

A component that handles behaviour (keyboard use, focus, accessibility) but has no visual style, so you can apply your own.

Headless libraries, such as Radix, provide components that work correctly (keyboard navigation, focus handling, screen-reader support) but come without any visual styling. You add your own look using your foundations. It's a bit like buying a well-engineered car chassis and designing your own body for it. Building accessible behaviour for complex components like menus, dialogs and comboboxes is genuinely hard; the W3C's own guide to these patterns runs to dozens of pages.[61] Borrowing tested behaviour can save months.

APPROACH	GOOD FOR	WATCH OUT FOR
Build your own	Full control; exactly your needs; no outside dependency.	Slowest. Accessible behaviour for complex components is hard to get right.
Headless library + your styles	Correct, accessible behaviour out of the box; your own look; much faster.	You depend on the library's updates and its way of doing things.
Fully styled library (themed)	Fastest to start; great for early-stage products and internal tools.	Hard to make it look truly yours; heavy theming can be harder than building.

PRACTICE **WHAT WE USUALLY RECOMMEND**

For most product companies: build simple components yourself (buttons, badges, cards), because they're easy and you'll want full control. Use a headless library for complex interactive ones (menus, dialogs, comboboxes, date pickers), because the behaviour is the hard part. Use a fully styled library only before product-market fit, or for internal tools where brand matters less.

13.5

Saying "no" and "not yet"

Once a design system exists, requests arrive constantly. "Can we have a special pricing card?" "Can the button have a gradient for the campaign page?" "We need a timeline component." A system team that says yes to everything ends up with a bloated, inconsistent library, which is exactly the problem it was created to solve.

Saying no doesn't mean refusing to help. It usually means one of three answers:

"USE THIS INSTEAD"

It already exists

Often an existing component, or a combination of them, already solves the problem. Showing how is a quick win for everyone.

"BUILD IT LOCALLY"

It's a one-off

If only one team needs it, they build it from system parts and own it. If a second team needs it later, it becomes a candidate for the core.

"YES, NEXT QUARTER"

It's real but not urgent

It goes on the public roadmap with an honest timeline, so the team can plan around it.

This is the rule of two from Chapter 05 in action: something enters the core when at least two teams need it. Brad Frost describes product-specific combinations that stay outside the core as "recipes",[21] and Nathan Curtis describes the same idea as tiers below a high-quality core.[20] Whatever you call it, having a respectable place for "not in the core" components is what makes it possible to say no without being unhelpful.

13.6

A realistic first year

Here's how Ledgerly might plan its first year, using everything in this chapter. Notice that each quarter delivers something teams can use straight away, and that the work is always tied to a real product project, a pilot, rather than built in isolation. Dan Mall argues that pilots are the best way to start and sustain a design system, because they keep the system honest about real needs.[29]

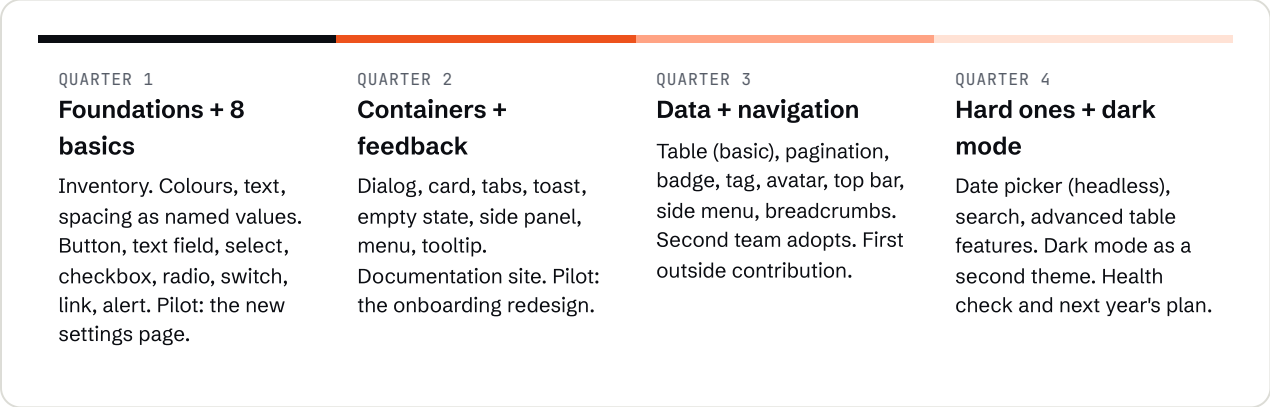


FIG 13.3 Ledgerly's first year. ILLUSTRATIVE About 32 components plus foundations, each quarter tied to a real product project. The hardest components come last, once the process is running smoothly.

THE SHORT VERSION

What to remember from this chapter

01 Start with an interface inventory, done together with designers and engineers.

02 Rank components by how often they're used and how much trouble they cause. The boring everyday ones usually win.

03 Borrow tested behaviour for complex components; build simple ones yourself.

04 Have a respectable home for "not in the core" components, so you can say "not yet" without being unhelpful.

Up next → **Chapter 14: Designing a component that lasts**

CHAPTER 14

14

Designing a component that lasts

A component in a design system is used in hundreds of places by people you'll never meet, with content you'll never see. Designing one is less about making it look good in a mock-up and more about predicting everything that will happen to it in real life.

READ THIS IF

You design components, review them, or keep finding that system components break when real content and real people get hold of them.

WHAT YOU'LL LEAVE WITH

01 A process that starts from real use, not from a blank canvas.

02 Checklists for states, edge cases and accessibility.

03 A clear "ready to build" standard for handing a component to engineering.

IN THIS CHAPTER

14.1 Start from real use, not a blank canvas

14.2 Design every state and every edge

14.3 Accessible from the start

14.4 The words are part of the component

14.5 "Ready to build": what a finished design includes

14.6 Test it in the real product

14.1

Start from real use, not a blank canvas

When designers start a new component, the instinct is to open a blank frame and design the ideal version. For a system component, that's the wrong starting point. The right one is the inventory from Chapter 13: every version of this component that already exists, and every situation it's used in.

SCENARIO **The alert that looked perfect until it met real messages**

THE SITUATION

A designer creates a beautiful alert component. In the mock-up, every message is one short line: "Changes saved."

WHAT GOES WRONG

In the real product, messages include "Your payment of ₹1,24,000 to Acme Supplies could not be processed because the bank declined the transaction. Try another payment method." The alert wraps onto four lines, the icon floats in the middle, and the close button overlaps the text.

WHAT WOULD HAVE HELPED

Collecting twenty real alert messages from the product *before* designing, and testing the design with the longest, shortest and strangest ones.

So before you design, gather the evidence:

- [ ] **Every existing version** of the component, from the inventory, and what each one was trying to do.
- [ ] **Real content**: actual labels, messages and data from the product, including the longest and the oddest.
- [ ] **Where it's used**: in a table, in a dialog, on a phone, in a dense dashboard, in a marketing page.
- [ ] **What teams have asked for**: open requests and complaints about the current versions.
- [ ] **How others do it**: look at how mature public systems handle the same component, for ideas and pitfalls.

14.2

Design every state and every edge

In Chapter 12 we saw a text field in eight states. Most components have more situations than people expect, and the ones you forget are the ones that end up looking broken in production. Here's a checklist we use for every component.

SITUATION	QUESTIONS TO ANSWER
Interaction states	Default, hover, pressed, keyboard focus, selected, disabled, read-only. What does each look like?
Data states	Empty, loading, partly loaded, error, success. What shows while we wait? What if there's nothing to show?
Too much content	A very long name, a huge number, 300 items in a list. Does it wrap, cut off with "…", or scroll?
Too little content	A one-character name, a single item. Does it still look intentional?
Other languages	Some languages need much more space than English for the same message; some read right to left. Does the layout survive?
Screen sizes	What happens on a phone? In a narrow side panel? On a very wide screen?
Bigger text and zoom	If someone sets their text size larger, or zooms to 200%, does anything get cut off?
Combinations	Error *and* disabled? Loading *and* selected? Which combinations are possible, and which should be prevented?

■ PRACTICE **THE "STRESS TEST" FRAME**

Keep a page in your design file next to every component called "stress test". Put the component in it with the longest real content, the shortest, an empty state, an error, a translated version and a phone-width version. Review that page, not the pretty one. It takes an hour and saves weeks of bug reports.

14.3

Accessible from the start

Layer One's fifth rule is *accessible by default, not by audit*. For components, this matters even more than anywhere else, because every accessibility problem in a component is copied into every screen that uses it. The good news is that the reverse is also true: get a component right once, and every screen gets it right.

Accessibility can sound intimidating, so here it is in plain terms. For each component, make sure that:

EVERYONE CAN…	WHICH MEANS	SOURCE
See it	Text has enough contrast (4.5:1), and so do the edges of fields and buttons (3:1). Information never relies on colour alone.	STANDARD WCAG 1.4.3, 1.4.11, 1.4.1[13]
Use it with a keyboard	Every action can be reached with Tab and triggered with Enter or Space. Menus and dialogs follow the standard keyboard patterns.	STANDARD WCAG 2.1.1; ARIA patterns[61]
See where they are	A clear focus outline appears when moving with the keyboard.	STANDARD WCAG 2.4.7
Hear what it is	Screen readers announce what it is ("button"), its name ("Send invoice") and its state ("expanded", "selected").	STANDARD WCAG 4.1.2
Hit it	Click and tap targets are at least 24 × 24px, and 44–48 on touchscreens.	STANDARD WCAG 2.5.8[54]
Understand errors	Error messages say what's wrong and how to fix it, and are connected to the field they describe.	STANDARD WCAG 3.3.1, 3.3.3
Avoid discomfort	Animations respect the "reduce motion" setting on people's devices.	PRACTICE

For complex interactive components like menus, tabs, dialogs and comboboxes, don't invent the keyboard behaviour. The W3C's ARIA Authoring Practices Guide documents the expected behaviour for each of these patterns, including which keys do what, and it's the reference most accessible component libraries follow.[61] DOCUMENTED

SCENARIO

The dialog you couldn't escape

THE SITUATION

A team builds its own confirmation dialog. It looks great and works perfectly with a mouse.

WHAT GOES WRONG

Keyboard users press Tab and the focus goes *behind* the dialog into the page. Escape doesn't close it. Screen readers don't announce it. It ships in 40 places before anyone notices.

THE SYSTEM FIX

One dialog component with focus kept inside, Escape to close, focus returned afterwards, and a proper announcement. Fixed once; every one of the 40 dialogs inherits it.

14.4

The words are part of the component

A button isn't finished when its colour and padding are decided. What it *says* matters just as much. Good systems include writing guidance alongside each component, because consistent words are as important as consistent pixels.

COMPONENT	INSTEAD OF…	WRITE…	WHY
Button	"Submit", "OK", "Yes"	"Send invoice", "Delete 3 files"	Say what will happen. People often read only the button.
Error message	"Invalid input"	"Enter a date after 1 January 2026"	Say what's wrong *and* how to fix it.
Empty state	"No data"	"No invoices yet. Create your first invoice to get paid faster."	Explain why it's empty and what to do next.
Confirmation	"Are you sure?"	"Delete this customer? Their 12 invoices will also be deleted."	Spell out the consequence.
Label	Placeholder text only	A visible label above the field	Placeholders disappear when you type, and are often low-contrast.

You don't need a full content style guide on day one. Start with one or two sentences of writing guidance on each component's documentation page, plus a handful of real examples. Those few lines will do more for consistency than a 40-page tone-of-voice document nobody reads.

14.5

"Ready to build": what a finished design includes

When is a component design ready to hand to engineering? Many disagreements between designers and engineers come down to different answers to that question. Agree on one checklist, and use it every time.

The design is ready when it has…

- [ ] **An anatomy diagram** with every part named
- [ ] **Every option** (types, sizes) and which combinations are allowed
- [ ] **Every state**, including empty, loading and error
- [ ] **The stress test:** long, short, translated and phone versions
- [ ] **Named values only:** every colour, size and space is a token, not a number

…and also

- [ ] **Behaviour notes:** what happens on click, keyboard use and resize
- [ ] **Accessibility notes:** keyboard pattern, focus, what screen readers announce
- [ ] **Writing guidance:** one or two lines plus examples
- [ ] **When to use it**, and when to use something else instead
- [ ] **An engineer's review** of the design before building starts

That last item is the most important. Bring an engineer in while the component is being designed, not after. A ten-minute conversation ("can the menu really open upwards when there's no space below?") prevents a week of rework. The best system teams pair a designer and an engineer on every component from the first sketch.

14.6

Test it in the real product

A component isn't proven until it's been used in a real product by a team that didn't design it. That's why pilots matter so much (see Chapter 05). Before calling a new component stable, put it into one or two live product screens and watch what happens.

01 **Release it as "beta"**

Label it clearly as new and still settling, so teams know it might change slightly.

02 **Use it in a pilot**

One real product team uses it in real work, with a direct line to the system team.

03 **Watch for workarounds**

If the team detaches it in Figma, overrides it in code or asks "how do I make it do X?", that's design feedback.

04 **Adjust, then promote to "stable"**

Fix what the pilot revealed, then announce it as stable and ready for everyone.

PRACTICE THE REVIEW QUESTIONS WE ALWAYS ASK

Can a new designer understand when to use this in under a minute? Does it work with only a keyboard? What happens with the longest real content we've ever seen? Which existing versions will this replace, and how will teams switch? If any answer is "not sure", it isn't ready yet.

THE SHORT VERSION

What to remember from this chapter

01 Start from real examples and real content, not a blank canvas.

02 Design every state and stress-test every edge before handing over.

03 Build accessibility into the component, and every screen inherits it.

04 Agree one "ready to build" checklist, involve engineers early, and prove components in a pilot before calling them stable.

Up next → **Chapter 15: Building components in code**

CHAPTER 15

15

Building components in code

This chapter is for engineers, but it's written so that designers, product managers and founders can follow it too. Understanding how components are built in code is the best way to understand why design systems succeed or struggle.

READ THIS IF

You build components, work closely with people who do, or keep running into "the design and the code don't match".

WHAT YOU'LL LEAVE WITH

01 What engineers mean by a component's "API" and "props", in plain words.

02 How to keep design and code in step, and avoid three versions of everything.

03 How components are tested and shipped so teams can trust them.

IN THIS CHAPTER

15.1 What engineers mean by a component

15.2 Keeping design and code in step

15.3 Settings or building blocks?

15.4 No hard-coded values, ever

15.5 How you know a component works

15.6 Packaging and shipping components

15.1

What engineers mean by a component

NEW WORD

Props

The settings you pass to a component in code, like the buttons on a TV remote: "variant", "size", "disabled".

Think of a TV remote. You don't need to know how the TV works inside. You just press buttons: volume up, channel 5, mute. A code component works the same way. Engineers use it by passing it a few settings, which they call **props**, and the component takes care of everything else: colours, spacing, keyboard behaviour, accessibility.

Here's what using the Ledgerly button looks like in code. Even if you've never written code, you can probably read it:

```
// Using the button: three settings and a label
<Button variant="primary" size="md" icon="send">
  Send invoice
</Button>

// The same button, disabled while saving
<Button variant="primary" loading>Saving…</Button>
```

NEW WORD

API

The list of settings a component accepts and what each one does. It's the "contract" between the component and the people using it.

The full list of settings a component accepts is called its **API**. A good component API is like a good remote: a few clearly labelled buttons that do exactly what they say, and nothing surprising. A bad one has sixty tiny buttons, half of which interfere with each other.

Once a component is in use, its API becomes a promise. If you rename `variant` to `kind`, every screen that uses the old name stops working. That's why changes to a component's API are treated as "breaking changes" (Chapter 05) and handled with care.

15.2

Keeping design and code in step

In Chapter 12 we described components as twins: one in the design tool, one in code. Keeping the twins identical is one of the hardest ongoing jobs in a design system, and when it slips, things get messy fast.

SCENARIO **Three dropdowns, all called "Dropdown"**

THE SITUATION

A mid-sized SaaS company has three engineering teams. Over two years, each built its own dropdown: one wraps an old open-source library, one is custom, and one was copied from the system and then modified.

WHAT GOES WRONG

A keyboard accessibility bug is reported. It exists in all three, fixed differently in each. A design update to the dropdown takes three separate projects. New engineers don't know which one to use.

THE WAY OUT

One dropdown in the system, matching the Figma version exactly; a migration guide; an automatic warning when the old ones are used; and a date after which the old ones are deleted.

One vocabulary, two places

The simplest and most effective rule: **the names and options in Figma match the names and options in code, exactly.** If the design file calls it "variant: primary / secondary / danger", the code does too. Not "kind", not "solid / outline", not "primary / default / destructive".

IN FIGMA	IN CODE	STATUS
Variant = Primary	`variant="primary"`	`match`
Size = Medium	`size="md"`	`match` (agreed short form, documented)
State = Disabled	`disabled`	`match`
Type = Tertiary	`variant="ghost"`	`mismatch` : rename one of them
Has icon = Yes	`icon="send"`	`match` (code also says which icon)

A shared workshop for components

Most system teams use a "workshop" tool where every component can be seen and tried in isolation, in every variant and state, outside of any product. Storybook is the most widely used one.[62] It gives designers, engineers and product managers one place to look at the *real* coded component, which makes it much easier to spot when the twins have drifted apart. Many teams also link each Figma component directly to its code and documentation, so a designer can jump from one to the other.

PRACTICE A MONTHLY "TWIN CHECK"

Once a month, a system designer and a system engineer spend an hour going through recently changed components side by side: Figma on one screen, the coded version on the other. List every difference and decide which twin is right. It's tedious, and it's the cheapest way we know to stop drift.

15.3

Settings or building blocks?

There are two broad ways to make a component flexible, and choosing between them is one of the most important decisions in building a component library.

The first is **configuration**: one component with lots of settings. "Card with `showImage`, `showAvatar`, `showFooter`, `footerAlign`…" It's easy to start with, but every new need adds another setting, and eventually you get the forty-setting monster from Chapter 12.

The second is **composition**: several small, simple pieces that fit together, like Lego. You combine them to build exactly what you need. This is the code version of the "slots" idea from Chapter 12.

```
// Configuration: one component, many settings
<Card
  title="Revenue"
  showMenu
  image={chart}
  footerText="Updated 4 min ago"
  footerAlign="right"
  primaryAction="View report"
  // …and the next team will need one more
/>
```

```
// Composition: small pieces that fit together
<Card>
  <Card.Header title="Revenue">
    <Menu … />
  </Card.Header>
  <Card.Media>{chart}</Card.Media>
  <Card.Footer>
    Updated 4 min ago
    <Button>View report</Button>
  </Card.Footer>
</Card>
```

Neither approach is always right. Our rule of thumb is the same one we gave for design in Chapter 12. Small, frequently used components (button, checkbox, badge) should use a few clear settings, because consistency matters most and the options are limited. Bigger, container-like components (card, dialog, table, page layout) should use composition, because the content varies endlessly and settings can't keep up. ■ PRACTICE

15.4

No hard-coded values, ever

Chapter 11 showed the button's code using only named values, with no colour codes and no pixel numbers. That's the rule for every component in the system, and it's worth enforcing automatically, because it's very easy to slip.

Imagine an engineer is fixing a small layout issue at the end of a long day. The quickest fix is to type `margin-top: 13px`. It works, it ships, and nobody notices. But that one value doesn't change in compact mode, doesn't follow the spacing scale, and will be wrong after the next spacing update. Multiply that by a hundred small fixes and the system's foundations are quietly bypassed.

NEW WORD	
Linter	A tool that automatically checks code for problems and breaks of agreed rules, and flags them before the code is merged.

The fix is a **linter** rule: an automatic check that flags any raw colour code, pixel value or font size in component code, and suggests the named value to use instead. It runs every time someone submits code, so nobody has to remember. The same check can run in product teams' code too, which is one of the best ways to measure how much of the product really uses the system (Chapter 03).

15.5

How you know a component works

When a component is used in 400 places, a bug isn't one bug: it's 400 bugs. That's why system components are tested more thoroughly than ordinary product code. Teams usually combine four kinds of testing:

KIND OF TEST	WHAT IT CATCHES	HOW IT WORKS, SIMPLY
Visual tests	Unintended changes to how a component looks	A tool takes a picture of every variant and state and compares it with the last approved picture. Any difference is flagged for a human to check.
Automated accessibility checks	Missing labels, low contrast, incorrect roles	A tool scans each component for common accessibility rule breaks. It catches a lot, but not everything.
Behaviour tests	Things that don't work, like a menu that won't open with the keyboard	Code that pretends to be a user: clicks, types, presses keys, and checks the right thing happens.
Manual checks	What automation can't judge	A person tries the component with a keyboard only and with a screen reader, especially for complex components.

Visual tests deserve special mention, because they catch exactly the kind of problem from the scenario in Chapter 05, where a small padding change broke layouts. If the system team had seen pictures of every button changing size, they'd have known it was a breaking change before shipping it.

A MISTAKE WE SEE OFTEN

"The automated checks passed, so it's accessible"

Automated accessibility tools are excellent at catching certain problems, like missing labels and poor contrast. They can't tell whether a keyboard user can actually complete a task, or whether a screen reader announcement makes sense. Always pair them with a real keyboard and screen-reader check for interactive components.

15.6

Packaging and shipping components

Finished components are bundled into a **package**, which is how they reach product teams. Product teams install the package, a bit like installing an app, and can update it when a new version comes out. Chapter 05 covered version numbers and breaking changes; here's what that means day to day for the people building components.

01 **Every change gets a note**

Each change includes a one-line entry for the release notes, written for the people using the system: what changed and what, if anything, they need to do.

02 **Releases happen on a rhythm**

For example, new features every two weeks and urgent fixes whenever needed. Predictable releases let product teams plan their updates.

03 **Breaking changes are rare, grouped and announced**

Collect them into one bigger release, a few times a year, with a migration guide and, where possible, a script that makes the changes automatically.

04 **Old versions get a notice period**

Product teams can't always update immediately. Say how long old versions will receive important fixes, and stick to it.

WHAT IF TEAMS USE DIFFERENT TECHNOLOGIES?

Large companies often have products built with different frameworks: one team on React, another on Angular, the mobile apps in Swift and Kotlin. Named values (tokens) work everywhere, because they're generated for each platform. Components are harder, because each technology needs its own version. This is one of the biggest cost drivers in Chapter 04, and a strong reason to agree on as few front-end technologies as possible.

THE SHORT VERSION

What to remember from this chapter

01 A component's settings (its API) are a promise. Keep them few, clear and stable.

02 Use exactly the same names and options in Figma and in code, and check them regularly.

03 Small components use a few settings; big ones are built from composable pieces.

04 Ban hard-coded values with automatic checks, test visually and for accessibility, and ship on a predictable rhythm.

Up next → **Chapter 16: A library people can actually use**

CHAPTER 16

16

A library people can actually use

A component that nobody can find, understand or trust might as well not exist. This chapter is about the everyday side of a growing library: organising it, documenting it, tracking it, and handling the moment a team needs something a little different.

READ THIS IF

Your Figma library has grown messy, people keep building their own versions of things that already exist, or you're not sure which components are really being used.

WHAT YOU'LL LEAVE WITH

01 A way to organise and name a design library so people can find things fast.

02 A documentation template and a simple life cycle for every component.

03 Practical ways to track usage and to handle teams that need something different.

IN THIS CHAPTER

16.1 If people can't find it, they'll rebuild it

16.2 Organising the design library

16.3 Documentation that actually gets read

16.4 A component's life, from idea to retirement

16.5 Knowing what's actually being used

16.6 When a team needs something different

16.1

If people can't find it, they'll rebuild it

Here's a pattern we see in almost every growing design system. In year one, the Figma library is small and everyone knows what's in it. By year three, it has hundreds of components, several abandoned experiments, old versions nobody deleted, and names that made sense to whoever created them.

SCENARIO **The designer who couldn't find the right component**

THE SITUATION

Meera joins as a product designer. She needs a simple tag to show an invoice's status. She opens the library and searches "tag".

WHAT SHE FINDS

"Tag", "Tag v2", "Chip", "Pill/Status", "Badge (old)", "Label - DO NOT USE", and three more in a page called "WIP". No descriptions. Two look identical.

WHAT HAPPENS NEXT

She picks one at random. It turns out to be deprecated. Engineering builds it from scratch because the code version was deleted months ago. The product now has another status tag.

Nothing in that story is about the quality of the components themselves. They may all have been well designed. The problem is **findability**, and in our experience it's one of the most underrated causes of low adoption. Every minute someone spends confused in your library makes building their own version more tempting.

16.2

Organising the design library

A well-organised library feels like a well-organised supermarket: you can guess which aisle something is in, things are labelled clearly, and nothing out of date is left on the shelves. Here's a structure that works for most teams.

```
Ledgerly · Foundations          ← colours, text styles, spacing, icons
Ledgerly · Components           ← the core, used by everyone
  ├─ Start here                  ← how to use this library
  ├─ Actions
  │    ├─ Button
  │    ├─ Icon button
  │    └─ Link
  ├─ Forms
  │    ├─ Text field
  │    ├─ Select
  │    └─ Checkbox · Radio · Switch
  ├─ Feedback
  │    ├─ Alert · Toast
  │    └─ Empty state
  └─ Data display
       └─ Table · Badge · Tag
Ledgerly · Analytics extensions ← shared by analytics products
Archive (hidden from search)     ← old versions, kept for reference
```

What makes this work

- **Separate libraries for separate jobs.** Foundations, core components and product-family extensions live in different files, so people only switch on what they need.
- **Groups that match how people think:** actions, forms, feedback. Not by date added or by who built it.
- **A "Start here" page** that explains the library in two minutes.
- **Old things archived and hidden**, not left lying around with "old" in the name.
- **No work-in-progress** in the published library. Experiments live in a separate file until they're ready.

FIG 16.1 A tidy library structure. Three published libraries (foundations, core and one extension) plus a hidden archive. People can find what they need by guessing which group it's in.

Names and descriptions

Remember Rule #3: *name things so people can guess them*. Use the same component names everywhere (Figma, code, documentation) and avoid version numbers or status in names. "Tag v2 NEW" is a cry for help, not a name. Status belongs in a label or description, not the name.

Most design tools let you add a short description and search keywords to each component. Use them. A good description answers "what is this for?" in one sentence and lists the words someone might search for. "Tag: a short label showing a status or category. Also known as: chip, pill, badge, label." That one line would have saved Meera half a day.

> PRACTICE **PUBLISH ON A RHYTHM, NOT ON A WHIM**
>
> Publish library updates on the same rhythm as code releases, with the same release notes. When designers get a library update notification, they should know exactly what changed and why. Random, unannounced updates teach people to ignore update notifications, and then they end up working with outdated components.

16.3

Documentation that actually gets read

Layer One's ninth rule says it bluntly: *docs decide adoption*. People use what they can understand in the five minutes before a deadline. If documentation is long, scattered or out of date, people stop reading it and start guessing, or building their own.

The trick isn't writing more. It's writing the same few things for every component, in the same order, so people know exactly where to look. Here's the template we use:

#	SECTION	WHAT IT ANSWERS	KEEP IT TO
1	**Summary**	What is it, and when should I use it?	One or two sentences
2	**When not to use it**	What should I use instead, and when?	Two or three bullet points
3	**Live examples**	What does it look like in each option and state?	Interactive examples
4	**Anatomy**	What are the parts called?	One labelled diagram
5	**How to build with it**	How do I use it in code and in Figma?	Copyable code, settings table, library link
6	**Accessibility**	How does it work with a keyboard and a screen reader? What must I add?	A short checklist
7	**Writing**	What should the words say?	One or two lines, plus examples
8	**Do's and don'ts**	What does correct and incorrect use look like?	Two to four visual pairs
9	**Status and history**	Is it stable? What changed recently?	A status label and a short change log

The UK government's GOV.UK Design System is a good public example of documentation done well: every component page follows a consistent structure, explains when *not* to use it, and shares the research behind it.[23] Notice also what's *not* in the template: long essays about design philosophy. Those belong on a separate page for people who want them.

16.4

A component's life, from idea to retirement

Components aren't permanent. They're proposed, tried out, settled, and eventually replaced. Giving each stage a clear label tells everyone how much they can rely on a component right now.

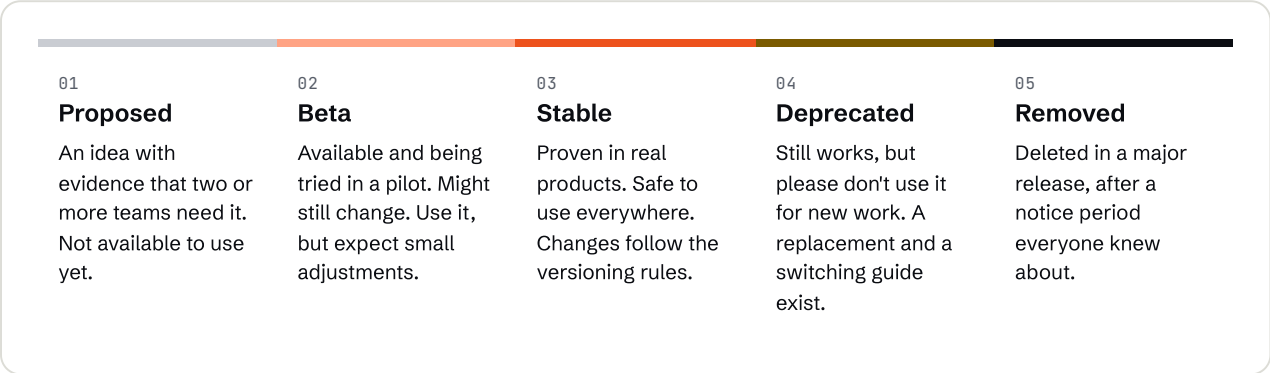


FIG 16.2 Five stages in a component's life. PRACTICE Show the current stage as a label in Figma, in code and on the documentation page, so nobody has to ask.

The two stages teams most often skip are **beta** and **deprecated**, and they're the two that build the most trust. Beta lets you release something before it's perfect without breaking anyone's work when it changes. Deprecated lets you retire something without surprising anyone. Chapter 05 describes the deprecation process in more detail: mark it, give a guide and a notice period, track who's still using it, and only remove it in a major release.

16.5

Knowing what's actually being used

You can't look after a library well if you don't know how it's used. Fortunately, both sides of the twins leave a trail.

IN DESIGN FILES

Usage and detachments

Design tools can typically show how often each library component is inserted, and how often designers *detach* it (break the link to customise it). Detachments are gold: they show exactly where a component isn't meeting people's needs.

IN CODE

Imports and versions

A simple script can count which system components each product imports, which version of the package each product uses, and where deprecated components are still in use.

SCENARIO

What the detach numbers revealed

THE SITUATION

Ledgerly's system team looks at its design library data for the first time. The alert component is used 900 times, and detached 310 times.

WHAT THEY FIND

They look at a sample of the detached copies. Almost all of them added a second button: "Retry" next to "Dismiss". The component only allowed one.

WHAT THEY DO

They add an optional second action, release it in a minor update, and the detach rate drops sharply. No survey or meeting needed; the data told them.

Two cautions. First, remember Rule #8: measure time saved, not just usage. High usage is good, but it isn't the goal in itself. Second, use the data to learn, not to police. If a team detaches a component a lot, the first question is "what does the component fail to do for you?", not "why aren't you following the rules?"

16.6

When a team needs something different

Sooner or later, a team will need a component to do something it doesn't do. The marketing team wants a bigger, bolder button for a campaign page. The analytics team needs a table that can freeze three columns. How you respond decides whether the system bends sensibly or breaks into pieces.

There are four common responses, and they're not equally healthy:

RESPONSE	WHAT HAPPENS	WHEN IT'S RIGHT
Extend the component	Add a new option to the core component for everyone.	When two or more teams need it and it fits the component's purpose.
Build a local component	The team builds its own, from system parts, and owns it.	When it's specific to one product. It can be promoted later.
Small override	The team adjusts one detail of a system component in its own code.	Rarely, as a short-term fix, and always reported to the system team.
Copy and modify	The team copies the system component's code and changes it.	Almost never. The copy stops getting fixes and updates, and silently drifts.

The last row is how companies end up with three dropdowns (Chapter 15). It usually happens when the system team is slow to respond, or when "build a local component" isn't presented as a respectable option. Make the healthy paths quick and easy, and copying becomes rare.

THE PRINCIPLE BEHIND ALL OF THIS

The system should make the right thing the easy thing. When teams work around it, treat that as feedback about the system, not a failure of the teams.

Library health checklist

- [ ] Separate libraries for foundations, core and extensions
- [ ] Groups that match how people think, plus a "Start here" page
- [ ] Same names in Figma, code and docs; no versions in names
- [ ] A one-line description and keywords on every component
- [ ] Old versions archived and hidden; no work-in-progress published
- [ ] One documentation template, used for every component
- [ ] A visible life-cycle label on every component
- [ ] Library updates published with release notes, on a rhythm
- [ ] Usage and detach data reviewed monthly
- [ ] Quick, respectable paths for local components

THE SHORT VERSION

What to remember from this chapter

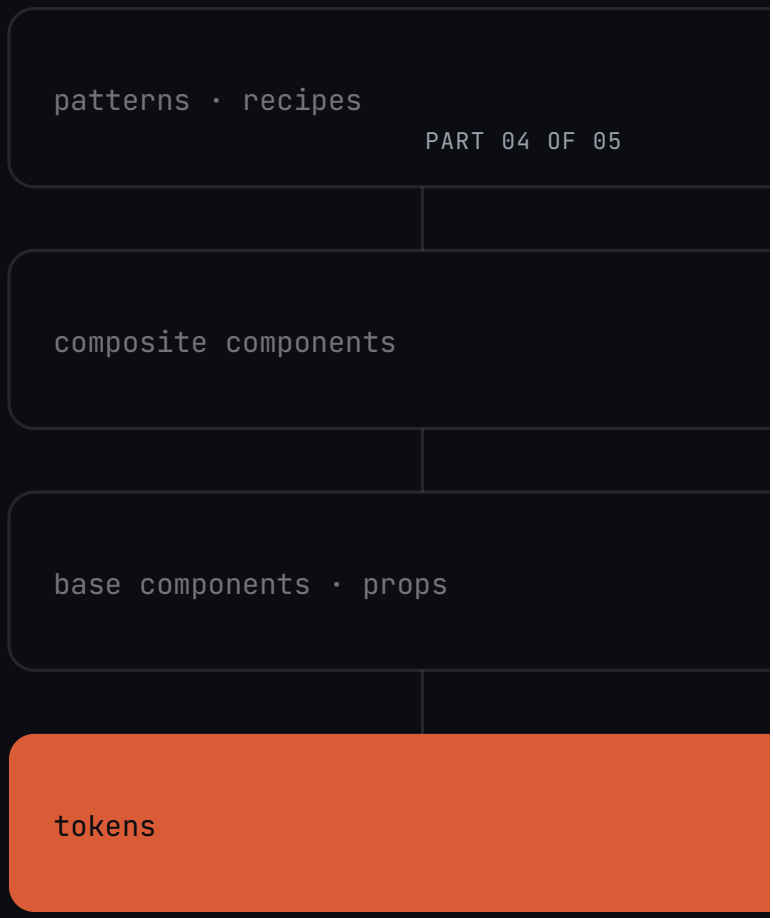
01 Findability is adoption. A tidy, well-named library prevents duplicates before they happen.

02 Document every component the same way, briefly, and say when *not* to use it.

03 Label every component's stage (beta, stable, deprecated) so people know what to rely on.

04 Use design and code data to learn where components fall short, and make the healthy paths quick and easy.

Up next → **Part 04: Component architecture**



04

Component architecture in depth

Part 03 explained what components are and how to look after a library. This part opens the bonnet. How complex is a component, really? How should a library be structured? How do settings (props) work, how do components use tokens, and what does good component code look like? It ends with a casebook of real situations you will meet.

17 Component complexity

18 Component architecture

19 Props, in depth

20 Components and tokens

21 Components as code

22 A casebook of real situations

CHAPTER 17

17

Component complexity

A badge and a data table are both "components", but one takes an afternoon and the other can take a quarter. Understanding complexity helps you plan honestly, choose the right process for each component, and pick the simplest component that does the job.

READ THIS IF

You estimate or plan component work, keep being surprised by how long some components take, or need to decide which component to use for a design problem.

WHAT YOU'LL LEAVE WITH

01 Four clear levels of component complexity, with examples.

02 A scoring method you can use to rate any component.

03 How complexity should change your process, and how to keep it under control.

IN THIS CHAPTER

17.1 Not all components are equal

17.2 Four levels of complexity

17.3 Measuring complexity: a simple scoring method

17.4 How complexity changes the way you work

17.5 Choosing the simplest component that works

17.6 Keeping complexity under control

17.1

Not all components are equal

Here's a planning conversation we hear all the time. "This quarter, the design system team will build ten components: tooltip, badge, avatar, divider, tag, button group, breadcrumbs, tabs, date picker and data table." Ten components, three months, so about a week each. It sounds reasonable. It isn't.

SCENARIO **"Ten components this quarter"**

THE SITUATION

A system team commits to ten components in a quarter. The first six (badge, avatar, divider, tag, tooltip, breadcrumbs) are done in three weeks. The team feels ahead of schedule.

WHAT GOES WRONG

Tabs take two weeks to get keyboard behaviour right. The date picker takes five weeks: time zones, locales, keyboard grids, screen readers. The data table isn't started. Leadership sees "60% done" and loses trust.

WHAT WOULD HAVE HELPED

Rating each component's complexity first. The data table alone was more work than the other nine combined. The plan should have said so, or left it for next quarter.

Counting components is like estimating a house renovation by counting rooms: the bathroom and the hallway are both "a room", but one involves plumbing, waterproofing and tiling. Complexity is what tells you which is which. The rest of this chapter gives you a shared language for it, so designers, engineers and managers can plan together and stop being surprised.

17.2

Four levels of complexity

We find it useful to sort components into four levels. They loosely echo Brad Frost's idea of building from small parts to large ones,[2] but they're organised by *how hard something is to get right* rather than how big it is.

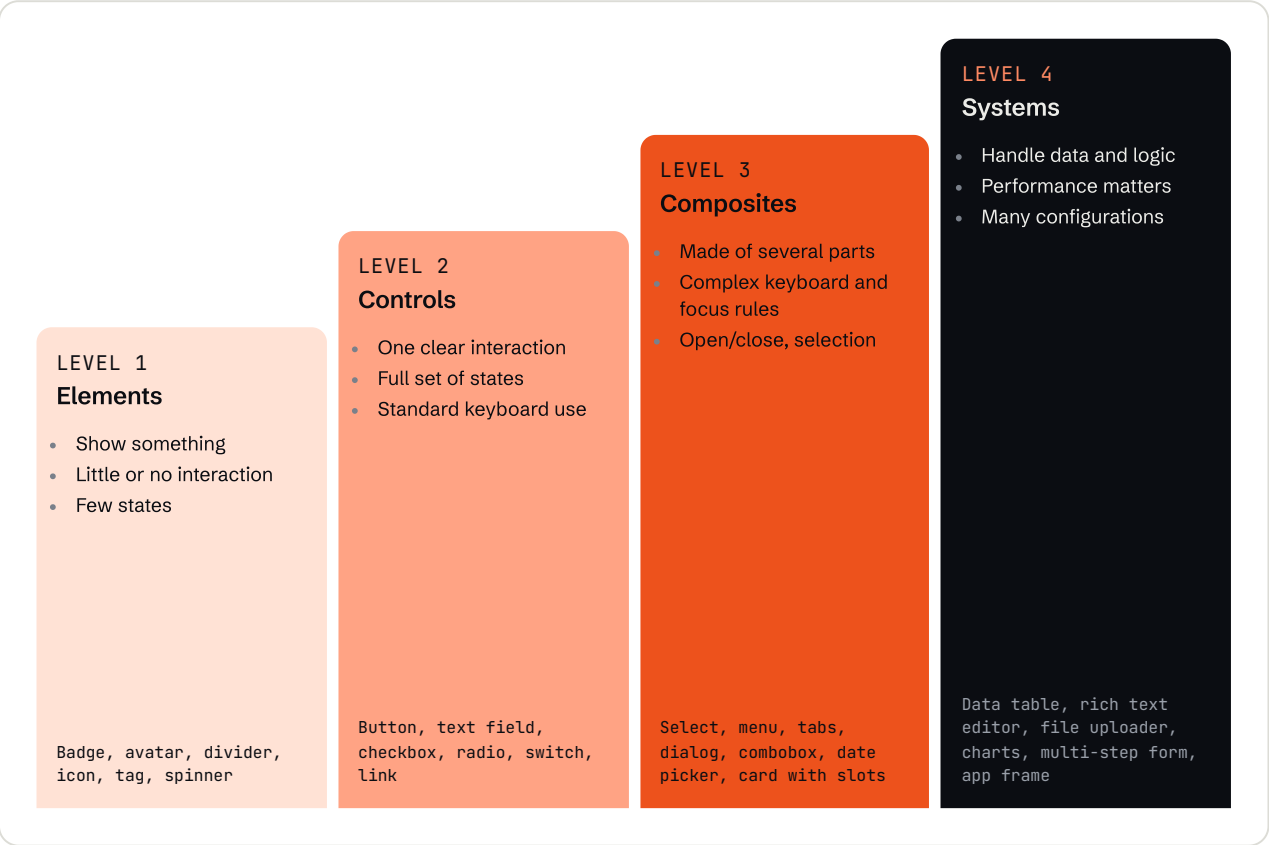


FIG 17.1 Four levels of component complexity. PRACTICE Each level up adds a new kind of difficulty: interaction at level 2, coordination between parts at level 3, and data, logic and performance at level 4.

Level 1, elements, mostly display something: a badge, an avatar, a divider, an icon. They're about visual consistency and they're quick to build. The main risk is making too many variations of them.

Level 2, controls, are the things people interact with directly: buttons, text fields, checkboxes. Each has one main job, but now every state matters (hover, focus, disabled, error), and keyboard and screen-reader support become essential.

Level 3, composites, are built from several parts that have to work together: a menu has a trigger and a list; tabs have a tab list and panels; a dialog manages focus for everything inside it. This is where accessibility gets genuinely hard, because there are established keyboard patterns people expect, documented in the W3C's ARIA Authoring Practices Guide.[61]

Level 4, systems, are small applications in their own right. A data table sorts, filters, paginates, selects rows, resizes columns and must stay fast with thousands of rows. A rich text editor is a famously hard engineering problem. These components often take longer than everything else in the library combined.

17.3

Measuring complexity: a simple scoring method

Levels are a good first guess, but you'll want something more precise when planning. Here's a scoring method we use. Rate the component from 0 to 3 on each of eight questions, then add them up. You don't need to be exact; the point is to have the conversation, as a team, before committing to dates. PRACTICE

QUESTION	0 — NONE	1 — A LITTLE	2 — QUITE A LOT	3 — A LOT
Options	One version	2–3 variants	Variants × sizes	Many combinations
States	Static	Hover, focus	Full interactive set	Plus data states (loading, empty, error)
Interaction	None	Click	Typing, selection	Drag, resize, multi-select
Keyboard & screen reader	Nothing special	Standard tab and enter	Arrow keys, announcements	Complex focus management, live updates
Data	None	Simple value	Lists, formatting	Large data sets, async loading
Screen sizes	Same everywhere	Minor adjustments	Different layout on mobile	Different behaviour on mobile
Composition	Single part	Fixed sub-parts	Slots or children	Nested components, coordination
Platforms	Web only	Web, one framework	Web + one native	Web + iOS + Android

Then use the total like this: **0–6** is level 1, **7–12** is level 2, **13–18** is level 3, and **19–24** is level 4. Here's how three familiar components score:

COMPONENT	OPTIONS	STATES	INTERACTION	KEYBOARD	DATA	SCREENS	COMPOSITION	PLATFORMS	TOTAL	LEVEL
Badge	1	0	0	0	0	0	0	1	**2**	1 · Element
Button	2	2	1	1	0	1	1	1	**9**	2 · Control
Date picker	2	3	2	3	2	2	2	1	**17**	3 · Composite
Data table	3	3	3	3	3	3	3	1	**22**	4 · System

Notice the date picker. Most people would guess it's similar to a select menu. The scoring shows why it isn't: dates are data (time zones, locales, ranges, disabled days), and a calendar grid has some of the most demanding keyboard behaviour of any common component.

17.4

How complexity changes the way you work

Here's the practical payoff. Once you know a component's level, you can match the process to it. Running a level 4 process for a badge wastes everyone's time; running a level 1 process for a data table guarantees bugs in production.

	LEVEL 1 · ELEMENTS	LEVEL 2 · CONTROLS	LEVEL 3 · COMPOSITES	LEVEL 4 · SYSTEMS
Who's involved	One designer, one engineer	Pair, plus a quick accessibility check	Pair, accessibility specialist, a pilot team	A small squad, several pilot teams, product managers
Research	Inventory only	Inventory and real content	Usage research across teams	Requirements gathering, like a product feature
Accessibility	Contrast, labels	Keyboard and screen-reader basics	Full ARIA pattern; manual testing	Expert review; testing with assistive-technology users
Testing	Visual tests	Visual and behaviour tests	Plus manual keyboard and screen-reader tests	Plus performance tests with real data volumes
Beta period	None needed	A week or two	A month, with a pilot	A quarter, with several pilots
Documentation	Short page	Full template	Full template plus behaviour notes	Guides, recipes and examples
Typical effort	Days	1–2 weeks	3–6 weeks	1–3 months or more

Effort ranges are ■ PRACTICE estimates for one web framework, assuming foundations already exist. Each extra platform adds roughly the same again.

■ PRACTICE **PLAN BY POINTS, NOT BY COUNT**

Instead of "ten components this quarter", plan with the complexity totals: "about 90 points this quarter". A badge is 2 points, a data table is 22. It sounds a little nerdy, but it instantly makes plans more honest, and it gives leadership a much better picture of progress than a count of finished components.

17.5

Choosing the simplest component that works

Complexity isn't just a planning tool for system teams. It's also a design decision for product teams. Every time a designer chooses a component, they're choosing a level of complexity for their users *and* for the engineers who'll build the screen. The simplest component that does the job is almost always the right choice.

IF YOU NEED PEOPLE TO…	PREFER THIS (SIMPLER)	OVER THIS (MORE COMPLEX)	UNLESS…
Choose one of 2–5 options	Radio buttons	A dropdown select	Space is very tight
Choose one of 6–15 options	A select	A searchable combobox	Options are unfamiliar or long
Choose from hundreds of options	A combobox with search	A custom picker dialog	Options need images or grouping
Enter a date they know (birthday)	Three simple fields or one text field	A calendar date picker	They need to see days of the week
Show extra information	Text on the page	A tooltip	It's genuinely secondary
Confirm a decision	An inline message or undo	A confirmation dialog	The action is destructive and can't be undone

The date example is worth a moment. Calendar pickers feel modern, but for a date someone already knows, like their date of birth, typing it is usually faster than scrolling back decades in a calendar. The GOV.UK Design System, which is heavily user-tested, recommends simple separate day, month and year fields for exactly this case.[63] DOCUMENTED Simpler for the user, and a level 2 component instead of a level 3.

When you do need something complex, build it from simpler parts. A combobox is a text field plus a list plus some coordination; a data table is cells, checkboxes, buttons and menus arranged in a grid. If the simple parts are solid, the complex component inherits their quality, and fixes to the parts improve everything built from them.

17.6

Keeping complexity under control

Components tend to get more complex over time. Every reasonable request adds a little: one more option, one more state, one more special case. We call this **complexity creep**, and it's how a level 2 button quietly becomes a level 3 problem nobody wants to touch.

Three habits keep it in check. The first is a **scope statement** on every component's documentation page: a short list of what this component will *never* do. "The button will never contain a dropdown. Use the split button instead." It sounds negative, but it saves enormous amounts of debate.

The second is **splitting instead of stretching**. When a component starts serving two quite different jobs, it's usually better to split it into two simpler components than to add options until it does both badly. A "card" that is sometimes a link, sometimes a form and sometimes a dashboard widget is probably three components.

The third is **re-scoring once a year**. Run the eight questions again for your most-used components. If a score has jumped, ask why, and whether all that complexity is still earning its keep.

A MISTAKE WE SEE OFTEN

Building the "universal" component

A team tries to build one table that handles every possible case: simple lists, editable spreadsheets, trees, pivot tables. It takes a year and is so complex that most teams avoid it. It's usually better to build a simple table that handles 80% of cases well, and let the rare, complex cases use a specialised component or a well-chosen third-party library.

THE SHORT VERSION

What to remember from this chapter

01 Components range from simple elements to whole systems. A data table can be more work than ten badges.

02 Score components on eight questions to plan honestly, and plan by points, not by count.

03 Match the process (people, testing, beta period) to the level of complexity.

04 Choose the simplest component that works, and fight complexity creep with scope statements and splits.

Up next → **Chapter 18: Component architecture**

CHAPTER 18

18

Component architecture

Architecture sounds intimidating, but the idea is simple: decide which parts of your library are built from which, and keep those relationships tidy. Get it right and the library is easy to change for years. Get it wrong and every small change breaks something unexpected.

READ THIS IF

You're setting up a component library, your library has become hard to change, or you want to understand how mature systems are organised under the surface.

WHAT YOU'LL LEAVE WITH

01 A layered model for organising components, and the rules that keep it healthy.

02 The main ways components are put together, explained with simple examples.

03 How to organise the files and packages, and how to manage dependencies as the library grows.

IN THIS CHAPTER

18.1 What "architecture" means for components

18.2 Layers, and the one rule that keeps them healthy

18.3 Separating how it works from how it looks

18.4 Four ways to put components together

18.5 Organising the files and packages

18.6 Dependencies, ownership and change

18.1

What "architecture" means for components

Think about the wiring in a building. You don't see it, but it decides a lot: whether you can add a new socket easily, whether a fault in the kitchen trips the whole house, whether an electrician can understand it ten years later. Good wiring is organised in circuits, each with a clear job, all running back to one board.

Component architecture is the wiring of your design system. It answers three questions. **What is each component built from?** (A dialog uses buttons, which use text styles, which use tokens.) **Which way do dependencies run?** (A button can use an icon, but an icon should never need a button.) And **where does each kind of logic live?** (Is keyboard behaviour inside the dialog, or in a shared piece that menus use too?)

SCENARIO

The text change that broke forty components

THE SITUATION

An engineer adjusts the line spacing of the library's internal Text component to fix a problem in one tooltip.

WHAT GOES WRONG

Forty other components use the same Text component. Buttons grow by 2 pixels, table rows shift, and three product teams report layout bugs after the next update.

WHAT ARCHITECTURE CHANGES

A clear map of which components depend on which would have shown the "blast radius" before the change. The fix belonged in the tooltip, not in a shared building block.

18.2

Layers, and the one rule that keeps them healthy

Most well-run component libraries are organised in layers, a little like the layers of a design system in Chapter 01, but zoomed in on the components themselves. Each layer is built only from the layers beneath it.

6 · Product screens	Owned by product teams	`Invoices page`
5 · Recipes & patterns	Proven combinations for common problems	`Filter bar, invoice summary card`
4 · Composite components	Several parts working together	`Dialog, menu, tabs, combobox, table`
3 · Base components	Single-purpose controls and elements	`Button, text field, checkbox, badge`
2 · Primitives	Invisible building blocks for layout, text and behaviour	`Box, Stack, Text, Icon, focus helpers`
1 · Tokens	Named decisions from Part 02	`color.action.primary.bg, space.4`

Rule: arrows only point down. A layer may use anything below it, never anything above it.

FIG 18.1 The six layers of a component library. PRACTICE Tokens and primitives are rarely seen by product teams, but they're what make every layer above them consistent.

NEW WORD

Primitive — A low-level building block, often invisible, that other components are made from: a box with spacing, a piece of styled text, a stack that spaces its children evenly.

The layer most people haven't heard of is **primitives**. These are tiny, mostly invisible building blocks: a *Box* that applies spacing and colour tokens, a *Stack* that lines things up with even gaps, a *Text* that applies a text style. Primitives mean that base components don't each reinvent "how do I apply spacing tokens". They're also handy for product teams building local components, because using primitives keeps local work on-system.

The one rule that keeps layers healthy is: **dependencies only point down**. A dialog may use a button. A button must never use a dialog. A primitive must never know that tables exist. When this rule is broken, you get circular dependencies (two components each needing the other), and changes start causing surprises far away from where they were made.

18.3

Separating how it works from how it looks

Every interactive component has two quite different jobs. One is **behaviour**: what happens when you click, which element has keyboard focus, what a screen reader announces, when a menu opens and closes. The other is **appearance**: colours, spacing, corner rounding, text styles. Mature libraries increasingly keep these two jobs apart.

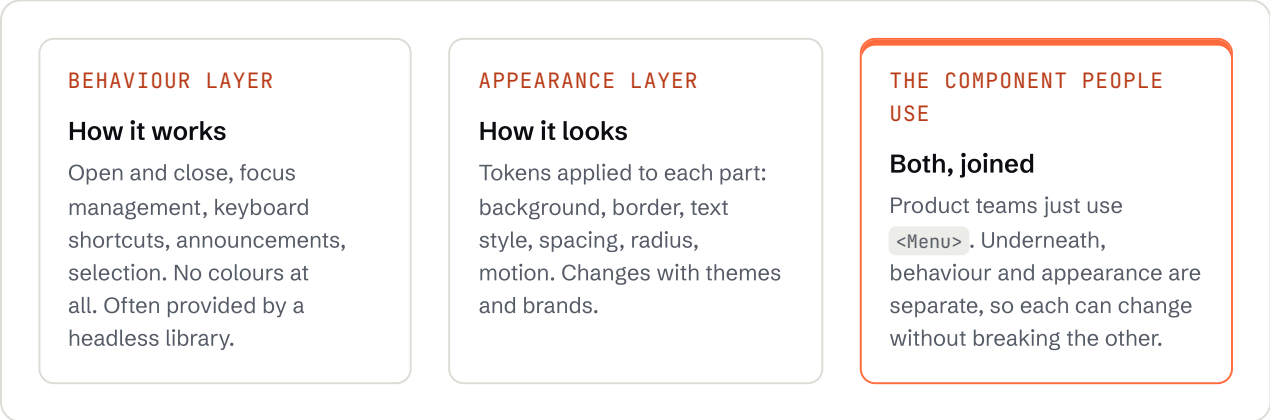


FIG 18.2 Behaviour and appearance, kept apart. A rebrand touches only the appearance layer; an accessibility fix touches only the behaviour layer. Neither risks breaking the other.

This is why headless libraries (Chapter 13) have become so popular. They provide the behaviour layer, which is the genuinely difficult part, already tested against the expected keyboard and screen-reader patterns.[61] Your team supplies the appearance layer using your own tokens. The result looks entirely like your brand but behaves like a well-tested standard component.

Even if you build everything yourself, the separation is worth keeping. Put behaviour into shared pieces of logic that several components can use. A "dismiss on Escape and return focus" behaviour, for example, is needed by dialogs, side panels, menus and popovers. Write it once, test it once, and use it in all four.

18.4

Four ways to put components together

When you look inside mature component libraries, you see the same few construction techniques again and again. Here they are in plain language, with a tiny code example of each for the engineers. You don't need to read the code to follow the idea.

1. Compound components: a family that works together

Some components are really a small family of parts that only make sense together. Tabs are the classic example: a list of tabs, the tabs themselves, and a panel for each tab. Instead of one giant component with dozens of settings, you provide each part separately and let them coordinate behind the scenes. It reads almost like a sentence.

```
<Tabs defaultValue="invoices">
  <Tabs.List>
    <Tabs.Tab value="invoices">Invoices</Tabs.Tab>
    <Tabs.Tab value="payments">Payments</Tabs.Tab>
  </Tabs.List>
  <Tabs.Panel value="invoices">…</Tabs.Panel>
  <Tabs.Panel value="payments">…</Tabs.Panel>
</Tabs>
```

The parts share information quietly (which tab is selected, which panel to show, where keyboard focus goes), so the person using them doesn't have to wire it up. Use this for any component made of a fixed family of parts: tabs, menus, accordions, selects.

2. Slots: spaces for any content

We met slots in Chapter 12. In code, a slot is usually just "put whatever you like here", with the component controlling the frame around it. A card with header, media, body and footer slots is the typical example. Use slots for containers whose content varies endlessly: cards, dialogs, side panels, page layouts.

3. Changing what something renders as

Sometimes you want something that *looks* like a button but is really a link to another page, because it navigates rather than performs an action. That distinction matters for accessibility: screen readers announce links and buttons differently, and people expect different keyboard behaviour. Many libraries let a component render as a different underlying element while keeping its appearance.

```
// Looks like a button, behaves like a link (because it navigates)
<Button as="a" href="/invoices/new" variant="primary">New invoice</Button>
```

Use this sparingly and document it clearly. "Actions use buttons, navigation uses links" is a rule worth keeping, even when they look alike.

4. Controlled and uncontrolled

Some components hold a piece of information that can change: whether a dialog is open, which tab is selected, what's typed in a field. The question is *who keeps track of it*. In an **uncontrolled** component, the component keeps track itself; you just give it a starting value. In a **controlled** component, the product screen keeps track and tells the component what to show. React's documentation explains the idea well.[64]

```
// Uncontrolled: "start on invoices, then look after yourself"
<Tabs defaultValue="invoices">…</Tabs>

// Controlled: "I'll tell you which tab is selected, and you tell me when it changes"
<Tabs value={tab} onValueChange={setTab}>…</Tabs>
```

Good libraries support both. Uncontrolled is simpler for everyday use. Controlled is needed when the screen must react to changes, for example to save the selected tab in the web address so people can share a link to it.

18.5

Organising the files and packages

A tidy codebase is architecture you can see. When every component is organised the same way, engineers can find things instantly, reviewers know what to expect, and nothing important (tests, documentation, examples) gets forgotten. Here's a structure that works for most teams.

```
design-system/
├ packages/
│  ├ tokens/          ← source JSON + generated outputs
│  ├ icons/           ← SVG source + generated components
│  ├ primitives/      ← Box, Stack, Text, focus helpers
│  ├ core/            ← base + composite components
│  └ analytics/       ← extensions for one product family
├ apps/
│  ├ docs/            ← documentation site
│  └ workshop/        ← Storybook
└ tools/              ← lint rules, codemods, scripts
```

```
core/src/button/
├ Button.tsx          ← the component
├ Button.css          ← styles (tokens only)
├ button.tokens.json  ← component tokens
├ Button.test.tsx     ← behaviour tests
├ Button.stories.tsx  ← every variant and state
├ Button.a11y.md      ← accessibility notes
├ README.md           ← usage docs source
└ index.ts            ← what's public
```

FIG 18.3 **A library split into packages, and one component's folder.** PRACTICE Separate packages let teams install only what they need and let each part be versioned on its own. Every component folder has the same eight files, so nothing is forgotten.

A few details matter more than they appear. The **index file** decides what's public. Anything not exported there is internal and can change freely, which gives the system team room to improve things without breaking anyone. The **stories file** shows every variant and state in the workshop (Chapter 15), which doubles as the source for visual tests. And the **tokens file** keeps a component's design decisions next to its code, which Chapter 20 explains.

18.6

Dependencies, ownership and change

As a library grows, the relationships between components become as important as the components themselves. Nathan Curtis's series on releasing design systems devotes a whole article to this: how a change to one component travels up to everything that depends on it.[28] Three habits make that manageable.

Keep a dependency map. Know which components use which. Many tools can generate this automatically from the code. Before changing a shared piece, look at what depends on it. In our scenario, that would have shown forty components relying on Text.

Name an owner for every component. In larger organisations, each component (or group of components) has a named owner who reviews changes to it. Code platforms let you record this in a simple file, so the right person is asked automatically whenever someone proposes a change.

Change the right layer. When fixing a problem, fix it at the lowest layer where it genuinely belongs, and no lower. If only the tooltip needs tighter lines, change the tooltip. If *every* component needs tighter lines, change the text token. The mistake in our scenario was fixing a local problem in a shared layer.

> ■ PRACTICE **ARCHITECTURE REVIEW, QUARTERLY**
>
> Once a quarter, look at the dependency map with fresh eyes. Are there arrows pointing upwards? Components nobody uses? Shared pieces with an enormous blast radius? An hour of review prevents the slow tangle that eventually makes teams say "we need to rebuild the design system".

THE SHORT VERSION

What to remember from this chapter

01 Organise the library in layers (tokens, primitives, base, composite, recipes) where dependencies only point down.

02 Keep behaviour separate from appearance, so rebrands and accessibility fixes don't collide.

03 Use compound components for families of parts, slots for containers, and support both controlled and uncontrolled use.

04 Give every component the same file structure, a named owner and a place on a dependency map.

Up next → **Chapter 19: Props, in depth**

CHAPTER 19

19

Props, in depth

Props are the settings on a component: the knobs and switches people use to get the version they need. They look like a small detail, but they decide how easy a component is to use, how consistent the product stays, and how painful it is to change things later.

READ THIS IF

You design or build components, review component proposals, or have ever wondered why some components are a joy to use and others are a puzzle.

WHAT YOU'LL LEAVE WITH

01 The different kinds of props, and how each one maps to Figma.

02 Rules for designing props that are clear, safe and consistent.

03 How props work under the surface, and how to change them without breaking anyone.

IN THIS CHAPTER

19.1 Props are a promise
19.2 The kinds of props
19.3 Designing good props
19.4 How props actually work
19.5 Props in Figma, and documenting them
19.6 Changing props without breaking anyone
19.7 Prop patterns to avoid

19.1

Props are a promise

Go back to the TV remote from Chapter 15. A good remote has a few clearly labelled buttons that do exactly what they say. Nobody has to read a manual to turn the volume up. Now imagine a remote where "volume up" only works if "mode B" is on, and pressing "mute" while "input 2" is selected changes the channel. That's what a component with badly designed props feels like.

Props matter for three reasons. They are **the interface** designers and engineers use every day, so clarity saves time on every screen. They **decide consistency**: if a prop lets anyone pick any colour, they will, and the product drifts. And they are **a promise**: once teams use a prop, changing or removing it breaks their screens, so mistakes in props are expensive to undo.

That's why mature system teams spend more time discussing props than almost anything else. A useful mindset is to treat each prop as something you'll have to support for years. Add one only when you're sure it earns its place.

19.2

The kinds of props

Almost every prop in every component library falls into one of eight kinds. Knowing them helps you spot what a component needs, and spot props that don't belong.

KIND	WHAT IT DOES	EXAMPLE IN CODE	IN FIGMA
Variant	Picks one approved version from a fixed list	`variant="primary" | "secondary" | "danger"`	Variant property
Size	Picks one approved size	`size="sm" | "md" | "lg"`	Variant property
Switch	Turns one thing on or off	`disabled`, `loading`, `fullWidth`	Boolean property
Content	Text or information to show	`label="Email"`, `helperText="…"`	Text property
Slot	Lets you put other components inside	`icon={<SendIcon />}`, children	Instance swap or slot
State	The current value, and its starting value	`value`, `defaultValue`, `open`	Usually a variant ("state")
Event	What to do when something happens	`onClick`, `onChange`, `onOpenChange`	Not represented (prototype only)
Accessibility	Extra information for assistive technology	`aria-label="Close dialog"`	Usually a note in the spec

The Figma column matters. Figma's component properties come in a few types (variant, boolean, text and instance swap) that line up neatly with most of these kinds.[65] When the two sides use the same property names and values, a designer's choices in Figma translate directly into code, which is the "twins" idea from Chapter 12 working at the level of individual settings.

19.3

Designing good props

Here are the rules we use when reviewing props. Each one comes from a mistake we've seen cause real pain.

Prefer one choice over many switches

The most common prop mistake is using several on/off switches for things that are really one choice. If a button has `isPrimary`, `isDanger` and `isGhost`, what happens when someone turns on two at once? Nobody knows. A single `variant` prop with a list of allowed values makes impossible combinations impossible.

```
// × Three switches, eight combinations,
//   most of them meaningless
<Button isPrimary isDanger>
  Delete
</Button>
```

```
// ✓ One choice, three valid options
<Button variant="danger">
  Delete
</Button>
```

Use the same names everywhere

If buttons use `variant`, badges shouldn't use `type` and alerts shouldn't use `kind`. If buttons use `size="sm"`, inputs shouldn't use `size="small"`. Write down a short list of standard prop names (variant, size, tone, disabled, loading, label, onChange…) and use them across the whole library. People then learn the library once, not component by component.

Make the default the right answer

Whatever a prop is set to when nobody sets it should be the most common, safest choice. A button should default to the medium size and the secondary variant, so the "big orange button" is a deliberate decision, not an accident. A text field should default to having a visible label, not to hiding it.

Don't let props bypass the system

Props like `color="#FF3300"` or `padding={13}` let anyone escape the foundations, and someone always will. Props should offer *choices from the system* (`tone="critical"`, `size="lg"`), never raw values.

Only require what's essential

A component should work with as few props as possible. A button needs a label and something to do when clicked; everything else should have a sensible default. Every required prop is something someone will forget.

19.4

How props actually work

You don't need to be an engineer to understand the basic mechanics, and knowing them helps designers and engineers talk about components much more precisely.

Information flows one way

In most modern frameworks, information flows **down** through props, and news flows **up** through events. The screen tells the component what to show (via props). When something happens inside the component, like a click or a change, the component tells the screen (via an event prop such as `onChange`). The screen decides what to do, and if needed, sends new props back down.

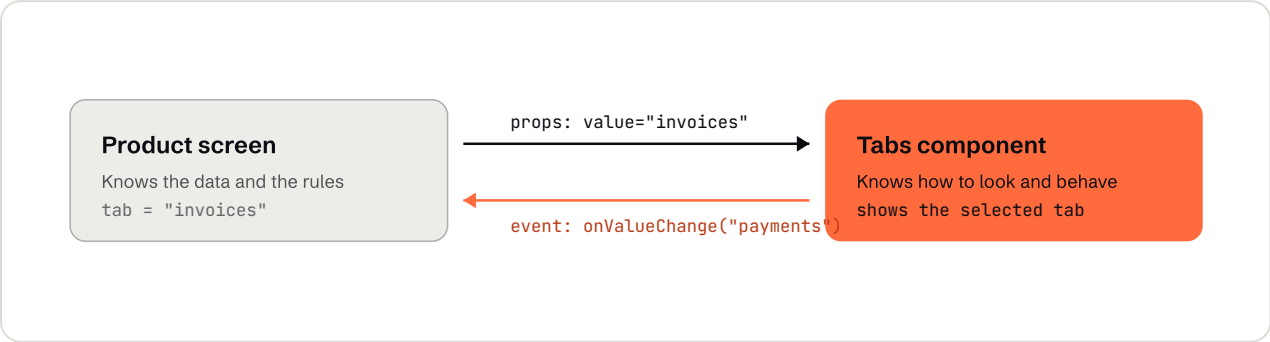


FIG 19.1 **Props flow down, events flow up.** The screen owns the information; the component owns how it looks and behaves. When someone clicks a tab, the component reports it, and the screen decides what happens next.

This one-way flow is what makes components predictable. Given the same props, a component always looks the same. That's also what makes visual testing (Chapter 15) possible: render each combination of props, take a picture, compare.

Controlled and uncontrolled, again

Chapter 18 introduced the idea that either the screen or the component can keep track of a value. In terms of props, an uncontrolled component takes a `defaultValue` and looks after itself; a controlled component takes a `value` plus an event like `onValueChange`, and the screen stays in charge.[64] Good libraries name these consistently across every component that has a value, so engineers only learn the pattern once.

Passing things through

Components should usually pass along standard properties they don't use themselves, such as an ID for testing or an accessibility attribute, to the underlying element. Engineers call this "spreading" props. It means the system doesn't block teams from doing ordinary, legitimate things, without having to add a special prop for every case.

19.5

Props in Figma, and documenting them

For the twins to stay identical, props need to look the same in both places. Here's how Ledgerly's button lines up.

CODE PROP	TYPE	DEFAULT	FIGMA PROPERTY	WHAT IT'S FOR
`variant`	primary · secondary · danger	secondary	Variant: *Variant*	The importance of the action. One primary per screen area.
`size`	sm · md · lg	md	Variant: *Size*	Match the density of the surrounding UI.
`icon`	icon name	none	Boolean *Has icon* + instance swap *Icon*	An optional icon before the label.
`loading`	true / false	false	Boolean: *Loading*	Shows progress and prevents double clicks.
`disabled`	true / false	false	Variant: *State = Disabled*	Can't be used right now. Explain why nearby.
`children`	text	—	Text: *Label*	The label. Start with a verb: "Send invoice".
`onClick`	event	—	— (prototype only)	What happens when it's pressed.

Notice two small differences that are agreed and documented: Figma shows the icon as two properties (a switch and a choice of icon), while code uses one; and Figma treats "disabled" as a visual state, while code treats it as a switch. Differences like this are fine **as long as they're written down**. What causes trouble is differences nobody decided.

This table is also the right format for documentation. Every component's documentation page should list its props with the same five columns: name, type, default, the Figma equivalent and a plain-language purpose. Many teams generate the first three columns automatically from the code, so they're never out of date, and write the last two by hand.

19.6

Changing props without breaking anyone

Props are promises, so changing them needs care. Here's how each kind of change should be handled, using the version-number rules from Chapter 05.

CHANGE	RELEASE TYPE	HOW TO DO IT SAFELY
Add a new optional prop	Minor	Give it a default that keeps today's behaviour. Nobody notices unless they want it.
Add a new value to a variant	Minor	As above. Update Figma at the same time.
Change a default value	Major	Everyone relying on the old default sees a change. Announce it, with screenshots.
Rename a prop	Major (eventually)	Accept both names for a while, warn when the old one is used, provide an automatic fix, remove the old name in the next major release.
Remove a prop or value	Major	Deprecate first, track usage, give a notice period, then remove.

SCENARIO

Renaming "kind" to "variant" across 600 buttons

THE SITUATION

Ledgerly's early button used `kind`. Every other component uses `variant`. The inconsistency confuses new engineers.

WHAT THEY DO

Version 4.3 accepts both `kind` and `variant`, and shows a gentle warning in development when `kind` is used. They publish a small script that rewrites `kind=` to `variant=` automatically.

THE RESULT

Most teams run the script in a few minutes. Usage tracking shows when the old name is gone. Version 5.0 removes `kind`, and nobody's screen breaks.

NEW WORD

Codemod

A small script that automatically rewrites code, for example renaming a prop everywhere it's used. It turns a day of manual edits into a minute.

19.7

Prop patterns to avoid

Finally, here are the prop patterns we most often recommend removing in audits. If you spot these in your library, they're usually signs of a component that's trying to do too much.

AVOID

Raw style props

`color`, `margin`, `fontSize` as free values. They bypass the foundations. Offer named choices instead.

AVOID

Switch explosions

Many booleans that interact. Combine related ones into a single choice.

AVOID

One-team props

A prop added for one screen in one product. It becomes everyone's maintenance burden. Build it locally instead.

AVOID

"Config" objects

A single prop that takes a big bundle of settings. It hides what the component can do and makes changes hard to track.

AVOID

Contradictory props

Props that make no sense together, like `loading` and `disabled` with different meanings. Define clearly which wins.

AVOID

Hidden behaviour

A prop whose name doesn't match what it does, like `compact` that also hides the icon. Names should never surprise.

THE SHORT VERSION

What to remember from this chapter

01 Props are a long-term promise. Add one only when it earns its place.

02 Use one choice instead of many switches, the same names everywhere, and safe defaults.

03 Props flow down and events flow up; support both controlled and uncontrolled use.

04 Match props in Figma and code, document differences, and change props through deprecation and automatic fixes.

Up next → **Chapter 20: Components and tokens**

CHAPTER 20

20

Components and tokens

Part 02 showed how tokens hold your foundation decisions. This chapter shows how components plug into them: which parts of a component become tokens, how to name them, when a component deserves its own tokens, and how to keep the connections healthy as everything changes.

READ THIS IF

You're connecting components to tokens, planning theming or multiple brands, or your dark mode keeps breaking in specific components.

WHAT YOU'LL LEAVE WITH

01 A clear map from a component's parts to the tokens it uses.

02 A naming pattern and a decision guide for component-level tokens.

03 How tokens connect to Figma and code, and how to audit them over time.

IN THIS CHAPTER

20.1 Where components meet tokens
20.2 Which parts of a component become tokens
20.3 Naming component tokens
20.4 When does a component deserve its own tokens?
20.5 Tokens in Figma and in code
20.6 Keeping tokens and components in sync

20.1

Where components meet tokens

A quick reminder of the three levels of tokens from Chapter 07. **Primitive** tokens are the raw paints and sizes (`brand.700` , `space.4`). **Semantic** tokens name a job (`action.primary.bg` , `text.secondary`). **Component** tokens name a specific use inside a specific component (`button.primary.background`). Each level points to the one below.

The question this chapter answers is: *how should a component get its values?* There are two sensible answers and one bad one.

GOOD

Component uses semantic tokens

The button's background is `action.primary.bg`. Simple and enough for most single-brand products.

GOOD

Component has its own tokens

The button's background is `button.primary.background`, which points to `action.primary.bg`. More flexible, more to maintain.

BAD

Component uses primitives or raw values

The button's background is `brand.700` or `#AB2800`. Dark mode and theming break here first.

SCENARIO

Dark mode works everywhere except the date picker

THE SITUATION

Ledgerly launches dark mode. Every screen looks right, except the date picker, whose selected day stays bright orange on a white square.

THE CAUSE

The date picker was built quickly and used `brand.700` and `neutral.50` directly. Those are paints, not jobs, so dark mode had no way of switching them.

THE FIX, AND THE LESSON

Point the date picker at job names (`action.primary.bg` , `bg.surface`) and add an automatic check that flags any component using primitives. One component fixed, a whole class of bugs prevented.

20.2

Which parts of a component become tokens

Start from the component's anatomy (Chapter 12). For each part, ask: which of its visual properties could ever need to change with a theme, a brand, a density or a platform? Those properties should come from tokens. Here's the map for Ledgerly's text field.

PART	PROPERTY	COMPONENT TOKEN	POINTS TO (SEMANTIC)	CHANGES WITH
Label	Text style	`input.label.font`	`type.label.md`	Density
	Colour	`input.label.color`	`text.primary`	Theme
Container	Background	`input.container.background`	`bg.surface`	Theme
	Border	`input.container.border`	`border.strong`	Theme
	Border, error	`input.container.border.error`	`feedback.danger.text`	Theme
	Border, focus	`input.container.border.focus`	`focus.ring`	Theme
	Height	`input.container.height`	`size.control.height`	Density
	Padding	`input.container.padding-x`	`space.control.inset-x`	Density
	Corners	`input.container.radius`	`radius.control`	Brand
Helper text	Colour	`input.helper.color`	`text.secondary`	Theme
Error message	Colour	`input.error.color`	`feedback.danger.text`	Theme

FIG 20.1 Mapping a text field's anatomy to tokens. Eleven properties, each with a component token that points to a semantic token. The last column shows which kind of change each property responds to.

Two things are worth noticing. First, every property that responds to a theme, brand or density is a token. Properties that never change, like the fact that the label sits above the field, are just part of the component's design and don't need tokens. Second, this map is useful even if you decide *not* to create component tokens. It tells you exactly which semantic token each part should use, which is what prevents the date picker problem.

20.3

Naming component tokens

Component tokens multiply quickly. Ten components with ten tokens each is already a hundred names. A consistent naming pattern is what keeps them guessable (Layer One's Rule #3). This pattern works well:

```
{component}.{variant}.{part}.{property}.{state}

button.primary.container.background             // default state can be left out
button.primary.container.background.hover
button.primary.label.color
input.container.border.error                // no variant needed
table.row.background.selected
```

Use the anatomy part names from the component's documentation, so the token name, the anatomy diagram and the Figma layer names all match. Leave out levels that don't apply rather than inventing filler like "default". And use the same property words everywhere: if one component says `background`, none should say `bg` or `fill`.

> ■ PRACTICE **TEST YOUR NAMES WITH A NEWCOMER**
>
> Show a new designer or engineer the anatomy diagram of a component they haven't seen, and ask them to guess the token for, say, the border of the error state. If they get it right, or nearly right, your naming works. If they can't, simplify.

20.4

When does a component deserve its own tokens?

Component tokens are powerful, but they have a real cost: more names to create, document and keep in step across Figma and code. Nathan Curtis has even cautioned that the industry may be "focused too much on design tokens" at the expense of other parts of a system.[25] So don't create them by default. Create them when you have a reason.

SITUATION	COMPONENT TOKENS?	WHY
One brand, light and dark mode	`usually no`	Semantic tokens already switch with the theme.
Several brands that differ only in colour	`usually no`	A brand level under the semantic tokens handles it (Chapter 07).
Brands that style specific components differently (rounder buttons, flat cards)	`yes`	Each brand can override just those components' tokens.
White-label products where customers adjust specific components	`yes`	Gives a safe, limited list of things customers can change.
A component that genuinely differs from the semantic rule	`yes, for that part`	Records the exception in one visible place instead of hiding it in code.
Native apps that need platform-specific sizes	`yes, for sizes`	iOS and Android can map component sizes to their own guidelines.

A good middle path, which many teams use, is to define component tokens *only for the properties that actually vary*. Ledgerly might give buttons a `button.container.radius` token, because one brand wants pill-shaped buttons, while letting the button's colours use semantic tokens directly. That keeps the token list short and purposeful.

20.5

Tokens in Figma and in code

The connection between a component and its tokens has to exist on both sides of the twins.

In Figma

Figma's variables let designers store token values and attach them to the properties of a component's layers: a fill, a stroke, padding, corner radius.[66] Variables can have several modes, such as light and dark or brand A and brand B, so switching a mode restyles every component that uses them. When the variable names match the token names in code, designers and engineers are literally talking about the same thing.

In code

The token pipeline generates the same values for code, typically as CSS variables on the web. The component's styles then reference only those names. Here's how a second brand changes just the button's shape, without touching any component code:

```
/* tokens: component level, in the shared
format */
{
  "button": {
    "container": {
      "radius": {
        "$type": "dimension",
        "$value": "{radius.control}"
      }
    },
    "primary": { "container": { "background":
{
      "$type": "color",
      "$value": "{color.action.primary.bg}"
    } } }
  }
}
```

```
/* generated CSS */
:root {
  --button-container-radius: var(--radius-
control);
  --button-primary-container-background:
    var(--color-action-primary-bg);
}
/* brand B overrides one component token */
[data-brand="b"] {
  --button-container-radius: 999px;
}

/* Button.css never changes */
.button { border-radius: var(--button-
container-radius); }
```

That's the whole point of component tokens in one picture: brand B gets pill-shaped buttons by changing one value, and no component code is touched. The token format used here is the shared Design Tokens Community Group format from Chapter 06.[32]

20.6

Keeping tokens and components in sync

Tokens and components drift apart over time, just like Figma and code do. A component gets a quick fix with a raw value. A token is renamed but one component still uses the old name. A token is created for a feature that was later removed. A light quarterly audit catches most of this.

CHECK	WHAT IT FINDS	HOW
Raw values in components	Colours, sizes or spacing typed directly into component styles	An automatic lint rule on every code change
Primitives used in components	Components pointing at paints instead of jobs (the date picker problem)	A lint rule that only allows semantic and component tokens
Unused tokens	Tokens no component or product uses any more	A script comparing the token list with usage in code
Missing tokens	Properties that change with themes but have no token	Switch every theme and density in the workshop and look for parts that don't change
Figma–code mismatch	Variables in Figma that don't match tokens in code	Compare the exported variable list with the token source

Before changing any token, check its blast radius: which components and products use it? The same dependency map from Chapter 18 works here. A token used by one component is safe to change; a token used by forty is a release with notes and screenshots.

THE SHORT VERSION

What to remember from this chapter

01 Components should use semantic or component tokens, never primitives or raw values.

02 Map each part of a component's anatomy to the token it uses; that map prevents theme bugs.

03 Create component tokens only where something genuinely varies by brand, platform or customer.

04 Connect tokens in Figma (variables) and code (CSS variables) with matching names, and audit them every quarter.

Up next → **Chapter 21: Components as code**

CHAPTER 21

21

Components as code

Chapter 15 introduced how components are built and shipped. This chapter goes deeper: what a well-built component looks like line by line, how to style it, how to make it accessible in code, which technology to choose, and how design and engineering stay aligned once the code exists.

READ THIS IF

You're an engineer building system components, a design lead who wants to review them with confidence, or a decision-maker choosing technologies for the system.

WHAT YOU'LL LEAVE WITH

01 A complete, annotated example of a well-built component.

02 A comparison of styling approaches and technology choices.

03 A shared definition of done and a review checklist for design and engineering.

IN THIS CHAPTER

21.1 A well-built component, line by line

21.2 Ways to style components

21.3 Accessibility, in code

21.4 Choosing the technology

21.5 Keeping design and engineering aligned

21.6 Quality beneath the surface

21.1

A well-built component, line by line

Here's Ledgerly's button in React with TypeScript, two of the most common technologies for web design systems. If you don't write code, read the grey comments; they explain what each part is doing and why it matters. Everything in this chapter applies, with different syntax, to other frameworks.

```
// 1. The props, written down as a contract. TypeScript checks every use of the
//    button against this list, so a typo like variant="primery" is caught instantly.
type ButtonProps = {
  variant?: "primary" | "secondary" | "danger";   // one choice, not three switches
  size?: "sm" | "md" | "lg";
  icon?: IconName;                                  // a named icon from the system, not any image
  loading?: boolean;
} & React.ButtonHTMLAttributes<HTMLButtonElement>; // plus every normal button attribute

// 2. forwardRef lets screens reach the real button element, for focus management.
export const Button = React.forwardRef<HTMLButtonElement, ButtonProps>(function Button(
  // 3. safe defaults: secondary, medium, not loading
  { variant = "secondary", size = "md", icon, loading = false, disabled, children, ...rest },
  ref
) {
  return (
    <button                                     // 4. a real <button>: keyboard support is free
      ref={ref}
      type="button"                             // 5. never submits a form by accident
      className="button"
      data-variant={variant}                    // 6. styles read these; no colours in this file
      data-size={size}
      disabled={disabled || loading}            // 7. no double clicks while saving
      aria-busy={loading || undefined}          // 8. screen readers hear that it's working
      {...rest}                                 // 9. pass through ids, tests, aria-labels…
    >
      {loading ? <Spinner size="sm" /> : icon && <Icon name={icon} aria-hidden />}
      <span className="button__label">{children}</span>
    </button>
  );
});
```

A few of these lines deserve a little more explanation, because they're exactly the details that separate a system component from one built quickly for a single screen.

Using a real `<button>` element (comment 4) is the single most important accessibility decision in the file. Native buttons can be reached with the Tab key, pressed with Enter and Space, and are announced as "button" by screen readers, all without any extra code. Teams that build buttons out of plain boxes have to recreate all of that by hand, and usually miss something.

No colours or sizes in the file (comment 6). The component only says *which* variant and size it is; the stylesheet decides what that looks like, using tokens. That's why a rebrand never needs to touch this file.

Passing through everything else (comment 9) means teams can add a test ID or an accessible label without asking the system team for a new prop. The system stays small, and teams aren't blocked.

21.2

Ways to style components

There are several ways to attach styles to components in code, and teams debate them passionately. The good news is that if your tokens are CSS variables, every approach can use them, so the choice matters less than it seems.

APPROACH	HOW IT WORKS, SIMPLY	GOOD FOR	WATCH OUT FOR
Plain CSS with CSS variables	A normal stylesheet per component, using token variables	Simple, fast, works with any framework	Needs naming discipline to avoid clashes
CSS modules	Like plain CSS, but class names are made unique automatically	No clashes between components; easy to read	Requires a build step
CSS-in-JS	Styles written in JavaScript next to the component	Styles that depend on props; everything in one file	Can affect performance and server rendering; less popular than a few years ago
Utility classes	Many tiny ready-made classes combined in the markup	Very fast for product teams; tokens map to classes	Markup gets long; harder to enforce component-level rules

PRACTICE **OUR USUAL RECOMMENDATION**

For the system's own components: plain CSS or CSS modules, reading token variables, with variants selected by data attributes as in 21.1. It's fast, framework-independent and easy for anyone to read. Product teams can then use whatever approach they like for their own screens, as long as it reads the same token variables.

Whichever approach you choose, two rules hold. Styles only ever use tokens (Chapter 20). And styles should use *logical* properties, such as `padding-inline` rather than `padding-left` and `padding-right`, so layouts mirror correctly for right-to-left languages like Arabic and Hebrew without extra work.[67]

21.3

Accessibility, in code

Chapter 14 covered accessibility from the design side. In code, it comes down to a few habits that, once learned, become second nature.

Use the right element first

HTML already includes accessible elements for most jobs: buttons, links, form fields, labels, lists, headings, tables. They come with keyboard support and screen-reader meaning built in. Reach for extra accessibility attributes (ARIA) only when no native element fits. A common saying among accessibility specialists is that no ARIA is better than bad ARIA, because incorrect attributes can make things worse than none at all.

Follow the established patterns for complex components

For composites like menus, tabs, comboboxes and dialogs, the W3C's ARIA Authoring Practices Guide describes exactly how each should behave: which keys do what, where focus goes, what gets announced.[61] Following it means your components behave the way assistive-technology users already expect. Here's what that means for a dialog:

REQUIREMENT	WHAT THE CODE MUST DO
Announce itself	Identify itself as a dialog and use its title as its name
Move focus in	When it opens, move keyboard focus into the dialog (usually to the first field or the close button)
Keep focus inside	Tab and Shift+Tab cycle within the dialog, never into the page behind it
Close on Escape	The Escape key closes it, unless closing would lose unsaved work
Return focus	When it closes, focus goes back to the button that opened it
Hide the background	The page behind is hidden from screen readers while the dialog is open

Six requirements, every one of which a quickly built dialog tends to miss, and all of which the system's dialog solves once for every product. That's the strongest argument for building complex components centrally.

21.4

Choosing the technology

A design system's components have to be built in the same technology as the products that use them. That makes technology choice one of the biggest cost decisions in the whole system (Chapter 04).

YOUR SITUATION	COMMON APPROACH	TRADE-OFF
All web products use one framework	Build components in that framework	Simplest and best developer experience. Tied to that framework.
Web products use several frameworks	Web Components (a browser standard) or a framework-neutral core with thin wrappers	One implementation works everywhere, but integration with each framework needs care.[68]
Web plus iOS and Android apps	Shared tokens; separate native components per platform	Native feel and performance, but each component is built two or three times.
Legacy products that can't adopt new components	Ship tokens (CSS variables) and styles only	Visual consistency without rewriting, but no shared behaviour.

Whatever you choose, notice the pattern: **tokens are the part that works everywhere.** Components may need to be built once per technology, but tokens are generated for every platform from one source. That's why tokens come first (Rule #4), and why they're the backbone when a company has a mix of old and new technology.

21.5

Keeping design and engineering aligned

Most design–engineering friction in design systems comes from one question: *when is a component actually done?* If designers think "done" means the Figma file is published and engineers think it means the code is merged, nothing is ever really finished. Agree one definition, for both twins.

Definition of done (both twins)

- [ ] Figma component and code component have the same name, props and values
- [ ] Every variant and state exists in both, and in the workshop
- [ ] Styles use tokens only; lint passes
- [ ] Keyboard and screen-reader behaviour tested by a person
- [ ] Visual, behaviour and automated accessibility tests pass
- [ ] Documentation page complete (Chapter 16 template)
- [ ] Release note written; Figma library and code package released together

Review questions for system pull requests

- [ ] Does this change any prop, default or visual size? Then which version number changes?
- [ ] Which components and products depend on what's changing?
- [ ] Do the workshop pictures show only the intended changes?
- [ ] Is Figma updated in the same release?
- [ ] Is any new token or prop genuinely needed by two or more teams?
- [ ] Has someone from design reviewed the result in the workshop?

The workshop (Chapter 15) is the meeting point. Designers review the *real* coded component there, in every state, rather than reviewing screenshots. Many teams also link each Figma component to its code and documentation, so it's one click from design to implementation and back. The tooling varies; the habit of reviewing the real thing together is what matters.

21.6

Quality beneath the surface

System components are used everywhere, so their quality affects every product's speed and stability. A few engineering concerns deserve attention even though users never see them directly.

CONCERN	WHY IT MATTERS	GOOD PRACTICE
Package size	Every product downloads the components it uses; heavy components slow every product.	Let products import only what they use; watch size on every release.
Dependencies	Each outside library a component relies on adds size, risk and upgrade work.	Keep outside dependencies few and well chosen; prefer browser features.
Server rendering	Many products render pages on the server for speed; components must work there too.	Test components in server-rendered pages before release.
Performance with real data	A list that's fine with 20 items may freeze with 10,000.	Test level 4 components with realistic data volumes (Chapter 17).
Backwards compatibility	Teams update at different speeds.	Follow the versioning rules; support the previous major version for a stated time.

THE SHORT VERSION

What to remember from this chapter

01 Good components use native elements, safe defaults, typed props, tokens only, and pass through standard attributes.

02 Any styling approach works if it reads token variables; use logical properties for right-to-left support.

03 Follow the established accessibility patterns for complex components, and test with a keyboard and a screen reader.

04 Agree one definition of done for both twins, and review the real coded component together.

Up next → **Chapter 22: A casebook of real situations**

CHAPTER 22

22

A casebook of real situations

Principles are easy to agree with and hard to apply on a Tuesday afternoon with a deadline. This chapter works through twelve situations design system teams meet again and again, showing the options, what we'd recommend and why.

READ THIS IF

You want to see how the ideas in Parts 03 and 04 play out in practice, or you're facing one of these situations right now.

WHAT YOU'LL LEAVE WITH

01 Worked answers to twelve common component situations.

02 A repeatable way of thinking: options, recommendation, reasons.

03 A migration plan you can adapt for retiring old components.

IN THIS CHAPTER

22.1 When teams need different things

22.2 Migrations: retiring the old and bringing in the new

22.3 Changes that could break things

22.4 When something goes wrong

22.5 Pushing the limits

22.6 Saying no, and listening

22.1

When teams need different things

Each case follows the same shape: the situation, the realistic options, what we'd recommend, and why. They're composites of situations we see in practice, not specific clients. ILLUSTRATIVE

CASE 1 Two teams, two very different date pickers

Payments needs a picker that shows bank holidays and blocks weekends. Reporting needs to select ranges of whole months. Both ask for "the date picker".

OPTIONS

(a) One picker with options for everything. (b) Two separate pickers. (c) A solid core picker, plus extensions built on it.

RECOMMENDATION

(c). The core picker handles single dates and ranges, with a way to mark days as unavailable. Reporting builds a month-range picker as a shared extension; payments supplies its holiday list to the core picker.

WHY

Option (a) creates a level 4 monster. Option (b) duplicates the hardest part, the calendar grid and its keyboard behaviour. Option (c) shares the hard part and keeps the special cases out of the core (Chapter 05, tiers).

CASE 2 Marketing wants a "special" button for a campaign

The campaign team wants a large gradient button with an animated arrow for a two-week launch page.

OPTIONS

(a) Add a "campaign" variant to the core button. (b) Refuse. (c) Build it locally on the campaign page, using system primitives and tokens.

RECOMMENDATION

(c), with a quick review from the system designer for accessibility (contrast on the gradient, reduced-motion support).

WHY

It's a one-off with a short life. Adding it to the core makes it everyone's maintenance forever; refusing makes the system look obstructive. A local build keeps both the campaign and the core healthy.

22.2

Migrations: retiring the old and bringing in the new

CASE 3 **Three dropdowns, one system: a migration plan**

The three dropdowns from Chapter 15 are used about 450 times across four products. The system now has one good dropdown. How do you move everyone across without stopping product work?

OPTIONS

(a) A "migration sprint" where every team stops and switches. (b) Leave it and hope. (c) A phased plan with tooling and a clear end date.

RECOMMENDATION

(c). See the timeline below.

WHY

Big-bang migrations compete with product deadlines and usually stall halfway. Phased migrations with automation and visible progress finish.

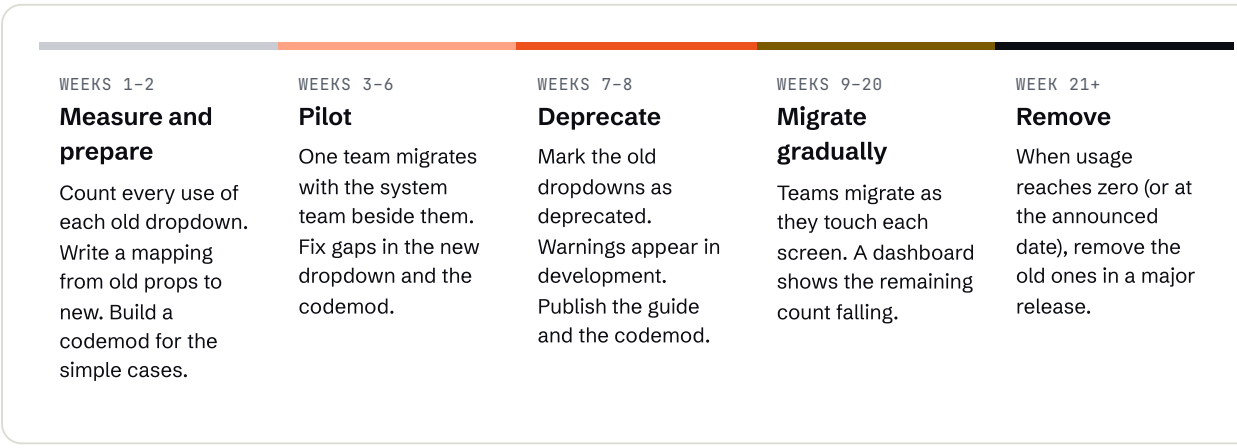


FIG 22.1 **A phased migration.** ILLUSTRATIVE The timings are illustrative. The shape is what matters: measure, pilot, deprecate, migrate gradually with visible progress, then remove.

CASE 4 **An acquired product built with a different technology**

The company acquires a startup whose product uses a different front-end framework. Leadership wants it to "look like us" within six months.

OPTIONS

(a) Rewrite the acquired product in your framework. (b) Rebuild all system components in their framework. (c) Start with tokens, then the most visible components.

RECOMMENDATION

(c). Ship your tokens as CSS variables into their product first: colours, type and spacing change in weeks. Then restyle their existing buttons, inputs and navigation to use those tokens.

WHY

Tokens work in any technology (Chapter 21). They deliver most of the visual consistency for a fraction of the cost, and buy time for a proper decision about the code later.

22.3

Changes that could break things

CASE 5 Making every button 4 pixels taller

Research shows buttons are slightly too small on touch laptops. The fix is to increase the default height from 36 to 40 pixels.

OPTIONS

(a) Ship it as a patch; it's "just a visual tweak". (b) Add a new size and leave the default alone. (c) Treat it as a breaking change, with notice.

RECOMMENDATION

(c). Release it in the next major version, with before-and-after screenshots from the workshop, a test period in a pre-release, and a note telling teams which layouts to check (table rows, toolbars).

WHY

Visual changes that affect layout can break products as surely as code changes.[26] Option (b) avoids the problem today but leaves most of the product with the worse default.

CASE 6 Renaming a widely used token

The token `color.text.muted` is confusingly close to `color.text.subtle`. The team decides to merge them into `color.text.secondary`.

OPTIONS

(a) Rename and fix everything at once. (b) Keep both names forever. (c) Add the new name, point the old names at it, deprecate, then remove.

RECOMMENDATION

(c). The old names become aliases of the new one, so nothing changes visually. A lint rule suggests the new name; a codemod replaces it automatically.

WHY

Aliases make renaming safe. The same deprecation path used for components (Chapter 16) works for tokens.

22.4

When something goes wrong

CASE 7 An accessibility bug found in production

A customer reports that screen readers don't announce errors in the system's text field. It's used in every form, in every product.

WHAT TO DO

Confirm it, fix it in the component, add a test that would have caught it, and ship a patch release the same week. Announce it in the system channel and release notes.

THEN

Check which products are on old versions that won't get the fix, and help them update. If the issue affects a legal commitment, tell the relevant owners directly.

WHY IT'S GOOD NEWS

This is the system working. One fix reaches every form. Without a system, it would be dozens of separate bugs in dozens of backlogs.

CASE 8 A component that ignores dark mode

After launching dark mode, three components stay bright. Investigation finds raw colours and primitive tokens in their styles.

IMMEDIATE FIX

Map each part of the three components to the right semantic tokens (the anatomy map from Chapter 20) and release a patch.

PREVENT IT

Add lint rules that block raw colours and primitive tokens in component styles. Add dark mode to every visual test.

WHY

The bug isn't the three components; it's the missing guardrail. Fix the class of problem, not just the instances.

22.5

Pushing the limits

CASE 9 A table that must handle 10,000 rows

The analytics team needs a table with 10,000 rows, frozen columns and inline editing. The system's table handles a few hundred rows comfortably and has no editing.

OPTIONS

(a) Extend the core table to do everything. (b) A specialised data grid as a separate component. (c) A well-chosen third-party grid, styled with system tokens.

RECOMMENDATION

(b) or (c), depending on how many teams need it. Keep the core table simple. If only analytics needs this, (c) is usually the fastest, safest route.

WHY

Spreadsheet-like grids are a level 4 problem of their own (Chapter 17). Stretching the simple table to do it would make it worse for the 90% of screens that just need a list.

CASE 10 Launching in Arabic and German

The company expands into the Middle East and Germany. Arabic reads right to left; German labels are often much longer than English ones.

WHAT TO CHECK

Do components use logical properties so they mirror automatically?[67] Do labels wrap or truncate gracefully? Do icons that imply direction (arrows) flip?

WHAT TO ADD

Right-to-left and long-text versions in every component's stress test (Chapter 14) and in visual tests.

WHY NOW

Fixing direction and text length once in each component is far cheaper than fixing them in every screen of every product.

22.6

Saying no, and listening

CASE 11 "Can we add a prop just for our screen?"

An engineer asks for a `hideLabelOnMobile` prop on the text field, for one compact filter bar.

QUESTIONS TO ASK

Do other teams need this? Is there an existing way (a visually hidden label that screen readers still read)? Would it cause accessibility problems?

RECOMMENDATION

Usually: show the existing accessible way to hide a label visually, and explain why the label must remain for screen readers. No new prop.

WHY

One-team props are how components grow forty settings (Chapter 19). A helpful "here's how" beats both a new prop and a flat no.

CASE 12 A designer who detaches everything

Library data shows one designer detaching most components they use. It looks like they're ignoring the system.

THE TEMPTING REACTION

Ask their manager to enforce the rules.

THE BETTER REACTION

Sit with them for an hour. In our experience, it often turns out they work on the product area the system serves worst, and their detachments are a list of what's missing.

WHY

Workarounds are feedback (Chapter 16). The heaviest "rule-breakers" are often your best source of requirements.

THE PATTERN BEHIND ALL TWELVE CASES

Keep the core simple and strict. Give special needs a respectable home outside it. Change things with notice, tools and visible progress. And treat every workaround as information.

THE SHORT VERSION

What to remember from this chapter

01 Shared core plus extensions handles most "different needs" without bloating components.

02 Migrations finish when they're phased, automated and visible, not when they're big-bang.

03 Treat layout-affecting visual changes and renames as breaking; use aliases and notice.

04 Fix the guardrail, not just the bug, and listen to the people working around the system.

Up next → **Part 05: Patterns, content and scale**



05

Patterns, content and scale

Components are the words; patterns are the sentences. This final part covers how components combine into proven solutions, how words become part of the system, how a system stretches across brands, platforms and business units, the tools that keep it running, and a twelve-month plan that ties the whole book together.

23 Patterns

24 Content and language

25 Scaling across products

26 Tools, automation and AI

27 Your first twelve months

CHAPTER 23

23

Patterns: from components to solutions

Components are the words of your interface. Patterns are the sentences: proven ways of combining components to solve a problem people meet again and again, like filling in a form, finding something in a long list, or recovering from an error.

READ THIS IF

Your product has consistent components but still feels inconsistent, or teams keep solving the same problems (forms, tables, empty screens) in different ways.

WHAT YOU'LL LEAVE WITH

01 What a pattern is, and how it differs from a component.

02 Practical guidance for the most common patterns: forms, tables, navigation and feedback.

03 How to document and look after patterns as part of the system.

IN THIS CHAPTER

23.1 Words and sentences

23.2 Forms

23.3 Tables, lists and finding things

23.4 Navigation

23.5 Feedback: loading, empty, error and success

23.6 Documenting and looking after patterns

23.1

Words and sentences

You can have a perfect dictionary and still write confusing sentences. The same thing happens with design systems. Every team uses the system's button, text field and table, so the components are consistent, but each team combines them differently. One form shows errors at the top, another under each field, a third in a pop-up. One list paginates, another scrolls forever. The product still feels like several products.

NEW WORD	
Pattern	A proven, documented way of combining components to solve a recurring user problem, such as "how we validate a form" or "how we show that there are no results".

That's what **patterns** fix. A pattern isn't a new component. It's guidance on how to use existing components together to solve a specific problem, with the reasoning and research behind it. The UK government's GOV.UK Design System is the best-known public example: alongside its components, it publishes patterns for things like asking for dates, addresses and names, each based on research with real users.[63] DOCUMENTED

SCENARIO

Four ways to say "nothing here"

THE SITUATION

A Ledgerly customer searches invoices, filters payments, opens an empty report and views a new project. Every component on those screens comes from the system.

WHAT THEY SEE

"No data", a blank white area, a sad-face illustration with "Oops!", and "0 results found" in small grey text. Four teams, four different answers to the same question.

WHAT A PATTERN CHANGES

One documented empty-state pattern with three versions (first use, no results, cleared), each with a clear message and a next step. Same components, now saying the same thing.

23.2

Forms

Forms are where products make money and where users get most frustrated, so they're usually the first pattern worth writing down. Here's the guidance we start from. PRACTICE

QUESTION	OUR DEFAULT ANSWER	WHY
Layout	One column, labels above fields	Easiest to scan; works on every screen size; labels stay close to their fields.
Required fields	Mark the optional ones ("optional") rather than starring the required ones, when most fields are required	Fewer marks, less noise; clear wording beats a symbol.
When to check for errors	When the person leaves a field (for format problems) and when they submit (for everything)	Checking while someone is still typing shows errors before they've finished.
Where to show errors	A message next to each problem field, plus a summary at the top on submit that links to each error	People and screen readers can find every problem; the GOV.UK error summary is a well-tested model.[69]
What errors say	What's wrong and how to fix it: "Enter a date after 1 January 2026"	Errors must be identified and described in text.[70] STANDARD
Long forms	Split into steps with a clear progress indicator, or into sections with headings	Smaller chunks feel achievable and reduce mistakes.
Buttons	One primary action at the end, labelled with what it does ("Send invoice")	The main action should be obvious and specific.

The point isn't that these answers are the only right ones. Some products have good reasons to differ. The point is that *your* product has one answer for each question, written down, so every form feels familiar and every team stops re-debating the same choices.

23.3

Tables, lists and finding things

Most business software is, at heart, a collection of lists: invoices, customers, orders, users. How people find, sort and act on items in those lists decides how fast they can work.

Table, list or cards?

TABLE

When people compare

Many items with the same attributes, where people scan columns, sort and compare: invoices, transactions.

LIST

When people pick

Items with one or two key details, where people mostly find one and open it: messages, notifications.

CARDS

When items are visual

Items where an image or a preview matters more than comparison: templates, projects, products.

Pagination, "load more" or endless scrolling?

APPROACH	BEST FOR	DOWNSIDES
Pages	Work tools where people need to find their place again or reach the end	Extra clicks; page size choice matters
"Load more" button	Feeds and search results where people mostly look at the first few	Long pages; footer hard to reach after loading lots
Endless scrolling	Browsing content with no particular goal	Hard to return to a spot; footer unreachable; heavy for keyboard and screen-reader users

Search, filters and bulk actions

Write down how filters work across the product: where they sit (above the list, or in a side panel), whether results update instantly or after pressing "Apply", how active filters are shown and cleared, and what the empty result looks like. For bulk actions, decide once how people select many items (checkboxes, with a "select all on this page" option), where the actions appear (a bar that appears when items are selected), and how destructive bulk actions are confirmed. These small decisions, made consistently, are what make a large product feel learnable.

23.4

Navigation

Navigation is the pattern people notice least when it works and most when it doesn't. The system should define the app frame (Chapter 10) and the rules for moving around inside it.

COMPONENT	USE IT FOR	DON'T USE IT FOR
Side menu	The main sections of an app with many areas (5+)	Actions; settings for the current page
Top navigation	A few main sections (up to about 5), often on websites	Deep hierarchies
Tabs	Switching between views of the *same* thing (an invoice's details, payments, history)	Moving between different areas of the app
Breadcrumbs	Showing where you are in a deep hierarchy and stepping back up	Flat products with one or two levels
Back link	Returning from a detail page or a step in a flow	Replacing the browser's own back button behaviour

■ PRACTICE **THREE LEVELS IS USUALLY ENOUGH**

If people need to go more than three levels deep to reach something they use often (section, page, tab), the problem is usually the structure of the product, not the navigation component. That's a conversation for product leadership, and the design system is often the first to notice it.

23.5

Feedback: loading, empty, error and success

Every screen has moments when things aren't normal: data is loading, there's nothing to show, something went wrong, or something just succeeded. These moments are where products most often feel inconsistent, because each team handles them as an afterthought.

Choosing how to tell people something

COMPONENT	USE WHEN THE MESSAGE IS…	EXAMPLE
Inline message	About one specific field or section	"Enter a full email address" under the field
Toast	A short confirmation that needs no action, and that it's fine to miss	"Invoice sent"
Banner	About the whole page or account, and important enough to stay visible	"Your trial ends in 3 days"
Dialog	Something people must decide before they continue	"Delete this customer and 12 invoices?"

Loading

For short waits, show a skeleton (grey shapes where the content will appear) so the page doesn't jump when it loads. For actions like saving, show progress on the button that was pressed. For long operations, show what's happening and, if possible, how long it will take. And always handle the case where loading fails, with a clear way to try again.

Empty states

FIRST USE

Nothing yet

Explain what will appear here and offer the first step: "No invoices yet. Create your first invoice to get paid faster."

NO RESULTS

Search or filters found nothing

Say what was searched, and suggest changing or clearing filters.

ALL DONE

Everything is cleared

A small, positive confirmation: "All invoices are paid." No action needed.

23.6

Documenting and looking after patterns

Patterns need documentation even more than components do, because there's no code to copy: the guidance *is* the pattern. A consistent template helps.

#	SECTION	WHAT IT COVERS
1	**The problem**	What people are trying to do, in one or two sentences
2	**The solution**	How we solve it, which components are used, and in what arrangement
3	**Examples**	Real screens from your products, good ones and ones to improve
4	**Variations**	Approved versions for different situations (for example, a short form vs a long one)
5	**When not to use it**	Situations where a different pattern fits better
6	**Evidence**	Research, test results or analytics that support the approach
7	**Accessibility and writing**	Keyboard flow, announcements, and the words to use

Patterns change more often than components, because they're closer to product decisions. Give each pattern an owner, often a senior product designer rather than the system team, and review patterns when research or analytics suggest something isn't working. When a pattern becomes stable and is built the same way everywhere, it may be worth turning into a "recipe" or composite component (Chapter 18), so teams can use it rather than rebuild it.

THE SHORT VERSION

What to remember from this chapter

01 Patterns are documented ways of combining components to solve recurring problems.

02 Start with forms, lists, navigation and feedback states; these cause the most inconsistency.

03 Have one written answer to each recurring question, with reasons, and evidence where you have it.

04 Document patterns with a consistent template, give each an owner, and promote stable ones into recipes.

Up next → **Chapter 24: Content and language in the system**

CHAPTER 24

24

Content and language in the system

A product can have perfectly consistent buttons and still feel chaotic if one says "Remove", another "Delete" and a third "Discard" for the same action. Words are part of the interface, so they belong in the design system too.

READ THIS IF

Your product's wording feels inconsistent, you're about to translate your product, or writers and designers keep debating the same phrases.

WHAT YOU'LL LEAVE WITH

01 How to define a voice and adjust tone for different moments.

02 A shared vocabulary and reusable writing formulas.

03 What it takes to write for everyone, and to prepare for translation.

IN THIS CHAPTER

24.1 Why words belong in the system

24.2 Voice and tone

24.3 A shared vocabulary

24.4 Formulas for common messages

24.5 Writing for everyone

24.6 Getting ready for other languages

24.1

Why words belong in the system

People read interfaces in fragments: a button label, a heading, an error message. When those fragments use different words for the same thing, people hesitate. Is "Archive" the same as "Remove"? Will "Discard" lose my work? Is a "workspace" the same as an "organisation"? Every hesitation is a small cost, and in a large product they add up.

SCENARIO **Five words for "make it go away"**

THE SITUATION

Across Ledgerly, removing something uses "Delete", "Remove", "Archive", "Discard" and "Clear", chosen by whoever built each screen.

WHAT GOES WRONG

Customers can't tell which actions are permanent. Support tickets ask how to recover "archived" invoices that were actually deleted. Translators get five different words to translate.

WHAT THE SYSTEM ADDS

A vocabulary entry: "Delete = permanent, always confirmed. Archive = hidden, recoverable. Remove = taken out of a list, not deleted." Three words, three meanings, used everywhere.

Content in a design system usually covers four things: **voice and tone** (how the product sounds), **vocabulary** (which words mean what), **formulas** (reusable shapes for common messages) and **localisation rules** (what has to work in other languages). None of it needs to be long. A few pages, well maintained, do more than a 40-page style guide nobody opens.

24.2

Voice and tone

Think of the difference between personality and mood. Your **voice** is your product's personality: it stays the same everywhere. Your **tone** is the mood: it adapts to the moment, the way a good colleague speaks differently when celebrating a win and when delivering bad news.

Ledgerly's voice, for example, might be: *clear, calm and helpful*. Its tone then adjusts:

MOMENT	TONE	EXAMPLE
Success	Brief and warm. Don't over-celebrate routine tasks.	"Invoice sent to Acme Ltd."
Error	Calm and specific. Never blame the person.	"We couldn't process this payment. The bank declined it. Try another card."
Warning	Clear about the consequence, and what to do.	"This invoice is overdue by 14 days. Send a reminder?"
Destructive action	Serious and explicit. Name exactly what will happen.	"Delete this customer? Their 12 invoices will also be deleted. This can't be undone."
First use	Encouraging and practical. Point to the next step.	"No invoices yet. Create your first one to get paid faster."

A MISTAKE WE SEE OFTEN

Jokes in error messages

"Oops! Something went wonky!" might feel friendly in a design review. To someone whose payroll just failed, it feels dismissive. Keep personality for calm moments; in stressful moments, be clear and kind.

24.3

A shared vocabulary

The single most useful content artefact is a simple vocabulary list: the words your product uses, what they mean, and the words to avoid. It helps designers, engineers, support and translators all at once.

USE	MEANING	DON'T USE
Delete	Permanently remove. Always confirmed.	Destroy, erase, discard
Archive	Hide from the main list; can be restored.	Hide, store
Customer	A business you invoice.	Client, account, buyer
Workspace	Your company's area in Ledgerly, shared by its team.	Organisation, tenant, account
Sign in / Sign out	Start or end a session.	Log in, login, log out
Due date	The date an invoice must be paid by.	Deadline, payment date

Small rules that add up

A handful of formatting rules prevent a surprising amount of inconsistency. Decide them once: sentence case or title case for headings and buttons (we recommend sentence case: "Send invoice", not "Send Invoice"); how dates are written; whether numbers use words or digits; how currencies are shown; and whether you write "you" and "your" when addressing people (we recommend it). Put them on one page, next to the vocabulary.

24.4

Formulas for common messages

Most interface text falls into a few types of message. Giving each type a simple formula means anyone can write a good message, quickly, in the product's voice. Chapter 14 introduced the idea for individual components; here are the formulas side by side.

MESSAGE	FORMULA	EXAMPLE
Button	Verb + object	"Send invoice", "Add customer"
Error	What happened + why (if known) + what to do	"Payment failed. The card has expired. Update the card details."
Field error	What to enter, specifically	"Enter an amount between ₹1 and ₹10,00,000"
Confirmation dialog	Question naming the action + consequence; buttons repeat the action	"Delete 3 invoices? This can't be undone." [Delete invoices] [Cancel]
Empty state	What will appear here + how to start	"Your payments will appear here. Connect a bank account to get started."
Success	What happened, briefly	"Customer added."

Notice the confirmation dialog's buttons. "Delete invoices" and "Cancel" are far clearer than "Yes" and "No", because people often read only the button. This one small habit prevents a lot of accidental deletions.

24.5

Writing for everyone

Good interface writing is accessible writing. Plain words, short sentences and clear structure help everyone, and they matter most for people reading in a second language, people with cognitive or learning disabilities, and anyone who is tired, stressed or in a hurry.

PRINCIPLE	IN PRACTICE
Plain words	"Use" not "utilise"; "start" not "initiate"; explain jargon or avoid it.
Front-load meaning	Put the important words first: "Invoice sent", not "We have successfully sent your invoice".
Descriptive links	"View payment history", not "Click here". Screen-reader users often navigate by a list of links.
Meaningful image text	Describe what an image tells people; mark purely decorative images so screen readers skip them.
Inclusive language	Avoid assumptions about gender, ability or culture; use "they" when you don't know; avoid idioms that don't translate.
Don't rely on direction or colour	"Select the highlighted option" or "click the button on the right" fails for many people. Name the thing.

24.6

Getting ready for other languages

Even if you only support one language today, a few habits now will save a painful rebuild later. Translation touches components, patterns and code, so it's very much a design system concern.

HABIT	WHY IT MATTERS
Leave room for longer text	Many languages need noticeably more space than English for the same message. Components should wrap or grow, not cut text off.
Support right-to-left	Arabic and Hebrew read right to left; layouts and directional icons should mirror. Logical CSS properties make this mostly automatic.[67]
Never build sentences from pieces	"You have " + count + " invoices" breaks in languages with different word order or plural rules. Keep whole sentences together, with placeholders.
Handle plurals properly	Some languages have several plural forms, not just "one" and "many". Use your platform's built-in plural rules.[71]
Format dates, numbers and money by locale	"1,00,000" in India is "100,000" elsewhere; dates differ too. Use built-in formatting rather than hand-written formats.[71]
Give translators context	"Post" could be a verb or a noun. Add a note or a screenshot to every string.

Who owns the words?

In larger companies, content designers or UX writers own the voice, vocabulary and formulas, just as system designers own the visual foundations. In smaller ones, it's often a senior designer or product manager with a little protected time. Either way, treat content guidance like any other part of the system: it has an owner, it's versioned, changes are announced, and teams can propose improvements. Some teams even add simple automatic checks, flagging words from the "don't use" list in code reviews.

THE SHORT VERSION

What to remember from this chapter

01 Words are part of the interface; inconsistent words confuse people as much as inconsistent buttons.

02 Keep one voice, adjust the tone to the moment, and be calm and specific when things go wrong.

03 A short vocabulary list and a few message formulas do more than a long style guide.

04 Write plainly and inclusively, and build components and code ready for other languages from the start.

Up next → **Chapter 25: Scaling across products, brands and platforms**

CHAPTER 25

25

Scaling across products, brands and platforms

A design system that works beautifully for one product and one team gets tested in new ways when the company grows: a second brand, a mobile app, an acquisition, a new business unit with its own ideas. This chapter is about stretching the system without breaking it.

READ THIS IF

Your system needs to serve several brands, several platforms or several business units, or you're planning for that future.

WHAT YOU'LL LEAVE WITH

01 How to support several brands and platforms from one source of decisions.

02 How "families of systems" work, and when you need one.

03 Governance and funding models that work across business units.

IN THIS CHAPTER

25.1 What changes when you scale

25.2 Several brands from one system

25.3 Several platforms from one source

25.4 Families of systems

25.5 Governance and funding across business units

25.6 Legacy products and long migrations

25.1

What changes when you scale

Scaling a design system isn't just "the same thing, bigger". New kinds of difference appear: different brands that must look different, different platforms that must feel native, and different business units with different priorities and budgets. Each one tempts teams to start their own system.

SCENARIO **One group, three brands, three systems**

THE SITUATION

A financial group runs three consumer brands: a bank, an insurance company and a lending app. Each brand has its own design team, and each built its own design system.

WHAT GOES WRONG

Three teams maintain three date pickers, three accessibility fixes, three token pipelines. When a regulation changes the rules for how fees must be displayed, it's implemented three times, differently.

WHAT SCALING WELL LOOKS LIKE

One shared core (behaviour, accessibility, structure, foundations) with a brand layer on top. Each brand still looks like itself; the hard work is done once.

The guiding principle for everything in this chapter is simple: **share what should be the same, and make what should differ easy to differ**. Behaviour, accessibility and structure should almost always be shared. Visual identity, and sometimes platform conventions, should be allowed to vary, in controlled ways.

25.2

Several brands from one system

Chapter 07 introduced brand tokens, and Chapter 20 showed how component tokens let a brand restyle specific components. Put together, they make a multi-brand system practical. Here's what usually stays shared and what varies.

LAYER	SHARED ACROSS BRANDS	VARIES BY BRAND
Behaviour	Keyboard use, focus, announcements, validation logic	Nothing
Structure	Component anatomy, props, patterns	Rarely; only through agreed extensions
Semantic tokens	The *names* of the jobs	Which values the names point to
Colour	The lightness curve and contrast rules	Brand hues, and sometimes neutral tints
Typography	The roles (heading, body, label) and accessibility rules	Font families, sometimes the scale
Shape and motion	The token names	Corner rounding, shadows, animation style
Imagery and voice	Accessibility rules	Illustrations, photography, tone of voice

The result is that a new brand is mostly a new set of values, not a new system. Because the lightness curve is shared (Chapter 07), every brand's colours meet the same contrast rules automatically, even when their hues are completely different. A regulation change to how fees are shown becomes one pattern update, inherited by all three brands.

> ■ PRACTICE **TEST WITH THE MOST DIFFERENT BRAND FIRST**
>
> When building a multi-brand system, design the first components against your two most different brands at once, not one brand first. It's much easier to find the right flexibility points while building than to add them afterwards.

25.3

Several platforms from one source

Web, iPhone and Android each have their own conventions. Apple's guidelines and Google's Material Design describe different navigation, different controls and different text sizes.[72] [52] A good multi-platform system respects those conventions while keeping the brand and the experience recognisably the same.

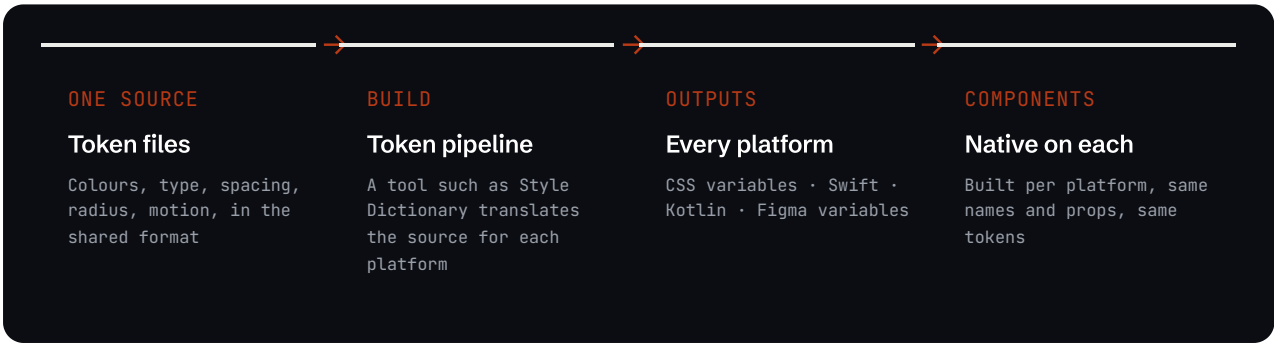


FIG 25.1 One source of decisions, every platform. Tokens are written once and generated for each platform by a build tool. Style Dictionary is a widely used open-source example.[73] Components are then built natively, sharing names, props and tokens.

DECIDE TO…	USUALLY	EXAMPLES
Share	Brand, foundations, content, component names and purposes	Colours, voice, "primary button", error message formulas
Adapt	Things where the platform has strong conventions	Navigation patterns, date pickers, switches, text sizes, haptics
Keep separate	Things that only exist on one platform	Widgets, watch faces, keyboard shortcuts on desktop

A useful test: would a regular user of this platform be surprised by how this behaves? If yes, adapt to the platform. If no, share it. People expect an iPhone app to feel like an iPhone app, while still recognising your brand.

25.4

Families of systems

At a certain size, one system can't sensibly hold everything. A bank's customer app, its internal operations tools and its marketing site have such different needs that forcing them into one library makes it worse for all three. The answer many large companies arrive at is a **family of systems**: a shared core, with specialised systems built on top of it.

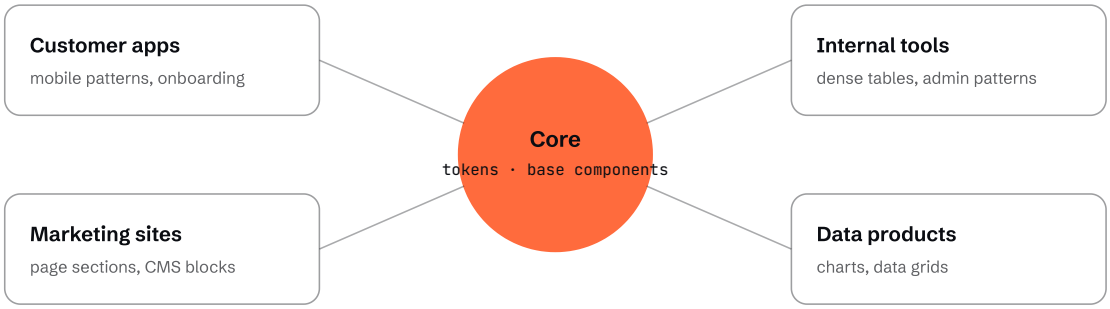


FIG 25.2 **A family of systems.** Each specialised system builds on the shared core and is owned by the teams closest to its users. Nathan Curtis describes this as tiers below a high-quality core;[20] Spotify's Encore is a well-known example, which brought 22 separate systems under one shared foundation.[19]

A family of systems only works with a few clear rules. **Everyone uses the core tokens**, so the brand and accessibility baseline stay consistent. **Specialised systems build on core components rather than copying them**, so fixes flow outwards. **Each specialised system has a named owning team**, usually the one closest to its users. And **things move into the core when they're needed widely**, following the rule of two from Chapter 05. Without these rules, a family of systems slowly becomes the "22 separate systems" problem again.

25.5

Governance and funding across business units

When several business units depend on one system, two questions become political very quickly: *who decides*, and *who pays*. Answering them openly, early, prevents a lot of quiet resentment.

Who decides

Most large organisations settle on a small **council**: the system lead plus a design and engineering representative from each major business unit, meeting on a regular rhythm for decisions that affect everyone (foundations, breaking changes, the roadmap). Day-to-day decisions stay with the system team and contributors, as described in Chapter 05. The council is for the few decisions that genuinely need everyone in the room.

Who pays

FUNDING MODEL	HOW IT WORKS	WORKS WELL WHEN	RISK
Central budget	The system is funded centrally, like shared infrastructure	Leadership sees the system as a platform	Business units feel no ownership
Contribution of people	Each business unit gives a share of designers and engineers to the system	Units are similar in size and need	Contributed people get pulled back to unit work
Mixed	A central core team plus contributors funded by units for their extensions	Most mature, multi-unit organisations	Needs clear rules about core vs extension

SCENARIO

"Our unit wants to opt out"

THE SITUATION

One business unit complains the system is too slow for its needs and asks to build its own.

WHAT TO DO

Treat it as feedback first: what's missing, and why is it slow? Often the answer is an extension owned by that unit, built on the core, with a fast lane for their needs.

WHY

Opting out fully costs the unit the shared accessibility, brand and maintenance work, and costs everyone consistency. A well-supported extension usually gives them the speed they want without either cost.

25.6

Legacy products and long migrations

At scale, there's always an older product that can't adopt the new system quickly: different technology, no team assigned, or a complete rewrite planned "next year". Don't let these products hold the system back, and don't leave them behind entirely.

01 **Tokens first**

Ship the token values (as CSS variables or a theme file) into the legacy product. Colours, type and spacing align quickly, and the product looks part of the family (Chapter 22, case 4).

02 **Replace the most visible parts**

Navigation, buttons and forms, where users notice inconsistency most. Replace them as screens are touched, not all at once.

03 **Measure the whole portfolio**

Track adoption per product on one simple dashboard, so leadership sees progress across the whole company, not just in the newest products.

04 **Agree an end state**

For each legacy product, decide openly: full adoption, tokens only, or retirement. A clear plan is better than an endless "someday".

THE SHORT VERSION

What to remember from this chapter

01 Share behaviour, accessibility and structure; let brand and platform conventions vary in controlled ways.

02 One token source can feed every brand and platform; components are built natively with the same names.

03 At large scale, use a family of systems: a shared core plus specialised systems with named owners.

04 Decide openly who decides and who pays, and bring legacy products along with tokens first.

Up next → **Chapter 26: Tools, automation and AI**

CHAPTER 26

26

Tools, automation and AI

Tools don't make a design system, but the right ones make it much cheaper to run. This chapter maps the toolchain of a typical system, shows which automation pays back first, and looks practically at where AI helps today, and where it doesn't.

READ THIS IF

You're choosing tools for a design system, spending too much time on manual upkeep, or wondering how AI fits into design system work.

WHAT YOU'LL LEAVE WITH

01 A map of the tools a design system uses, and what each one is for.

02 The minimum tool set for each stage of maturity, and the automations worth building first.

03 Realistic, useful ways to use AI in design system work, with the cautions that matter.

IN THIS CHAPTER

26.1 Tools serve the process, not the other way round
26.2 The toolchain, mapped
26.3 The right tools for your stage
26.4 Automation that pays for itself
26.5 Where AI helps today
26.6 Using AI responsibly

26.1

Tools serve the process, not the other way round

It's tempting to start a design system by choosing tools: a documentation platform, a token manager, a component workshop. Tools feel like progress. But a tool without an owner and a process quickly becomes an expensive, out-of-date mirror of what's really happening.

SCENARIO **The documentation site nobody updated**

THE SITUATION

A company buys a polished documentation platform and spends a month filling it with beautiful pages for every component.

WHAT GOES WRONG

Nobody's job includes keeping it current. Within six months, half the pages describe old versions. Teams stop trusting it and ask in Slack instead.

WHAT WOULD HAVE WORKED

Docs as part of the definition of done (Chapter 21), with parts generated from code automatically, so updating them is part of shipping, not a separate chore.

So the order matters: decide how the system will work (Chapter 05), then pick tools that make that way of working easier. Layer One's seventh rule applies here too: *build where the system lives*. Put the system into the files, repositories and tools your teams already use every day, rather than asking them to visit somewhere new.

26.2

The toolchain, mapped

Most design systems use tools in six areas. You don't need something in every area on day one, but it helps to see the whole picture and how the pieces connect.

AREA	WHAT IT DOES	TYPICAL TOOLS	CONNECTS TO
Design	Shared components and variables for designers	Figma libraries and variables [66]	Tokens (variables), docs (links)
Tokens	One source of decisions, translated for every platform	Token files in the shared format; [32] a build tool such as Style Dictionary [73]	Design variables, code packages
Code	The components, packaged and versioned	Your framework, a package registry, a component workshop such as Storybook [62]	Tokens, tests, docs
Documentation	How and when to use everything	A dedicated documentation platform, or a docs site generated from the code	Code (live examples), design (links)
Quality	Catch problems before they ship	Lint rules, visual regression tests, automated accessibility checks	Code, pull requests
Insight	Know what's used and where	Design library analytics; scripts that scan product code	The health scorecard (Chapter 05)

FIG 26.1 The six areas of a design system toolchain. The arrows that matter most run from the token source into both design and code. That single connection keeps the twins in step.

26.3

The right tools for your stage

A startup doesn't need an enterprise toolchain, and an enterprise can't run on a shared spreadsheet. Here's the minimum tool set we'd suggest for each of the maturity stages from Chapter 02. Add more only when a real problem calls for it. PRACTICE

STAGE	DESIGN	TOKENS	CODE	DOCS	QUALITY AND INSIGHT
1 · Shared styles	One Figma library	A shared file of values	Themed off-the-shelf library	A single page	Design reviews
2 · Basics in code	Library + variables	Token files + build tool	Your package + workshop	Workshop pages	Lint for raw values
3 · Looked after	Libraries split by purpose	Light and dark modes	Versioned releases	A docs site	Visual and accessibility tests; usage scripts
4 · Platform	Brand and platform libraries	Multi-brand, multi-platform	Several packages and platforms	Docs platform with search	Health dashboard; automated migrations

PRACTICE **KEEP YOUR DECISIONS PORTABLE**

Whatever tools you choose, keep your tokens in the shared, vendor-neutral format and your components in your own repository. Tools come and go; your decisions shouldn't be locked inside any single product.

26.4

Automation that pays for itself

Every hour the system team spends on repetitive upkeep is an hour not spent helping product teams. These automations, roughly in the order we'd build them, give the best return. PRACTICE

#	AUTOMATION	WHAT IT SAVES	EFFORT
1	**Token pipeline**	Hand-copying values into code and design; mismatches between them	Days
2	**Lint rules**	Hard-coded colours and spacing slipping in; reviewers policing by hand	Days
3	**Visual regression tests**	Accidental visual changes; nervousness about every release	1–2 weeks
4	**Automated accessibility checks**	Common accessibility mistakes reaching products	Days
5	**Release notes from changes**	Writing release notes from memory at the last minute	Days
6	**Usage dashboard**	Guessing which components and versions are used where	1–2 weeks
7	**Codemods for migrations**	Teams manually rewriting code for breaking changes	Per change

Notice that the first two are cheap and prevent the most common problems in the whole book: values drifting between design and code, and hard-coded values bypassing the foundations. If you automate nothing else, automate those.

26.5

Where AI helps today

AI tools are changing design system work quickly, and it's easy to either overhype them or dismiss them. Here's a grounded view of where they genuinely help, based on how teams are using them now. Layer One's founder has worked on exactly this kind of tooling: AI-assisted drafting of company-wide design patterns, and tools that check design files and code against a design system for hundreds of designers and over a thousand engineers.[1]

USE	HOW IT HELPS	KEEP A HUMAN…
Drafting documentation	Turns a component's code, props and examples into a first draft of its documentation page	…to check accuracy and add the "when not to use" judgement
Writing release notes	Summarises code changes into plain-language notes for product teams	…to judge what's breaking and what teams must do
Checking against the system	Flags designs or code that use off-system colours, spacing or components, and suggests the system equivalent	…to decide when a deviation is deliberate and fine
Answering "which component?"	An assistant that has read your docs answers everyday questions in chat	…to keep the docs it reads accurate
Migrations	Helps write and apply codemods for renames and replacements	…to review every change before it merges
Drafting patterns	Proposes a first pattern write-up from examples across products	…to bring research, evidence and taste

There's a pattern in that table. AI is good at *drafting* and *checking*, and it's only as good as the information it's given. A well-structured system, with named tokens, consistent props, documented components and clear rules, turns out to be excellent input for AI tools. A messy one gives messy results. In that sense, the work in this book is also the best preparation for using AI well.

26.6

Using AI responsibly

A few cautions are worth taking seriously, because the cost of a design system mistake is multiplied across every product.

WATCH OUT FOR

Confident mistakes

AI tools can produce plausible but wrong documentation or code. In a system used by hundreds of people, a wrong statement in the docs spreads quickly. Review everything.

WATCH OUT FOR

Accessibility by assumption

Generated components often look right but miss keyboard behaviour or announcements. Test them like any other component (Chapter 21).

WATCH OUT FOR

Inventing new things

Ask an AI to "build a settings page" and it may invent components and colours instead of using yours. Point it at your system and check the result against it.

WATCH OUT FOR

Confidentiality

Unreleased designs and internal code are sensitive. Use tools your company has approved, with appropriate data protections.

Questions to ask before adopting any tool

- [ ] What problem does it solve that we actually have today?
- [ ] Who will own it and keep it current?
- [ ] Does it work with our existing files, repositories and formats, or does it lock our decisions in?
- [ ] How will we know, in three months, whether it was worth it?
- [ ] What happens to our work if we stop using it?

THE SHORT VERSION

What to remember from this chapter

01 Decide how the system works first; choose tools that make that easier, and keep decisions portable.

02 Match the toolset to your maturity stage. More tools aren't better.

03 Automate the token pipeline and lint rules first; they prevent the most common problems.

04 Use AI for drafting and checking, keep humans for judgement, and remember that a well-structured system makes AI far more useful.

Up next → **Chapter 27: Putting it all together: your first twelve months**

CHAPTER 27

27

Putting it all together: your first twelve months

You've seen the whole system: why it matters, what it costs, how to run it, its foundations, its components and how it scales. This final chapter turns all of that into a practical plan for your first year, the rhythms that keep a system healthy, and one checklist for the whole journey.

READ THIS IF

You're about to start (or restart) a design system and want a single plan that connects everything in this book.

WHAT YOU'LL LEAVE WITH

01 A quarter-by-quarter plan across foundations, components, operations and adoption.

02 The weekly, monthly and quarterly rhythms of a healthy system.

03 A master checklist, and the ten most common ways systems fail.

IN THIS CHAPTER

27.1 The whole book on one page

27.2 A twelve-month plan

27.3 The rhythms of a healthy system

27.4 The master checklist

27.5 Ten ways design systems fail, and the antidote

27.6 A closing note

27.1

The whole book on one page

Before the plan, here's how the five parts of this book fit together. Each part answers a different question, and each one depends on the parts before it.

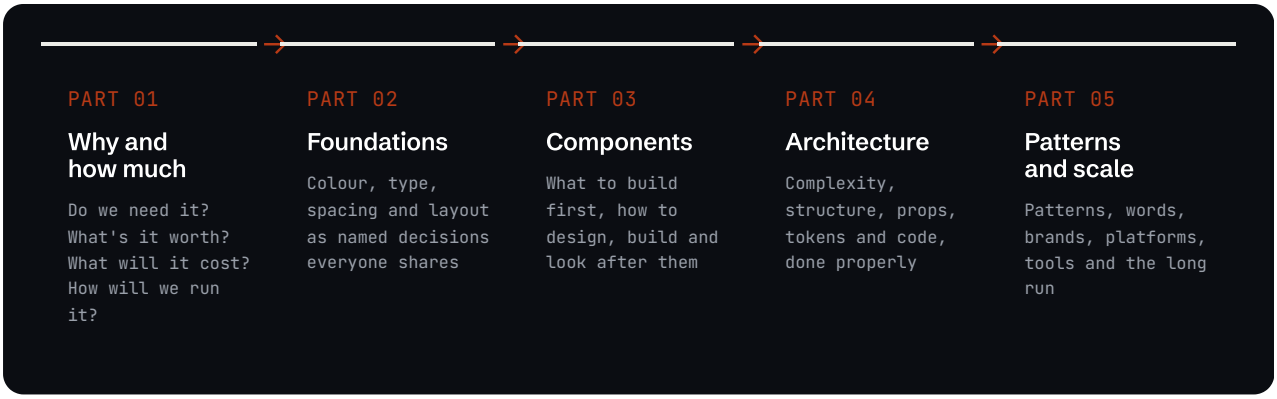


FIG 27.1 Five parts, one system. In practice you'll work on several at once: foundations and the first components overlap, and the operating model from Part 01 runs underneath everything.

If you remember only one idea from the whole book, make it this one: **a design system is an agreement, not a library**. The components, tokens and documentation are how the agreement is written down. The people, processes and rhythms are how it stays alive.

27.2

A twelve-month plan

Here's a realistic first year for a mid-sized product company, like Ledgerly, starting from Stage 1 or 2 (Chapter 02). It runs four workstreams side by side. Each quarter ends with a gate (Chapter 04): something you can point to that earns the next quarter's investment. ILLUSTRATIVE

	QUARTER 1 · START	QUARTER 2 · PROVE	QUARTER 3 · SPREAD	QUARTER 4 · SETTLE
Foundations	Audit. Colour, type, spacing, layout as tokens. Token pipeline.	Semantic tokens complete. Contrast checks automated.	Dark mode as a second theme.	Density modes. Token audit.
Components	8 basics: button, field, select, checkbox, radio, switch, link, alert.	Containers and feedback: dialog, card, tabs, toast, menu, empty state.	Data and navigation: table, pagination, badge, tag, side menu.	The hard ones: date picker, search, advanced table features.
Operations	Owner named. Who-decides-what agreed. Versioning rules written.	Request and contribution process live. Docs template.	First outside contribution. Monthly health check.	Deprecation of old components. Next year's roadmap.
Adoption	Pilot team: new settings page.	Pilot two: onboarding. Champions in each team.	Second and third teams adopt. Office hours.	Migration dashboard. Most new work on the system.
Gate	The pilot ships using only tokens.	The pilot team is measurably faster or has fewer UI bugs.	A team adopts without hand-holding.	Adoption trend and savings reported to leadership.

Two things make this plan work. First, every quarter delivers something teams can use immediately, so the system earns trust as it goes. Second, the components are always built inside a real product project (a pilot), never in isolation. That's Dan Mall's advice from Chapter 05, and in our experience it's the single biggest predictor of a system that people actually use.[29]

27.3

The rhythms of a healthy system

A design system stays healthy through steady habits, not heroic efforts. Here are the rhythms we set up with every team. Put them in the calendar in the first month; they're much harder to add later. PRACTICE

HOW OFTEN	WHAT	WHY	CHAPTER
Daily	Answer the support channel	Fast answers keep people using the system instead of working around it	05
Weekly	Office hours; triage new requests	Help teams build with the system; decide one-off vs shared quickly	05, 13
Every two weeks	Release, with notes for users	A predictable rhythm lets teams plan updates	05, 15
Monthly	Health check; Figma–code "twin check"; usage and detach data	Spot drift and problems early	05, 15, 16
Quarterly	Architecture and token audit; debt review; roadmap update; council (at scale)	Prevent slow tangles; keep the plan honest	18, 20, 25
Yearly	Re-score component complexity; satisfaction survey; budget and staffing review	Step back and decide the next stage	04, 17

27.4

The master checklist

Every chapter ended with its own checklist. Here's the condensed version for the whole journey. Use it as a quarterly review: anything unticked becomes a candidate for the next quarter's plan.

Decide and fund Part 01

- [ ] An audit with "before" numbers
- [ ] A sponsor, engineering commitment and a named owner
- [ ] The next maturity stage chosen, not the last one
- [ ] A budget in people and months, funded in steps
- [ ] An ownership model and a who-decides-what table

Foundations Part 02

- [ ] OKLCH palette; colours named by job; contrast tested
- [ ] Text styles by role; 4px spacing scale; breakpoints
- [ ] One token source feeding design and code

Components Part 03

- [ ] Components chosen by frequency and pain
- [ ] Every state designed; "ready to build" checklist
- [ ] A tidy, documented library with life-cycle labels

Architecture Part 04

- [ ] Complexity scored; process matched to level
- [ ] Layers with downward-only dependencies
- [ ] Consistent props, matched in Figma and code
- [ ] Components use semantic or component tokens only
- [ ] One definition of done for both twins

Patterns and scale Part 05

- [ ] Forms, lists, navigation and feedback patterns written down
- [ ] Voice, vocabulary and message formulas
- [ ] Ready for other languages and right-to-left
- [ ] A plan for brands, platforms and legacy products
- [ ] Token pipeline and lint rules automated
- [ ] The rhythms in 27.3 in everyone's calendar

27.5

Ten ways design systems fail, and the antidote

We've seen many design systems in audits. The ones that struggle almost always struggle for one of these ten reasons. None of them is about the quality of the buttons.

#	HOW IT FAILS	THE ANTIDOTE	CHAPTER
1	Built by design, ignored by engineering	Get engineering commitment before you start; staff the team with engineers	02, 04
2	Treated as a project, then abandoned	Run it as a product, with a team that stays after launch	04, 05
3	Started with the fun components	Start with an inventory and the everyday basics	13
4	Components before tokens	Tokens first, components second	06
5	The system team became a bottleneck	Contribution levels, local components and the rule of two	05, 16
6	Components grew forty settings	Scope statements, splitting, and saying "build it locally"	17, 19
7	Figma and code drifted apart	Same names and props; monthly twin checks; one definition of done	15, 21
8	Updates broke products without warning	Versioning rules that include visual changes; notes; notice periods	05, 22
9	Nobody could find anything	A tidy library, clear names, one documentation template	16
10	Success measured by usage, not value	Baseline first; report time saved and change made cheap	03

27.6

A closing note

When we started this book, we said a design system is a shared set of decisions, reusable parts and working agreements that helps many people build one consistent product. After twenty-six chapters, we hope that sentence means a lot more than it did.

The decisions are your foundations: colours named by their job, text styles with clear roles, a spacing scale, a grid. The reusable parts are your components and patterns, designed for every state and built for everyone. And the working agreements are what keep it all alive: who owns it, how people ask for things, how changes are made safely, how you know it's working.

You don't need to do all of it at once. Pick the next stage, not the last one. Start with an audit, name your basic values, build the ten components your teams use most, and run a pilot. Then keep going, one quarter at a time, letting each step earn the next.

And whatever you build, build it so your own team can run it without outside help. That's the last of Layer One's ten rules, and the best test of whether a design system will last.

> ## Leave the team able to run it without us.
>
> Rule #10 — Layer One

THE SHORT VERSION

What to remember from this book

01 A design system is an agreement, not a library: decisions, parts and the way of working together.

02 Start small and specific: audit, tokens, the everyday components, a real pilot.

03 Run it as a product with steady rhythms, and measure value, not just usage.

04 Build it so your own team can run it for years.

BACK MATTER

References

Every statistic and normative claim in this playbook links here. Accessed November 2026. Where a source is vendor-published or small-sample, the caveat is stated where it is cited.

1 **Layer One — Design systems studio.** Layer One. layerone.co.in, 2026.
www.layerone.co.in/

2 **Atomic Design.** Brad Frost. Self-published, 2016.
atomicdesign.bradfrost.com/

3 **Design tokens.** Salesforce. Lightning Design System, n.d..
v1.lightningdesignsystem.com/design-tokens/
Salesforce's design system team is widely credited with coining the term 'design tokens' (c. 2014).

4 **Design Tokens specification reaches first stable version.** Design Tokens Community Group. W3C, 2025.
www.w3.org/community/design-tokens/2025/10/28/design-tokens-specification-reaches-first-stable-version/

5 **The Mythical Man-Month: Essays on Software Engineering.** Frederick P. Brooks Jr.. Addison-Wesley, 1975.
Source of the observation that communication paths grow as n(n–1)/2 with team size.

6 **Design Systems Report 2025.** zeroheight. zeroheight, 2025.
zeroheight.com/how-we-document
Survey of 294 respondents (Sep–Nov 2024). 79% report a dedicated design system team; average team size ≈3 (<100 employees), ≈4 (100–499), ≈9 (500+).

7 **The Design System Maturity Model.** Ben Callahan. Sparkbox, n.d..
sparkbox.com/foundry/design_system_maturity_model
Four stages, beginning with 'Building Version One' and 'Growing Adoption'.

8 **Measuring the value of design systems.** Figma. Figma blog, n.d..
www.figma.com/blog/measuring-the-value-of-design-systems/
Vendor-run internal experiment: participants with a relevant, up-to-date design system completed a design objective 34% faster. Sample details are limited; treat as indicative.

9 **Yes, Design Systems Do Improve Developer Efficiency and Design Consistency.** Sparkbox. Sparkbox Foundry, n.d..
sparkbox.com/foundry/design_system_roi_impact_of_design_systems_business_value_carbon_design_system
Eight developers built a form from scratch, then with IBM Carbon: median 4.2 h vs 2 h (≈47% faster). Small sample; the scratch build always came first, so familiarity with the design favours the second build.

10 **The WebAIM Million: The 2025 report on the accessibility of the top 1,000,000 home pages.** WebAIM. Utah State University, 2025.
webaim.org/projects/million/2025
Low-contrast text detected on 79.1% of home pages; 94.8% had at least one detectable WCAG 2 failure.

11 **Apparent usability vs. inherent usability: experimental analysis on the determinants of the apparent usability.** Masaaki Kurosu, Kaori Kashimura. CHI '95 Conference Companion, ACM, 1995.
doi.org/10.1145/223355.223680
Origin of the 'aesthetic-usability effect': visual appeal correlated strongly with perceived ease of use.

12 **How do users evaluate the credibility of Web sites? A study with over 2,500 participants.** B. J. Fogg et al.. Proceedings of DUX 2003, ACM, 2003.
doi.org/10.1145/997078.997097
'Design look' was the most frequently mentioned factor in participants' credibility comments.

13 **Web Content Accessibility Guidelines (WCAG) 2.2.** W3C. W3C Recommendation, 2023.
www.w3.org/TR/WCAG22/

14 **Directive (EU) 2019/882 — European Accessibility Act**. European Parliament and Council. Official Journal of the EU, 2019.
eur-lex.europa.eu/eli/dir/2019/882/oj
Applies to many products and services placed on the EU market from 28 June 2025.

15 **A Design System isn't a Project. It's a Product, Serving Products.**. Nathan Curtis. EightShapes, 2015.
medium.com/eightshapes-llc/a-design-system-isn-t-a-project-it-s-a-product-serving-products-74dcfffef935

16 **Team Models for Scaling a Design System**. Nathan Curtis. EightShapes, 2015.
eightshapes.com/articles/team-models-for-scaling-a-design-system
Introduces the solitary, centralised and federated team models.

17 **The Fallacy of Federated Design Systems**. Nathan Curtis. EightShapes, n.d..
nathanacurtis.substack.com/p/the-fallacy-of-federated-design-systems-23b9a9a05542
Argues that successful systems always have a central team, plus federated participation.

18 **The Salesforce Team Model for Scaling a Design System**. Salesforce UX. Medium, n.d..
medium.com/salesforce-ux/the-salesforce-team-model-for-scaling-a-design-system-d89c2a2d404b
Describes the 'cyclical' hybrid: a central team plus a federated group of contributors.

19 **Reimagining Design Systems at Spotify**. Spotify Design. spotify.design, 2020.
spotify.design/article/reimagining-design-systems-at-spotify
Describes Encore, a 'system of systems' that brought 22 disconnected design systems under one framework.

20 **Design System Tiers**. Nathan Curtis. EightShapes, 2019.
medium.com/eightshapes-llc/design-system-tiers-2c827b67eae1
A high-standard core used by everyone, with lower tiers for product- or platform-specific parts.

21 **The Art of Design System Recipes**. Brad Frost. bradfrost.com, n.d..
bradfrost.com/blog/post/the-art-of-design-system-recipes/
Product-specific compositions that live outside the core so the core doesn't fill with one-off components.

22 **A Design System Governance Process**. Brad Frost. bradfrost.com, 2019.
bradfrost.com/blog/post/a-design-system-governance-process/

23 **Contribution criteria**. GOV.UK Design System. Government Digital Service, n.d..
design-system.service.gov.uk/community/contribution-criteria/
Proposals must be useful and unique; before publication, work is checked to be usable, consistent and versatile.

24 **Contributions to Design Systems**. Nathan Curtis. EightShapes, n.d..
medium.com/eightshapes-llc/contributions-to-design-systems-89261a9363d8
Separates small contributions (fixes, objective improvements) from large ones that need a formal review such as an RFC.

25 **"We're focused too much on design tokens": Nathan Curtis on design systems today**. Design Systems Collective. designsystemscollective.com, 2024.
www.designsystemscollective.com/were-focused-too-much-on-design-tokens-nathan-curtis-on-design-systems-today-a329fdd79d4c

26 **Visual Breaking Change in Design Systems**. Nathan Curtis. EightShapes, n.d..
eightshapes.com/articles/visual-breaking-change-in-design-systems/
Argues that changes to colour, type and space can break products as surely as API changes.

27 **Semantic Versioning 2.0.0**. Tom Preston-Werner. semver.org, 2013.
semver.org/

28 **Releasing Design Systems (six-part series: outputs, cadence, versions, breaking change, dependencies, process)**. Nathan Curtis. EightShapes, n.d..
eightshapes.com/articles/versioning-design-systems

29 **Design That Scales: Creating a Sustainable Design System Practice**. Dan Mall. Rosenfeld Media, 2023.
rosenfeldmedia.com/books/design-that-scales/
Includes a chapter on pilots: building the system through real product work.

30 **Spacing**. IBM. Carbon Design System, n.d..
www.carbondesignsystem.com/elements/spacing/overview/

31 **Color system: how the system works**. Google. Material Design 3, n.d..
m3.material.io/styles/color/system/how-the-system-works
Tonal palettes generated in the HCT colour space; colour roles mapped from tones.

32 **Design Tokens Format Module 2025.10**. Design Tokens Community Group. W3C Community Group Final Report, 2025.
www.designtokens.org/tr/2025.10/format/
A Community Group report, not a W3C Standard; widely implemented by token tooling.

33 **A perceptual color space for image processing**. Björn Ottosson. bottosson.github.io, 2020.
bottosson.github.io/posts/oklab/
The original definition of Oklab, from which OKLCH is derived.

34 **CSS Color Module Level 4**. W3C CSS Working Group. W3C, n.d..
www.w3.org/TR/css-color-4/

35 **Tailwind CSS v4.0**. Tailwind Labs. tailwindcss.com, 2025.
tailwindcss.com/blog/tailwindcss-v4
The default colour palette moved to OKLCH.

36 **Understanding Success Criterion 1.4.3: Contrast (Minimum)**. W3C WAI. W3C, 2023.
www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html

37 **Understanding the scale**. WorkOS. Radix Colors documentation, n.d..
www.radix-ui.com/colors/docs/palette-composition/understanding-the-scale
A 12-step scale where each step has a defined job: backgrounds, component states, borders, solids, text.

38 **Understanding Success Criterion 1.4.11: Non-text Contrast**. W3C WAI. W3C, 2023.
www.w3.org/WAI/WCAG22/Understanding/non-text-contrast.html

39 **APCA: Accessible Perceptual Contrast Algorithm**. Myndex Research. git.apcacontrast.com, n.d..
git.apcacontrast.com/documentation/APCA_in_a_Nutshell
A candidate contrast method explored for future WCAG versions. Not normative under WCAG 2.x.

40 **Design tokens**. Atlassian. Atlassian Design System, n.d..
atlassian.design/foundations/tokens/design-tokens

41 **Themes**. IBM. Carbon Design System, n.d..
www.carbondesignsystem.com/elements/themes/overview/

42 **Design tokens**. Adobe. Spectrum, n.d..
spectrum.adobe.com/page/design-tokens/

43 **Variable fonts guide**. MDN contributors. Mozilla, n.d..
developer.mozilla.org/en-US/docs/Web/CSS/CSS_fonts/Variable_fonts_guide

44 **Type scale tokens**. Google. Material Design 3, n.d..
m3.material.io/styles/typography/type-scale-tokens

45 **Utopia: fluid responsive design**. James Gilyead, Trys Mudford. utopia.fyi, n.d..
utopia.fyi/

46 **Typography**. Apple. Human Interface Guidelines, n.d..
developer.apple.com/design/human-interface-guidelines/typography

47 **Understanding Success Criterion 1.4.12: Text Spacing**. W3C WAI. W3C, 2023.
www.w3.org/WAI/WCAG22/Understanding/text-spacing.html

48 **Understanding Success Criterion 1.4.10: Reflow**. W3C WAI. W3C, 2023.
www.w3.org/WAI/WCAG22/Understanding/reflow.html

49 **The Elements of Typographic Style**. Robert Bringhurst. Hartley & Marks, 2004.
Recommends 45–75 characters per line for single-column text, with 66 widely regarded as ideal.

50 **Typography: type sets (productive and expressive)**. IBM. Carbon Design System, n.d..
www.carbondesignsystem.com/elements/typography/type-sets/

51 **Spacing**. Atlassian. Atlassian Design System, n.d..
atlassian.design/foundations/spacing
8px base; tokens from space.025 (2px) to space.1000 (80px), named as multiples of the base.

52 **Designing: structure**. Google. Material Design 3, n.d..
m3.material.io/foundations/designing/structure

53 **Space in Design Systems**. Nathan Curtis. EightShapes, 2016.
Introduces the inset, stack and inline vocabulary for spacing tokens.

54 **Understanding Success Criterion 2.5.8: Target Size (Minimum)**. W3C WAI. W3C, 2023.
www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html

55 **Accessibility**. Apple. Human Interface Guidelines, n.d..
developer.apple.com/design/human-interface-guidelines/accessibility
Recommends a minimum control size of 44×44 pt on iOS/iPadOS.

56 **Layout: breakpoints (window size classes)**. Google. Material Design 3, n.d..
m3.material.io/foundations/layout/breakpoints/overview
Compact < 600dp, medium 600–839dp, expanded 840–1199dp, large 1200–1599dp, extra-large ≥ 1600dp.

57 **2x Grid**. IBM. Carbon Design System, n.d..
www.carbondesignsystem.com/elements/2x-grid/overview/

58 **Design Systems: A practical guide to creating design languages for digital products**. Alla Kholmatova. Smashing Magazine, 2017.
www.smashingmagazine.com/design-systems-book/
Describes systems along three dimensions: strict–loose, modular–integrated, centralised–distributed.

59 **Interface Inventory**. Brad Frost. bradfrost.com, 2013.
bradfrost.com/blog/post/interface-inventory/

60 **Picking Parts, Products & People**. Nathan Curtis. EightShapes, 2016.
eightshapes.com/articles/picking-parts-products-people/

61 **ARIA Authoring Practices Guide (APG)**. W3C WAI. W3C, n.d..
www.w3.org/WAI/ARIA/apg/
Keyboard and screen-reader patterns for common widgets such as menus, tabs, dialogs and comboboxes.

62 **Storybook: frontend workshop for UI development**. Storybook. storybook.js.org, n.d..
storybook.js.org/

63 **Patterns**. GOV.UK Design System. Government Digital Service, n.d..
design-system.service.gov.uk/patterns/
Documented solutions to common user problems, such as asking for addresses or dates, and validation.

64 **Sharing state between components (controlled and uncontrolled components)**. React team. react.dev, n.d..
react.dev/learn/sharing-state-between-components

65 **Explore component properties**. Figma. Figma Help Center, n.d..
help.figma.com/hc/en-us/articles/5579474826519-Explore-component-properties
Variant, boolean, text and instance-swap properties.

66 **Guide to variables in Figma**. Figma. Figma Help Center, n.d..
help.figma.com/hc/en-us/articles/15339657135383-Guide-to-variables-in-Figma

67 **CSS logical properties and values**. MDN contributors. Mozilla, n.d..
developer.mozilla.org/en-US/docs/Web/CSS/CSS_logical_properties_and_values

68 **Web Components**. MDN contributors. Mozilla, n.d..
developer.mozilla.org/en-US/docs/Web/API/Web_components

69 **Error summary component**. GOV.UK Design System. Government Digital Service, n.d..
design-system.service.gov.uk/components/error-summary/

70 **Understanding Success Criterion 3.3.1: Error Identification**. W3C WAI. W3C, 2023.
www.w3.org/WAI/WCAG22/Understanding/error-identification.html

71 **Intl (internationalisation API)**. MDN contributors. Mozilla, n.d..
developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl
Built-in formatting for dates, numbers, currencies and plural rules by locale.

72 **Layout**. Apple. Human Interface Guidelines, n.d..
developer.apple.com/design/human-interface-guidelines/layout

73 **Style Dictionary**. Style Dictionary contributors (originally Amazon). styledictionary.com, n.d..
styledictionary.com/
An open-source build system that turns design tokens into code for any platform.

BACK MATTER

About Layer One and the author

Layer One is a design systems studio in Bangalore, India. We set up the foundations, scale them across products and markets, and put in the governance that keeps teams using them.

Aniket Pandey

Aniket leads Layer One. His design systems work spans three very different kinds of organisation:

- **JPMorgan Chase**, Senior Designer, Design Systems (2025–present): company-wide design patterns, an AI-assisted workflow for drafting them, and tooling that checks design files and code against the system, used by 500+ designers and 1,000+ engineers.
- **CARS24**, Design Systems Lead (2024–25): the design system for 70+ designers and 200+ engineers, a new palette and token architecture for the rebrand, governance, and a pattern library, putting 10+ products in three countries on one token system.
- **IBM**, UX Designer, design systems (2021–24): design systems for IBM clients including Cardinal Health and Riyadh Air, from foundations and tokens to governance, contribution and releases.

Source: layerone.co.in.[1]

The rules we work by

01	**A design system is an agreement, not a library**
02	**Audit before you build**
03	**Name things so people can guess them**
04	**Tokens first, components second**
05	**Accessible by default, not by audit**
06	**Every decision comes with a reason**
07	**Build where the system lives: your files, your repo**
08	**Measure time saved, not usage**
09	**Docs decide adoption**
10	**Leave the team able to run it without us**

WHERE TO START

Start with a two-week audit.

We map every style, component and token you have in Figma and code, check accessibility, and give you a ranked plan for what to fix first. Fixed scope, fixed price, and credited in full if you go on to a Foundations build.

→ layerone.co.in
→ layerone@proton.me
→ Bangalore, India · working worldwide

Where to go from here

START THIS WEEK

Run an inventory

Screenshot your buttons, fields and messages across your top screens (Chapter 13). It's the fastest way to see your own starting point.

START THIS MONTH

Name your basics

Colours by job, a type scale, a spacing scale, as tokens shared by design and code (Part 02).

START THIS QUARTER

Run a pilot

Build your first ten components inside a real product project, with the rhythms in Chapter 27 in the calendar.